



Real-Time Procedural Generation with GPU Work Graphs

BASTIAN KUTH, MAX OBERBERGER, CARSTEN FABER, DOMINIK BAUMEISTER,
MATTHÄUS CHAJDAS, QUIRIN MEYER

GPU Work Graphs

GPU Work Graphs

```
[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeMaxDispatchGrid(...)]
[NumThreads(32, 1, 1)]
void WorkCreator(
    ...
    [MaxRecords(32)]
    NodeOutput<Job> Worker
){
    ...
    GroupNodeOutputRecords<Job> outputRecord =
    Worker.GetGroupNodeOutputRecords(32);
    outputRecord.Get(gtid) = ...;
    outputRecord.OutputComplete();
    ...
}
```

GPU Work Graphs

```
[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeMaxDispatchGrid(...)]
[NumThreads(32, 1, 1)]
void WorkCreator(
    ...
    [MaxRecords(32)]
    NodeOutput<Job> Worker
){
    ...
    GroupNodeOutputRecords<Job> outputRecord =
    Worker.GetGroupNodeOutputRecords(32);
    outputRecord.Get(gtid) = ...;
    outputRecord.OutputComplete();
    ...
}
```

```
[Shader("node")]
[NodeLaunch("thread")]
void Worker
    ThreadNodeInputRecord<Job> input
){
    Job job = input.Get();
    ...
}
```


GPU Work Graphs

```
[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeMaxDispatchGrid(...)]
[NumThreads(32, 1, 1)]
void WorkCreator(
    ...
    [MaxRecords(32)]
    NodeOutput<Job> Worker
){
    ...
    GroupNodeOutputRecords<Job> outputRecord =
    Worker.GetGroupNodeOutputRecords(32);
    outputRecord.Get(gtid) = ...;
    outputRecord.OutputComplete();
    ...
}
```

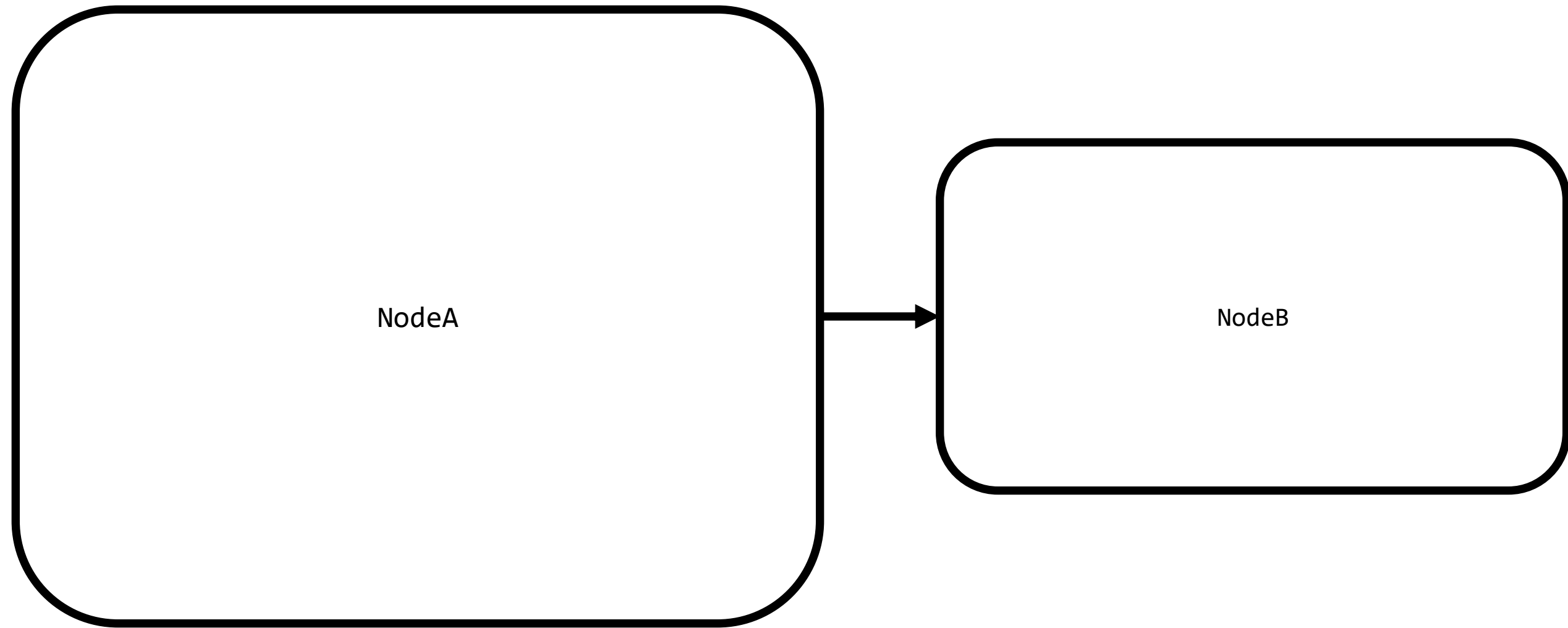
```
[Shader("node")]
[NodeLaunch("thread")]
void Worker(
    ThreadNodeInputRecord<Job> input
){
    Job job = input.Get();
    ...
}
```

GPU Work Graphs

```
[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeMaxDispatchGrid(...)]
[NumThreads(32, 1, 1)]
void WorkCreator(
    ...
    [MaxRecords(32)]
    NodeOutput<Job> Worker
){
    ...
    GroupNodeOutputRecords<Job> outputRecord =
    Worker.GetGroupNodeOutputRecords(32);
    outputRecord.Get(gtid) = ...;
    outputRecord.OutputComplete();
    ...
}
```

```
[Shader("node")]
[NodeLaunch("thread")]
void Worker(
    ThreadNodeInputRecord<Job> input
){
    Job job = input.Get();
    ...
}
```

GPU Work Graphs



GPU Work Graphs

1:30pm – 2:45pm



Papers Session 6:

Compression, Micromaps & Procedural Geometry

“Succinct Opacity Micromaps”

— Gustaf Waldemarson, Michael Doggett

“DGF: A Dense, Hardware Friendly Geometry Format for Lossy Compressing Meshlets with Arbitrary Topologies”

— Joshua Barczak, Carsten Benthin, David McAllister

“Real-Time Procedural Generation with GPU Work Graphs”

— Bastian Kuth, Max Oberberger, Carsten Faber,
Dominik Baumeister, Matthäus Chajdas, Quirin Meyer

2:45pm – 3:15pm

Coffee Break

3:15pm – 4:15pm

Hot3D Session 2: Standards

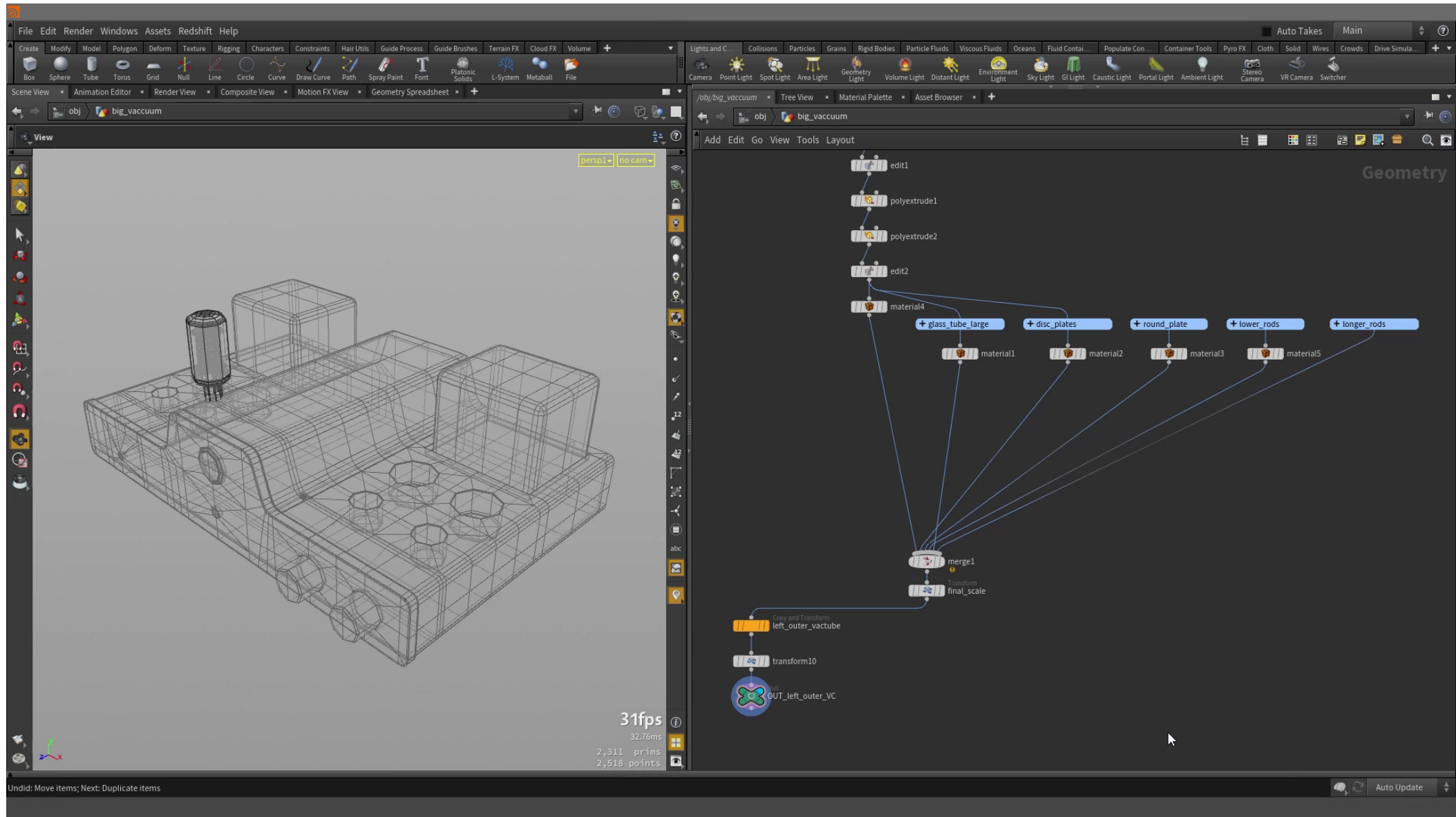
“Work Graphs: Hands-On with the Future of Graphics Programming”

— Max Oberberger, AMD

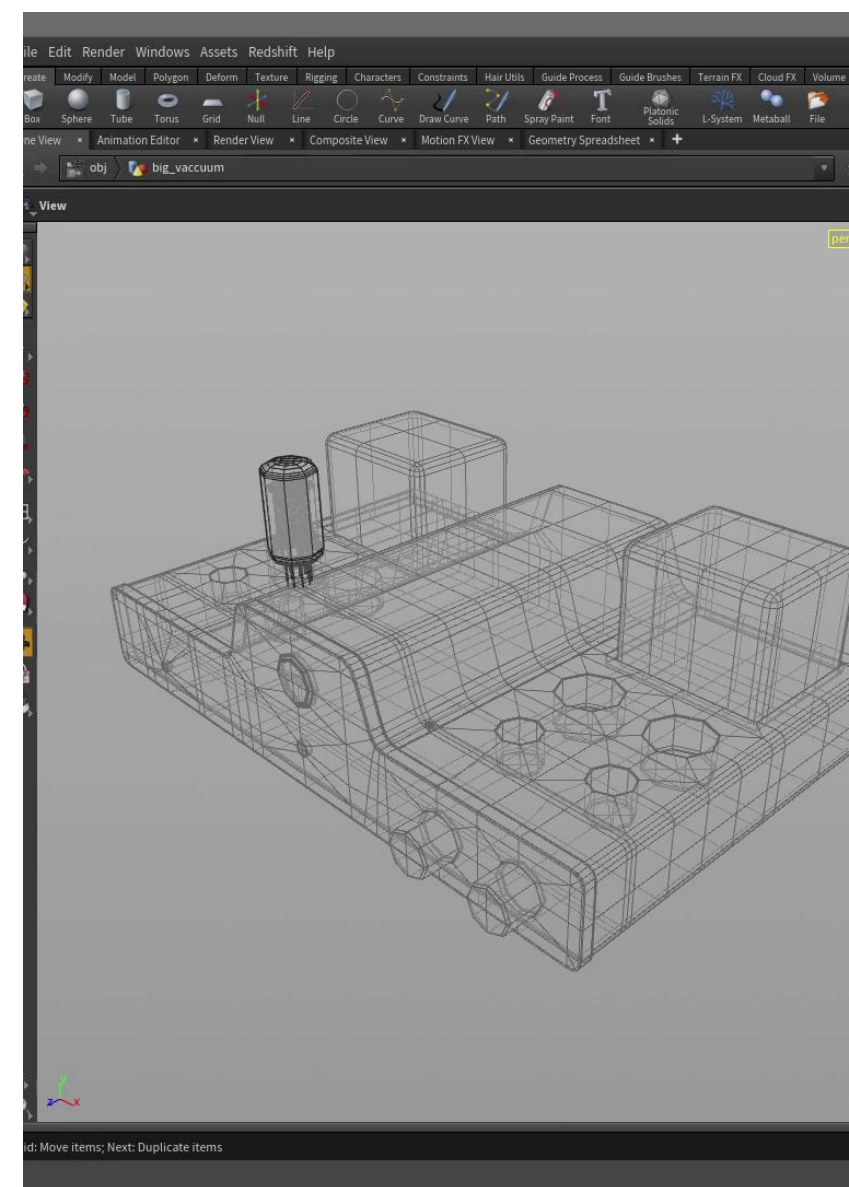
“Portable and Scalable 3D Rendering Using ANARI”

— Jefferson Amstutz, NVIDIA

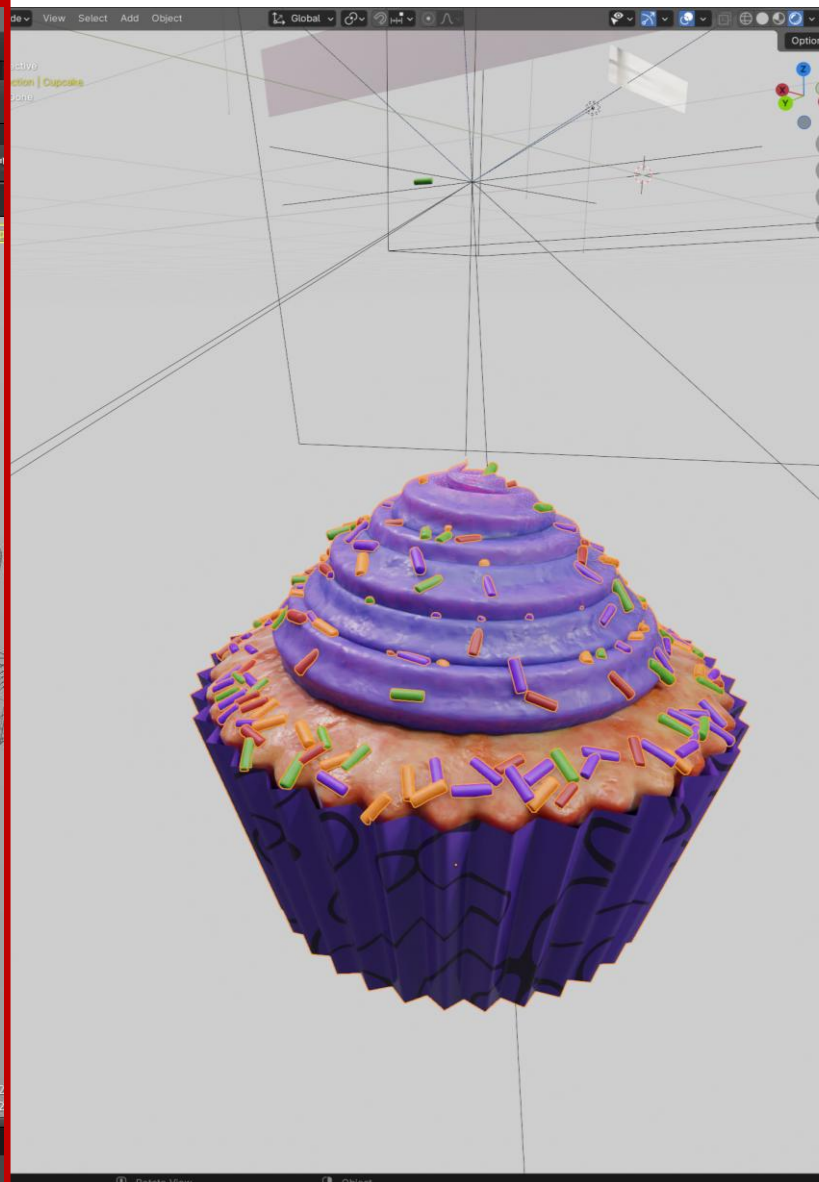
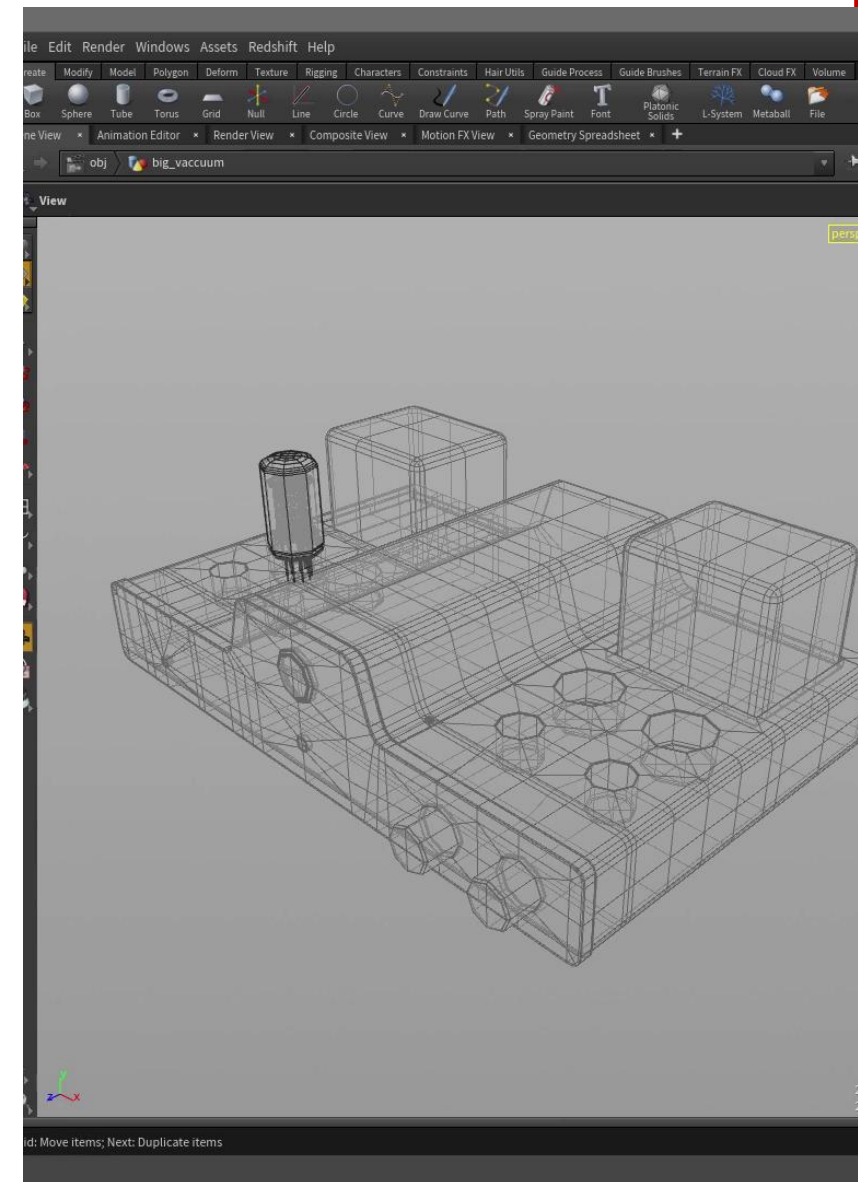
HoudiniTM



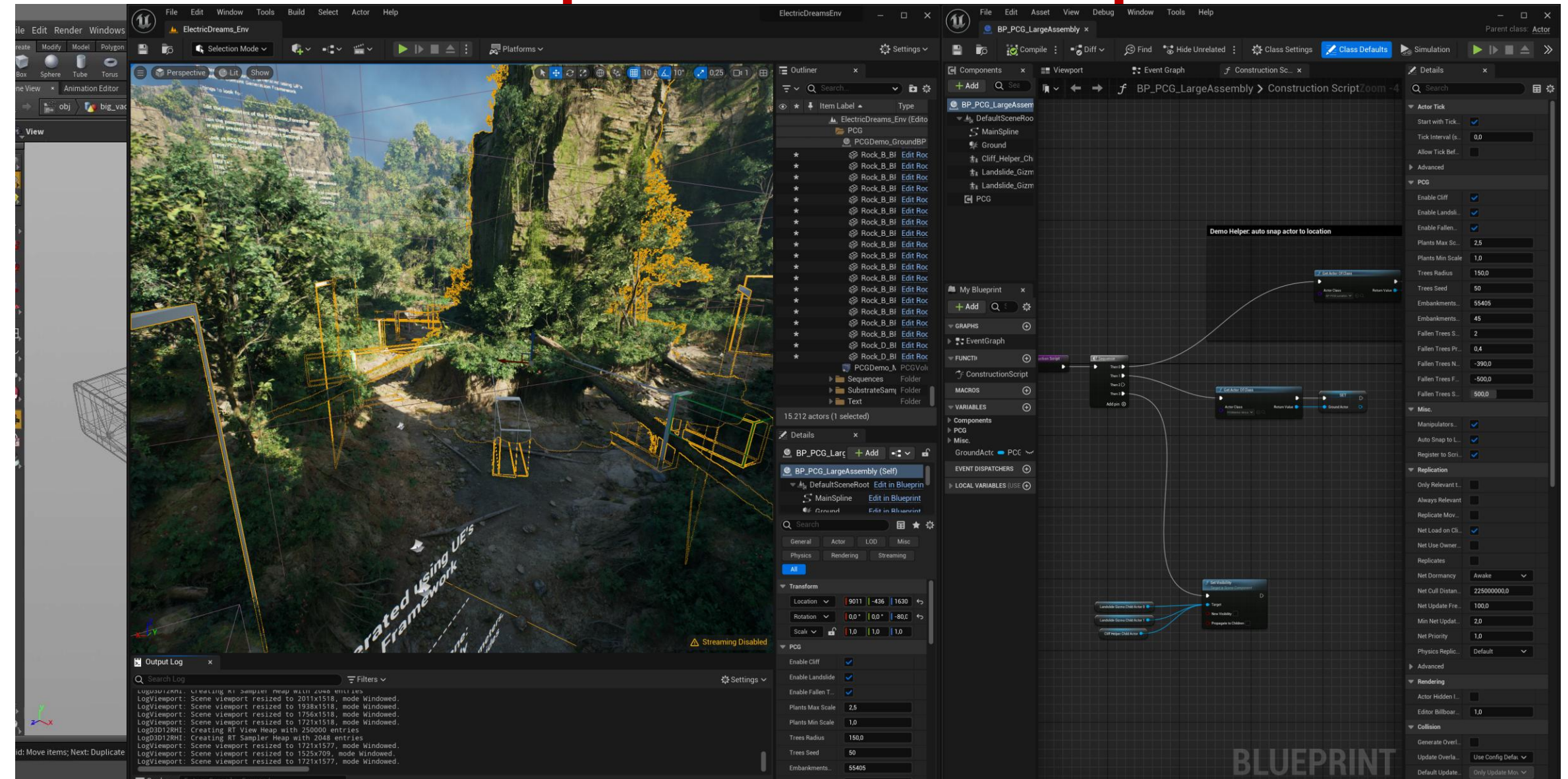
HoudiniTM



HoudiniTM



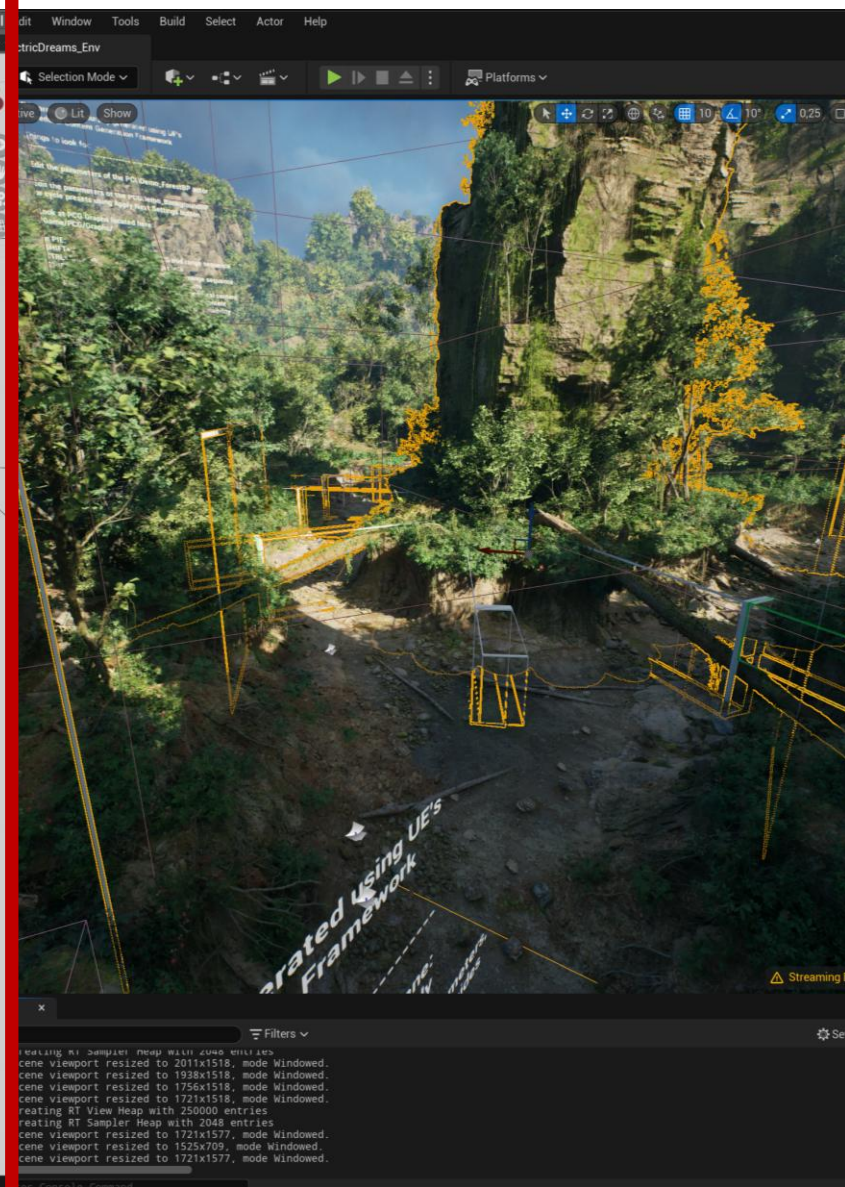
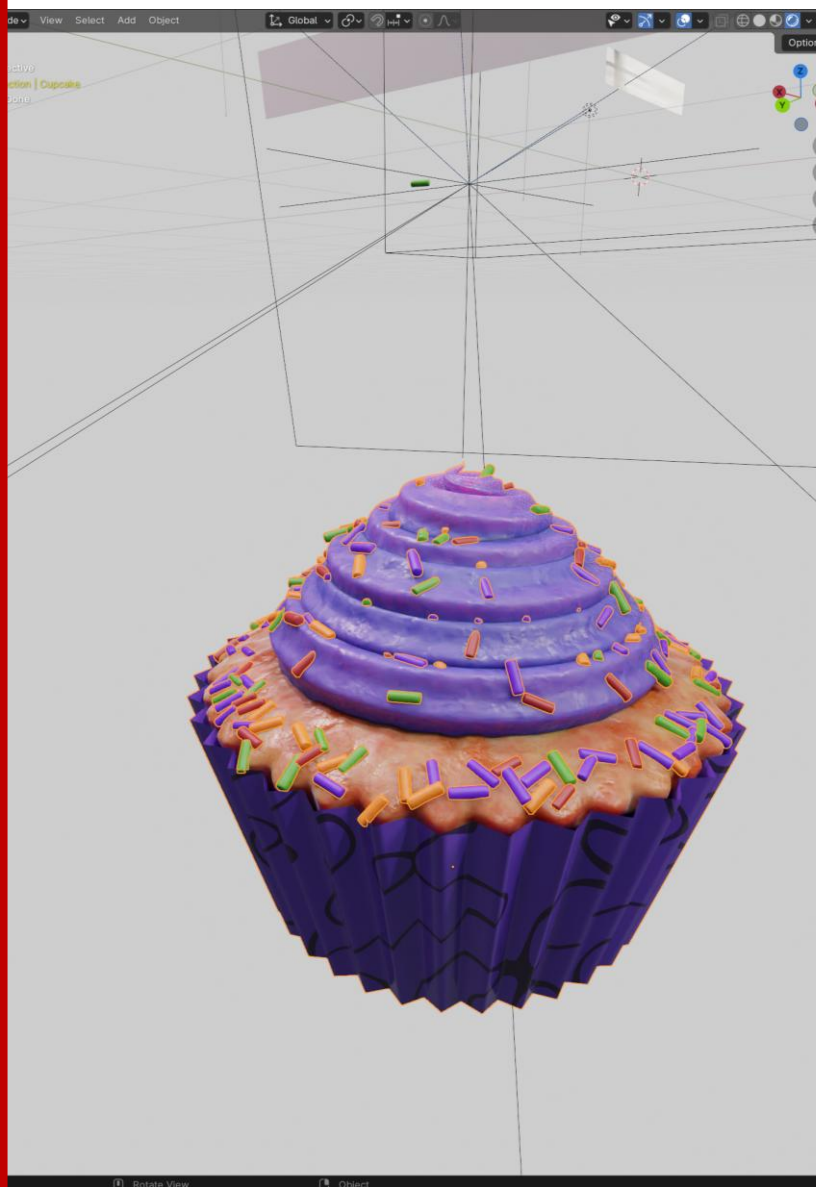
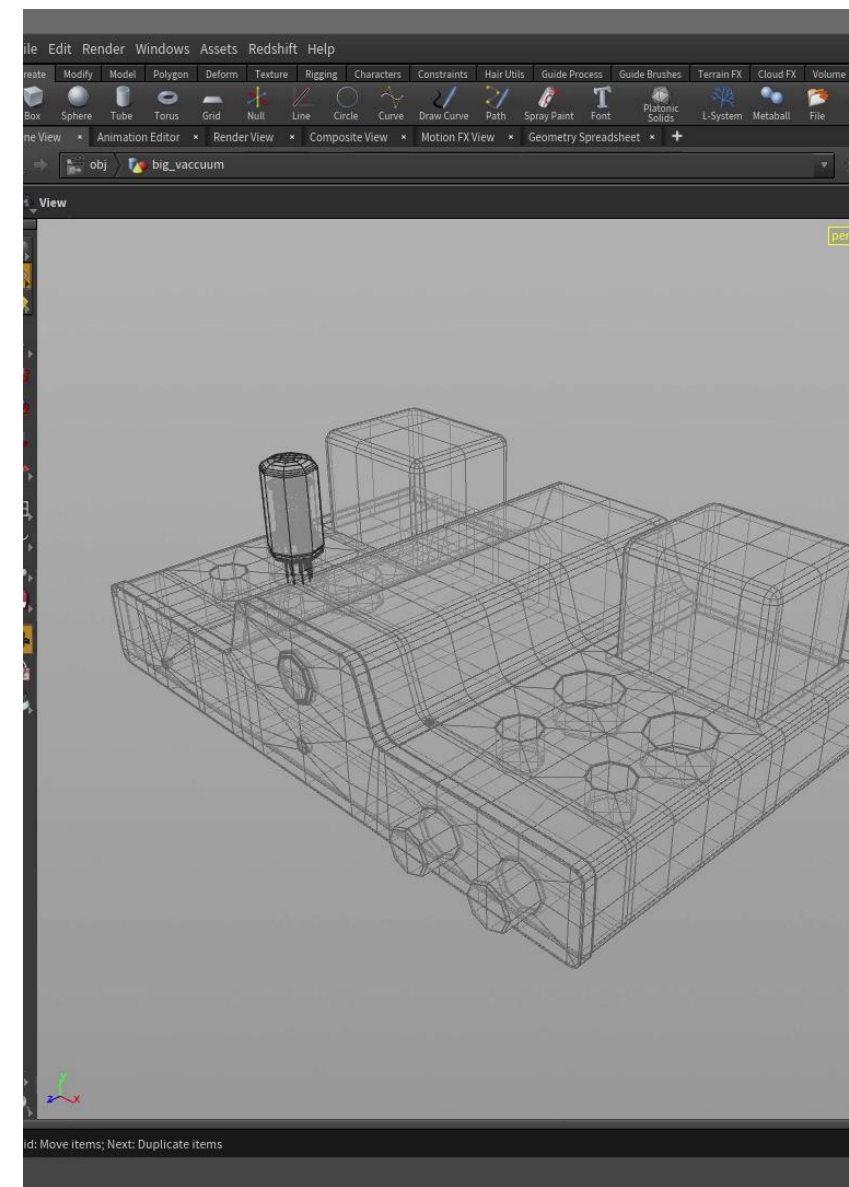
HoudiniTM

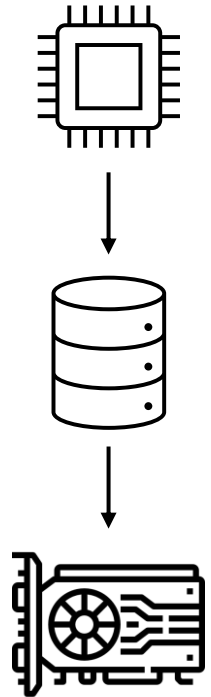


Houdini™

blender®

UNREAL ENGINE

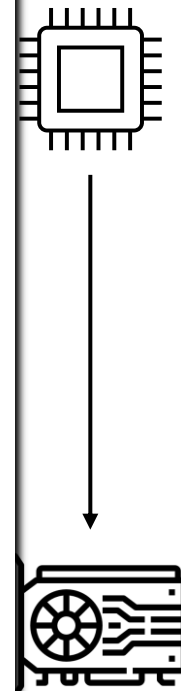




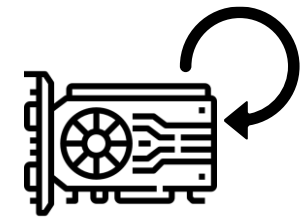


UNREAL ENGINE

Requirements for our system:



Ours

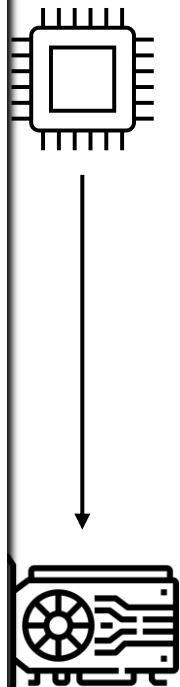




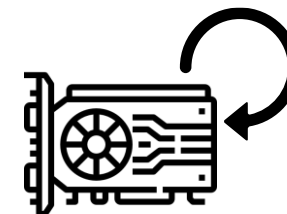
UNREAL ENGINE

Requirements for our system:

☐ GPU only

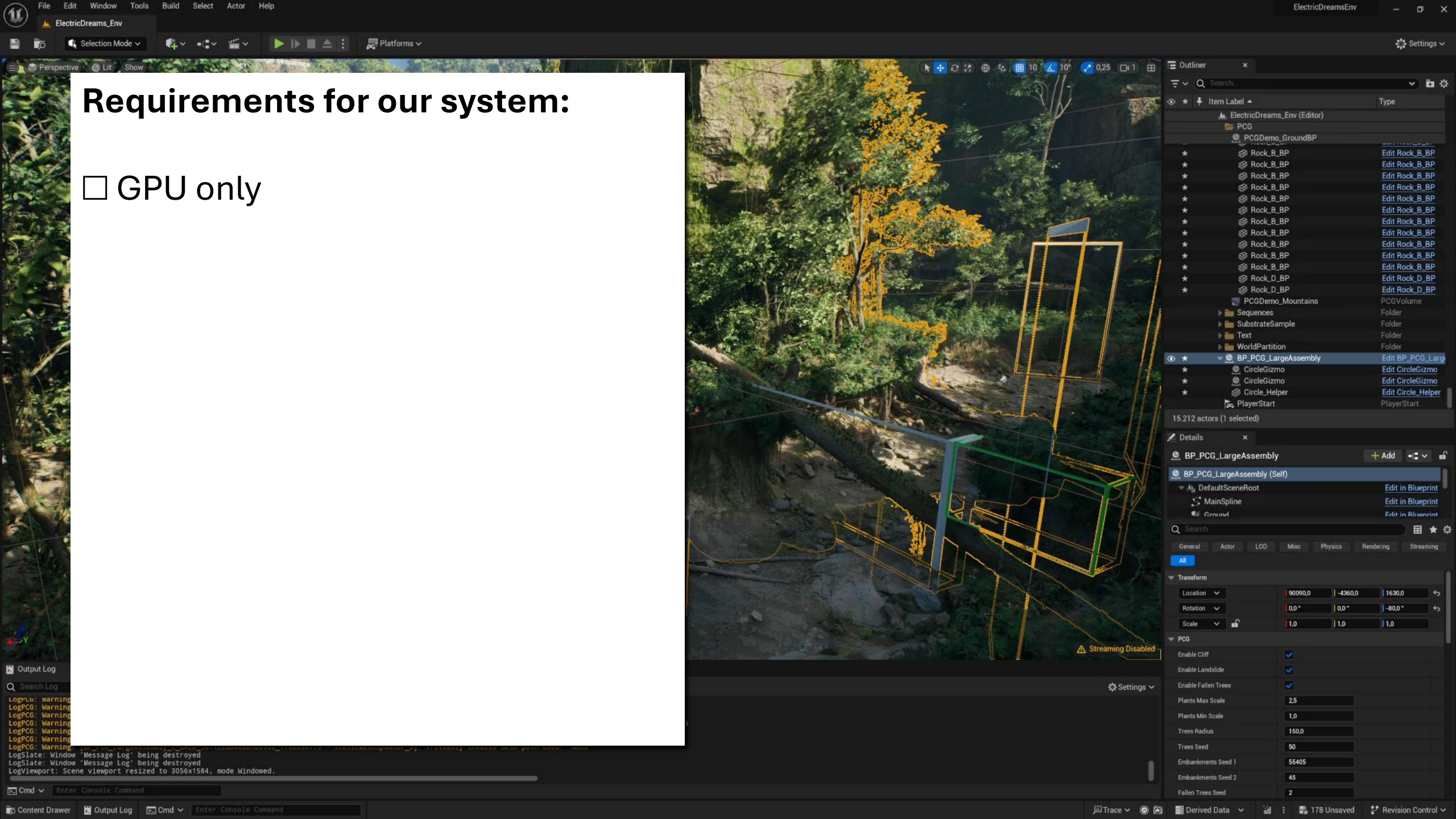


Ours



Requirements for our system:

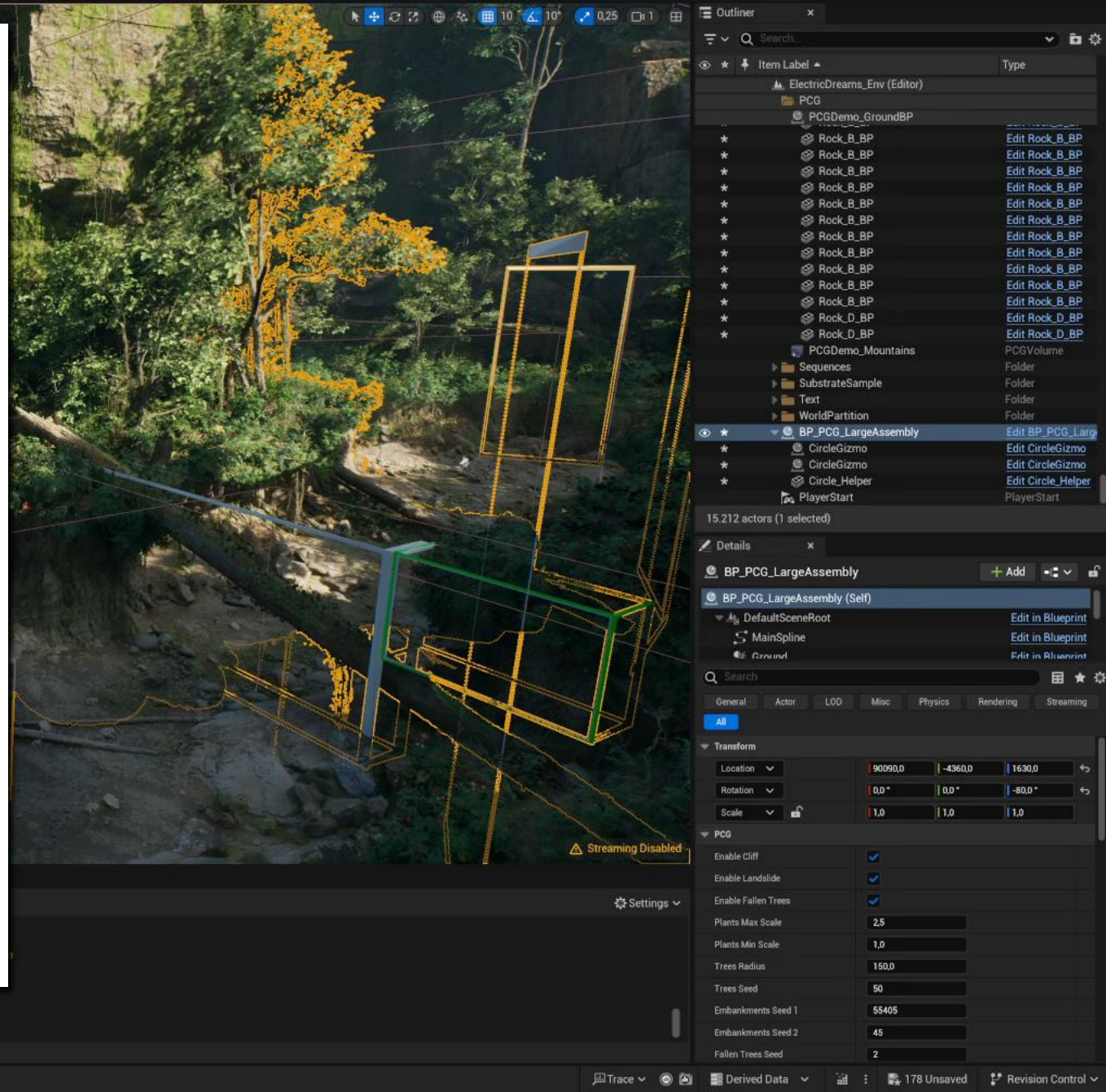
☐ GPU only



Requirements for our system:

☐ GPU only

☐ instant updates when editing



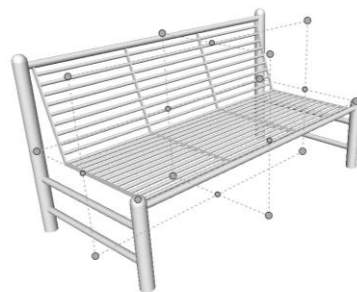
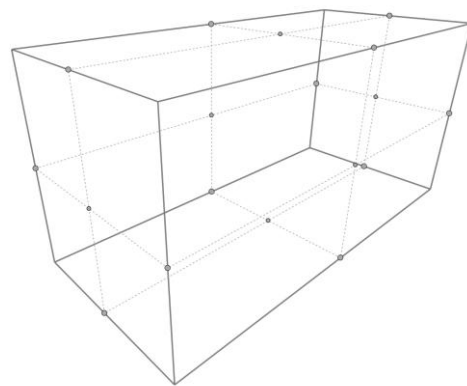
Radius 1.18

Height 1.04

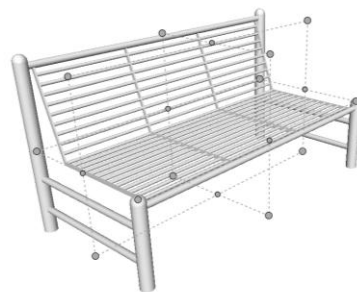
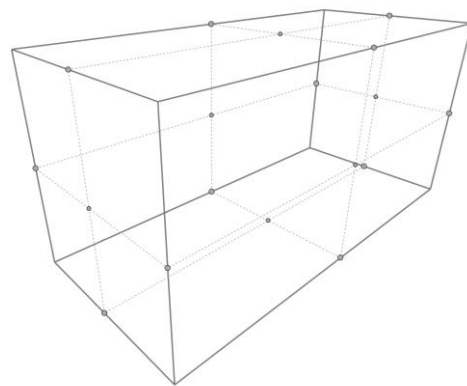
Folds 70



Radius 1.18
Height 1.04
Folds 70

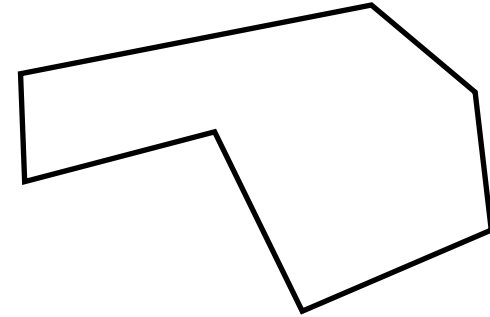
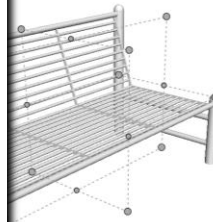
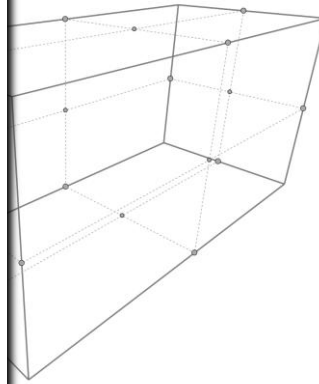


Radius 1.18
Height 1.04
Folds 70



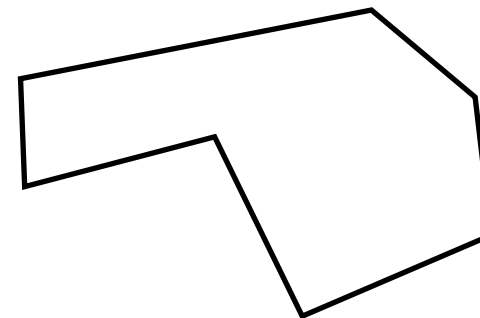
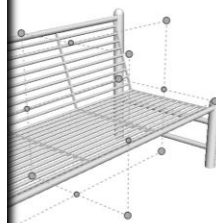
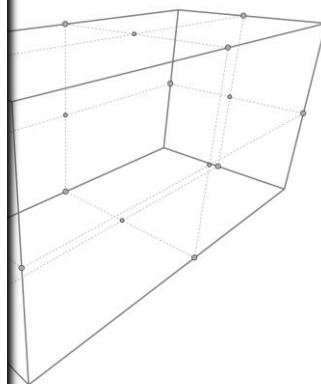
Requirements for our system:

- ☐ GPU only
- ☐ instant updates when editing



Requirements for our system:

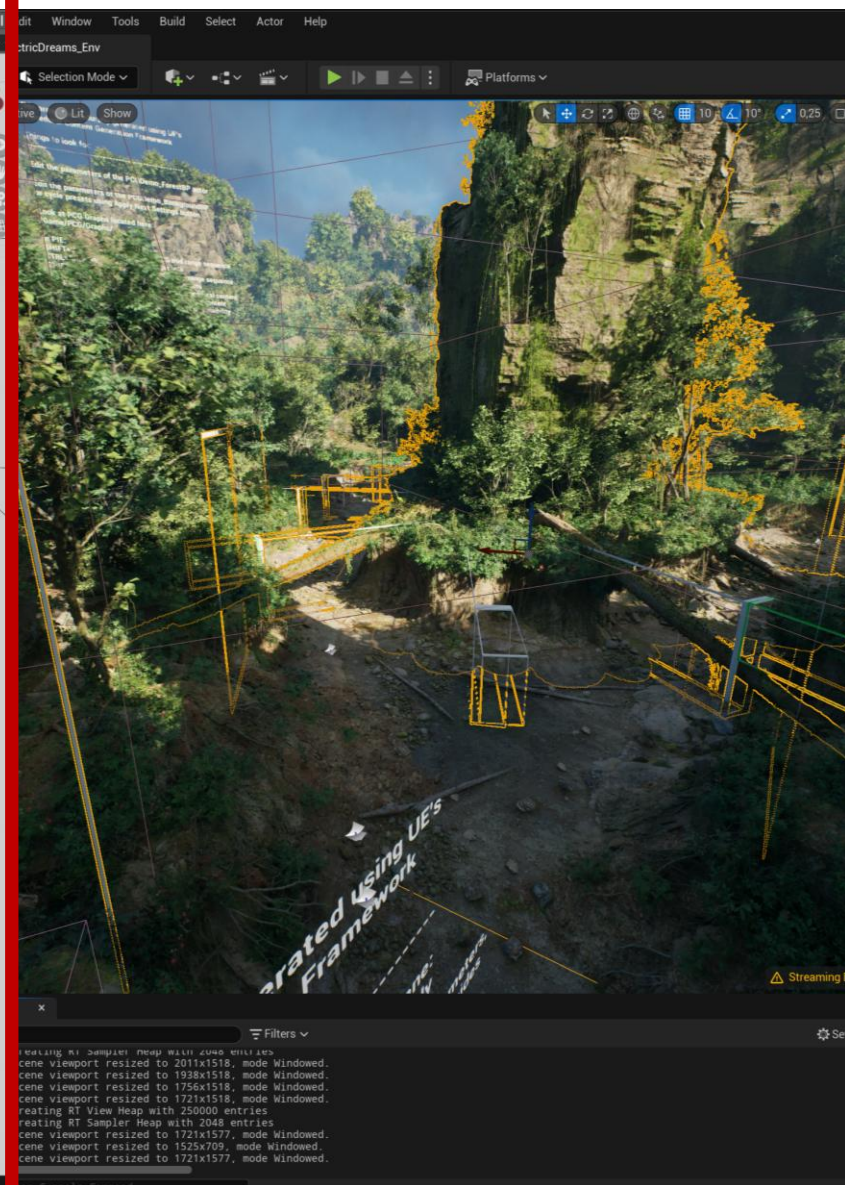
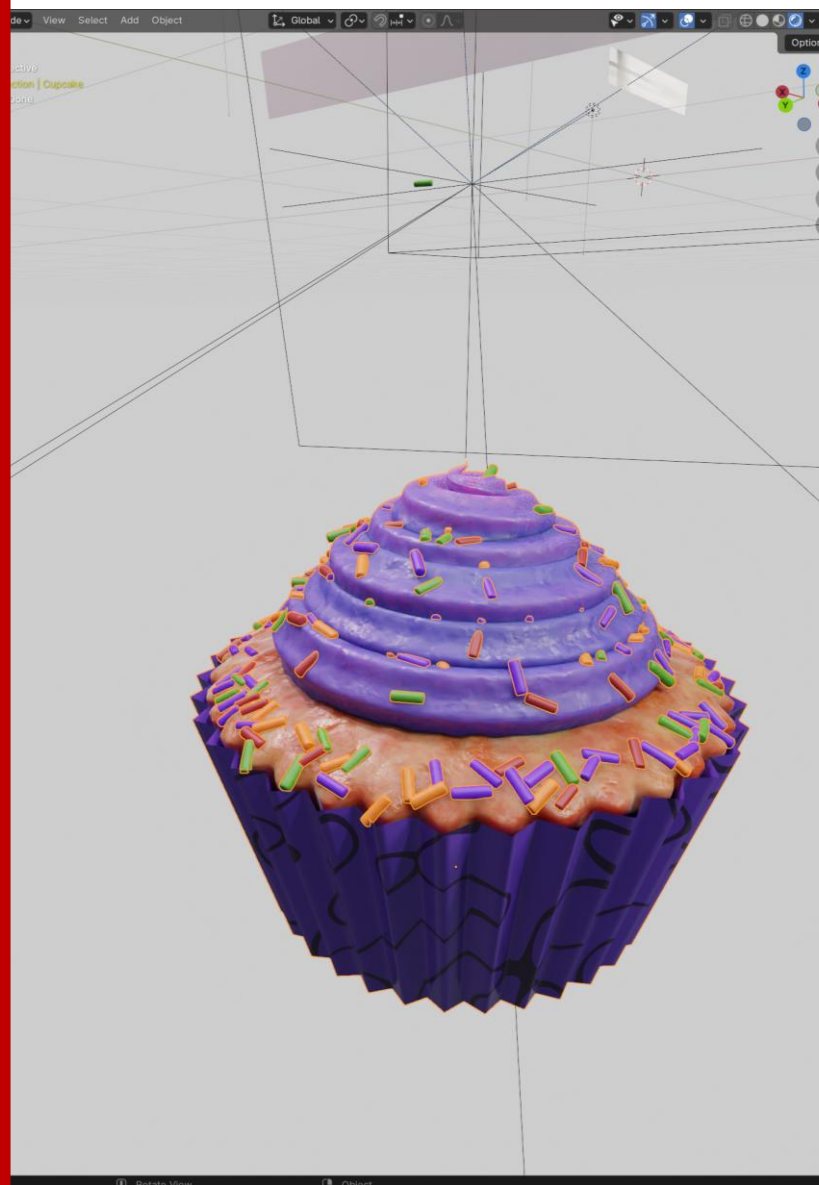
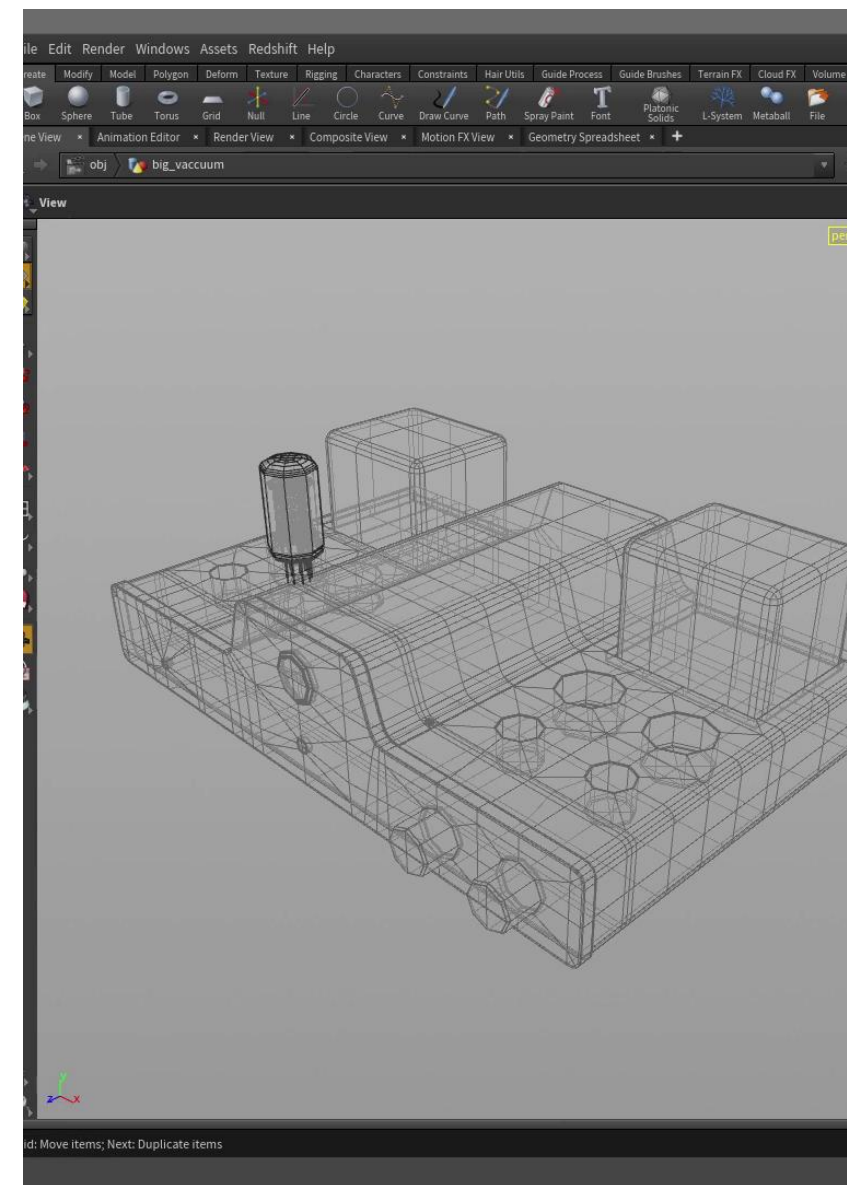
- ☐ GPU only
- ☐ instant updates when editing
- ☐ processes control parameters



HoudiniTM

blender[®]

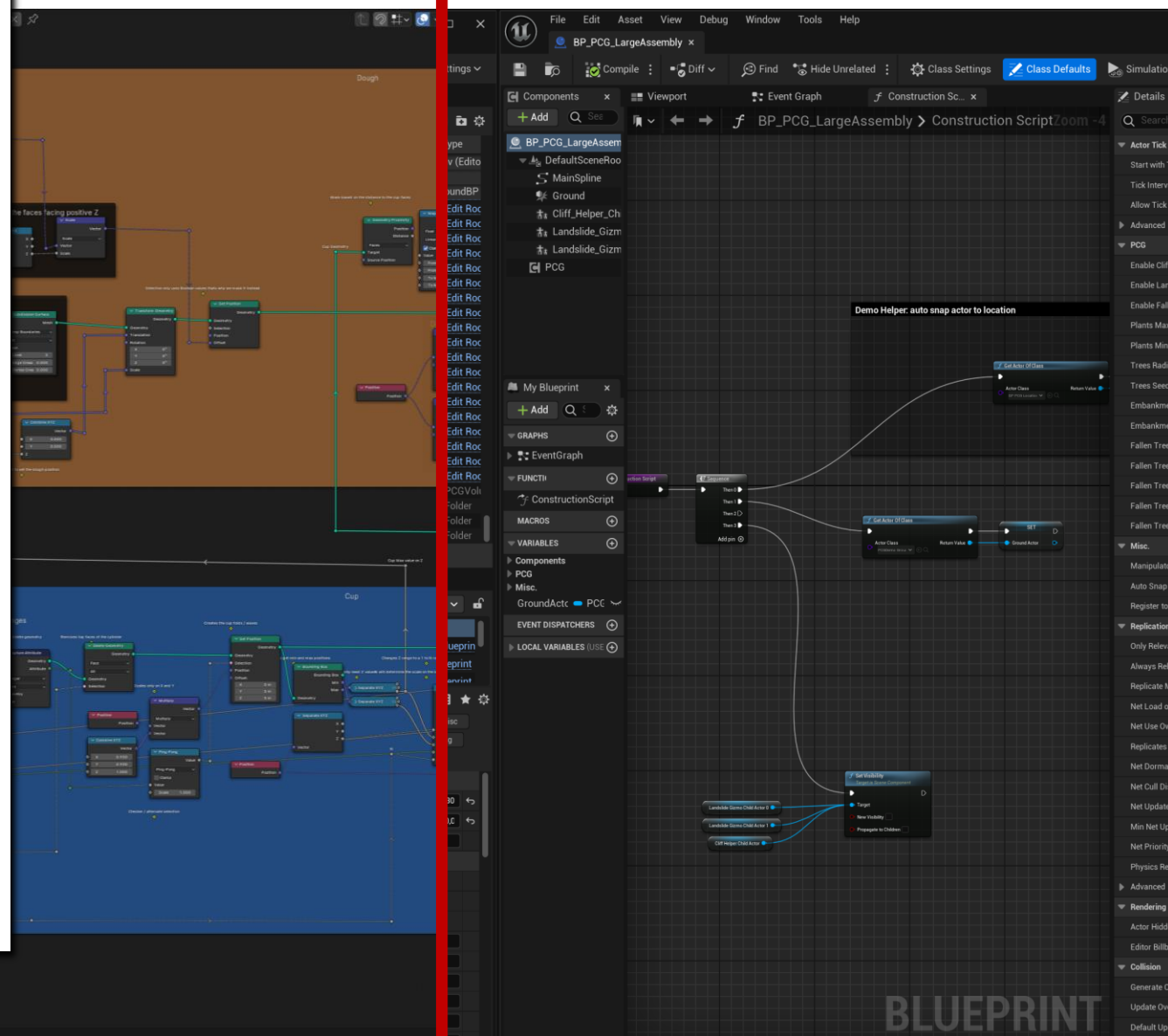
UNREAL ENGINE





Requirements for our system:

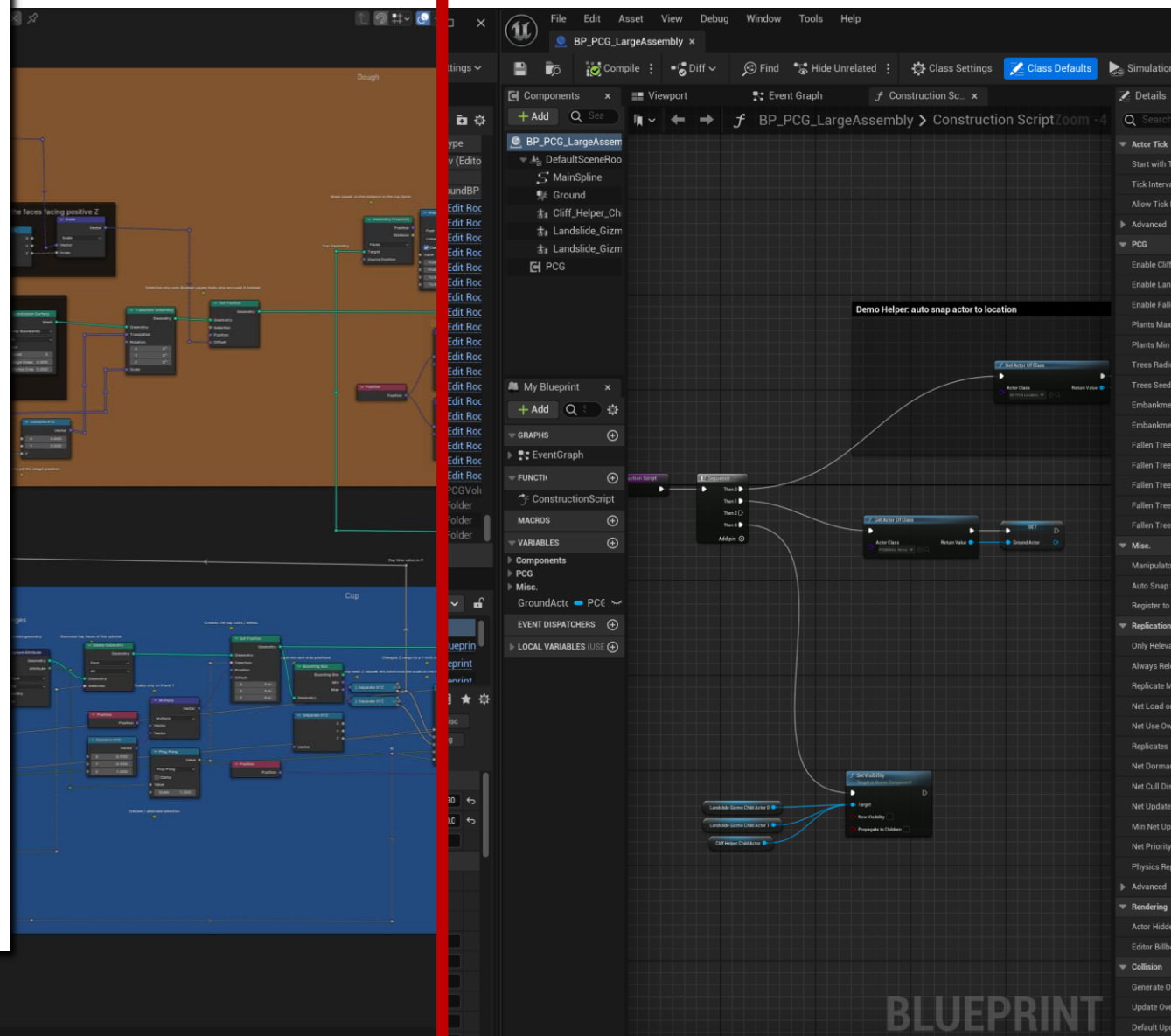
- ☐ GPU only
- ☐ instant updates when editing
- ☐ processes control parameters



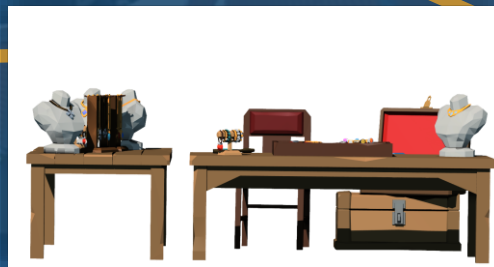
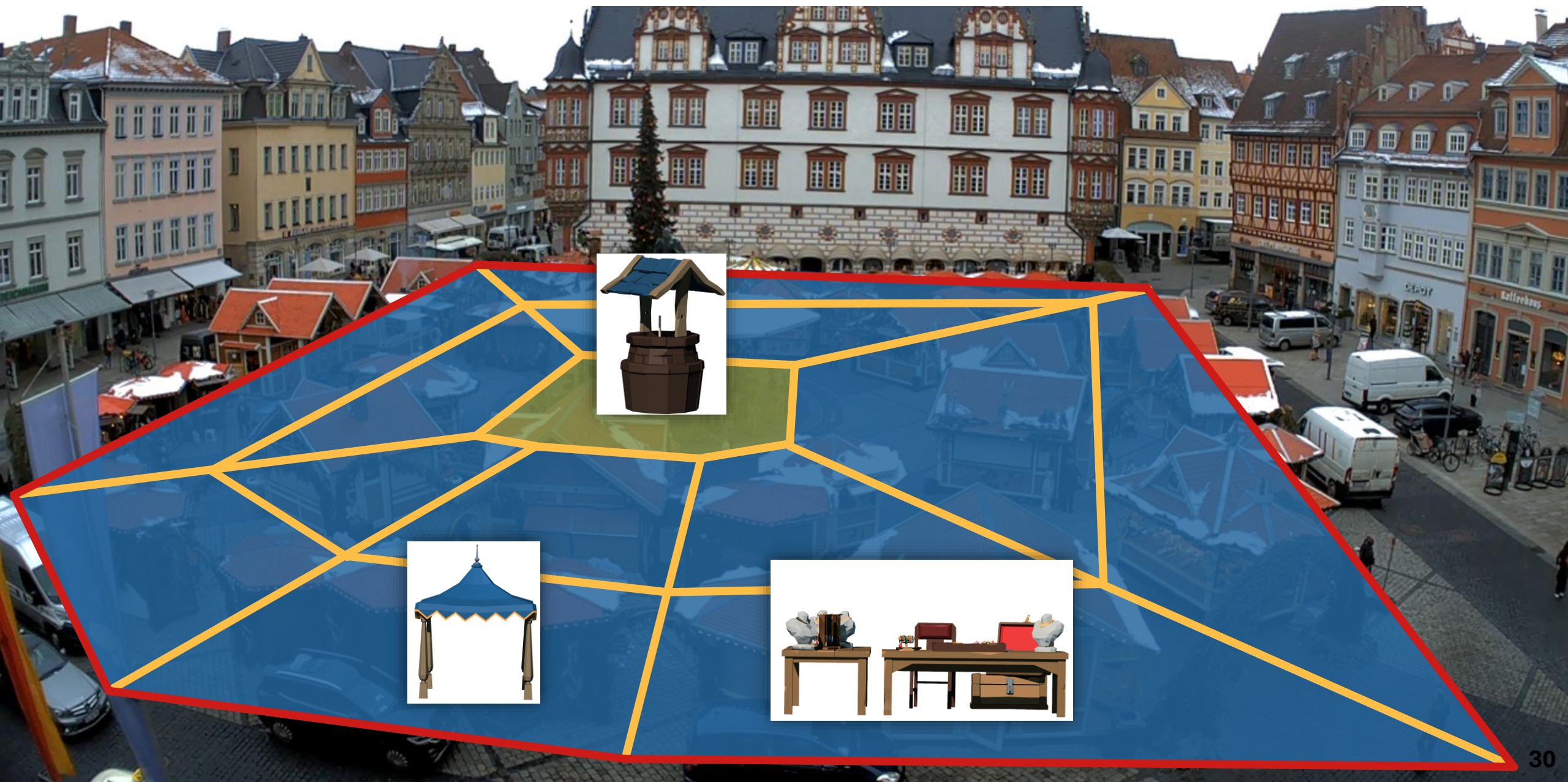


Requirements for our system:

- ☐ GPU only
- ☐ instant updates when editing
- ☐ processes control parameters
- ☐ hierarchy of reusable nodes







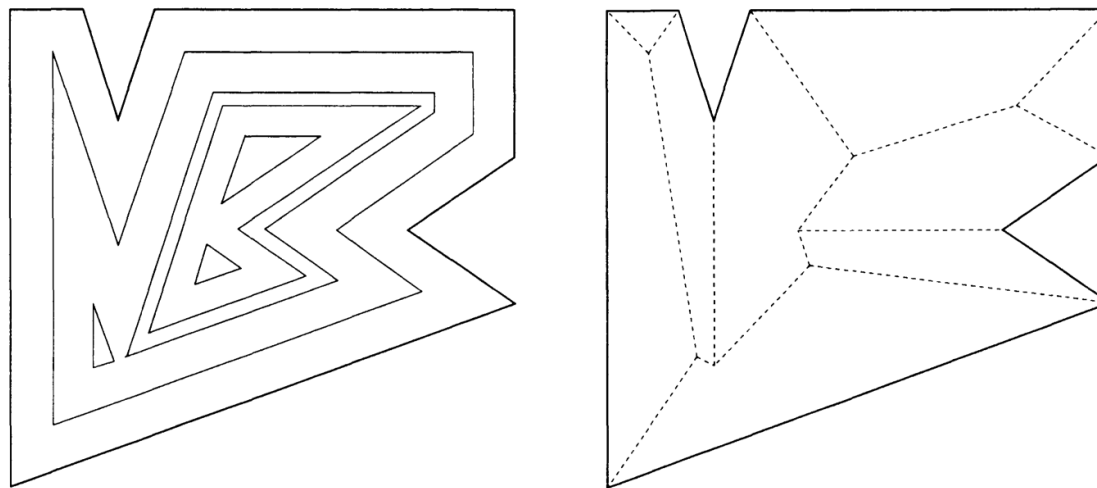


Figure 1: (a) Polygon hierarchy and (b) straight skeleton

Journal of Universal Computer Science, vol. 1, no. 12 (1995), 752-761
 submitted: 17/11/95, accepted: 21/12/95, appeared: 28/12/95 © Springer Pub. Co.

A Novel Type of Skeleton for Polygons

Oswin Aichholzer
 Franz Aurenhammer
 Institute for Theoretical Computer Science
 Graz University of Technology
 Klosterwiesgasse 32/2, A-8010 Graz, Austria
 {oaich,auren}@igi.tu-graz.ac.at

David Alberts
 Bernd Gärtner
 Institut für Informatik
 Freie Universität Berlin
 Takustraße 9, D-14195 Berlin, Germany
 {alberts,gaertner}@inf.fu-berlin.de

Abstract A new internal structure for simple polygons, the straight skeleton, is introduced and discussed. It is composed of pieces of angular bisectors which partition the interior of a given n -gon P in a tree-like fashion into n monotone polygons. Its straight-line structure and its lower combinatorial complexity may make the straight skeleton preferable to the widely used medial axis of a polygon. As a seemingly unrelated application, the straight skeleton provides a canonical way of constructing a polygonal roof above a general layout of ground walls.

Keywords: Simple polygon, angular bisectors, internal skeleton, roof construction

1 Introduction and basic properties

The purpose of this paper is to introduce and discuss a new and interesting internal structure for simple polygons in the plane. The new structure, called the *straight skeleton*, is solely made up of straight line segments which are pieces of angular bisectors of polygon edges. It uniquely partitions the interior of a given n -gon P into n monotone polygons, one for each edge of P .

The straight skeleton, in general, differs from the well-known *medial axis* of P which consists of all interior points whose closest point on P 's boundary is not unique; see e.g. Lee [L]. If P is convex then both structures are identical. Otherwise, the medial axis contains parabolically curved segments in the neighborhood of reflex vertices of P which are avoided by the straight skeleton. If P is rectilinear then the straight skeleton is the medial axis of P for the L_∞ -metric. Skeletons have numerous applications inside and outside computer science as is documented e.g. in Kirkpatrick [K].

While the medial axis is a Voronoi-diagram-like concept, the straight skeleton is not defined using a distance function but rather by an appropriate *shrinking process* for P . Imagine that the boundary of P is contracted towards P 's interior, in a self-parallel manner and at the same speed for all edges. Lengths of edges might decrease or increase in this process. Each vertex of P moves along the angular bisector of its incident edges. This situation continues as long as the boundary does not change topologically. There are two possible types of changes:

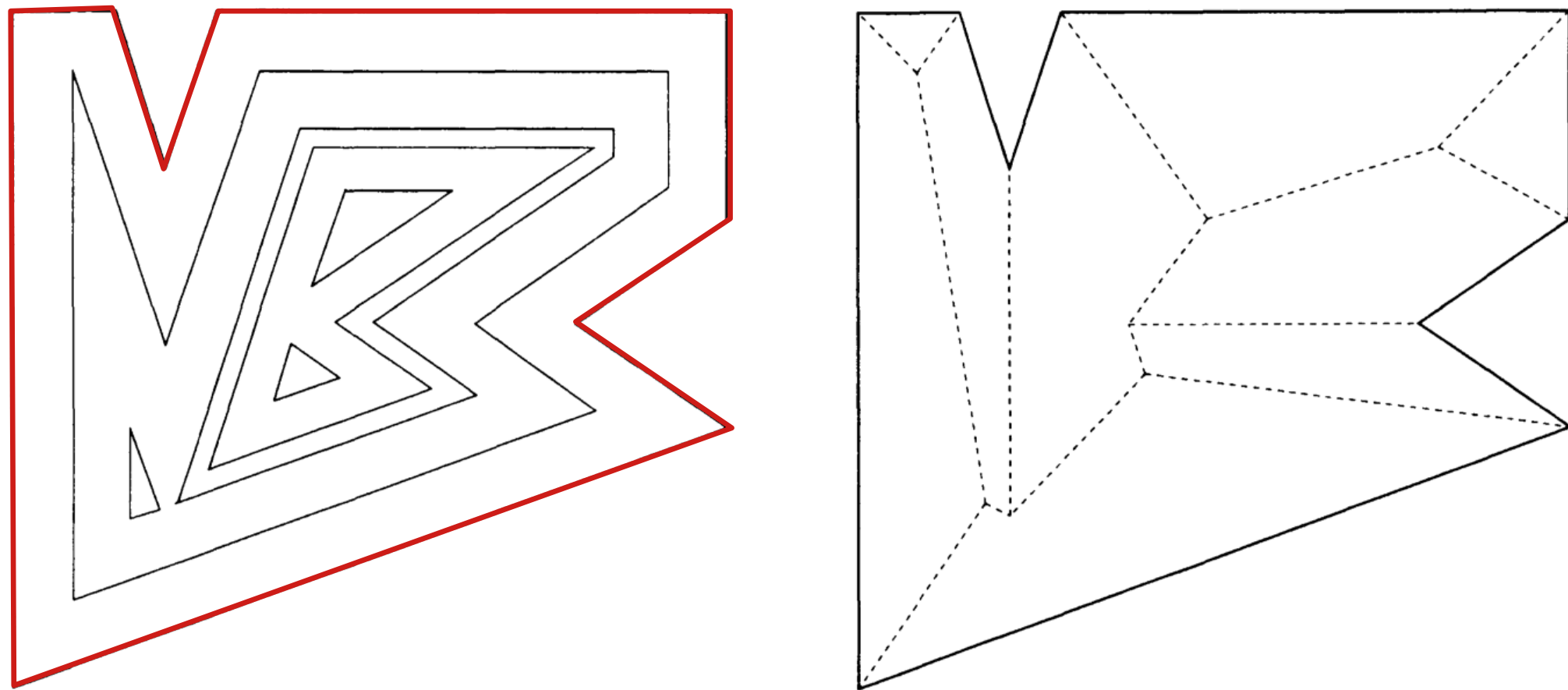


Figure 1: (a) Polygon hierarchy and (b) straight skeleton

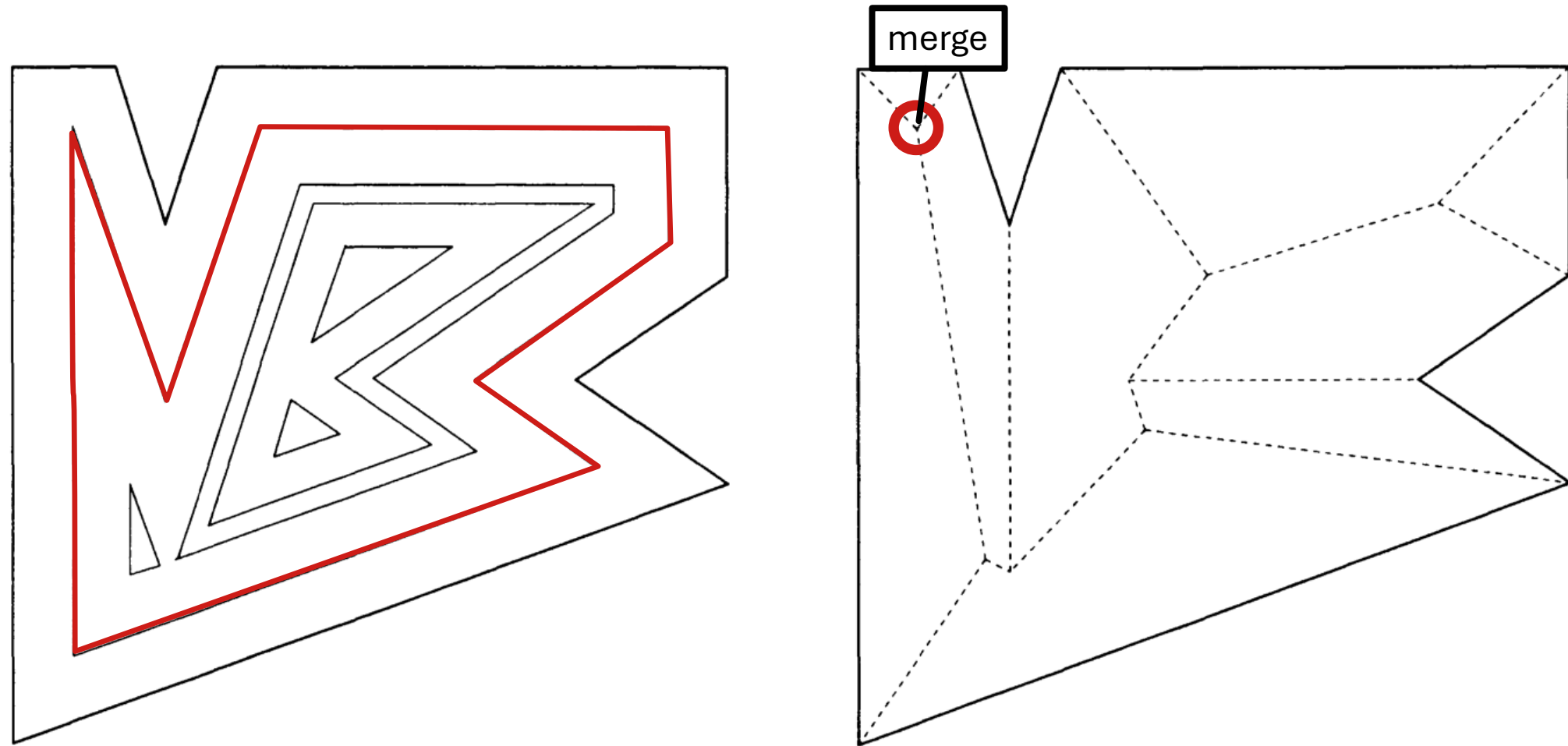


Figure 1: (a) Polygon hierarchy and (b) straight skeleton

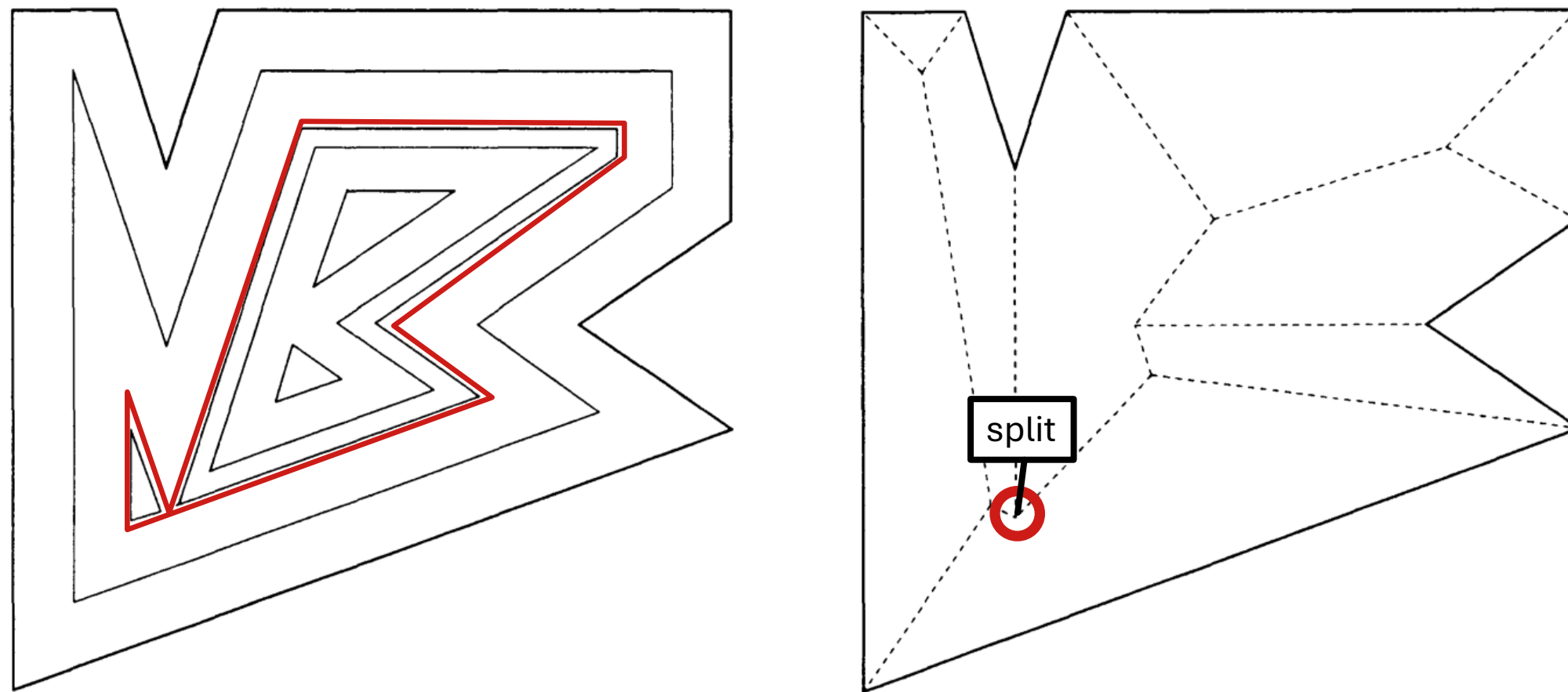
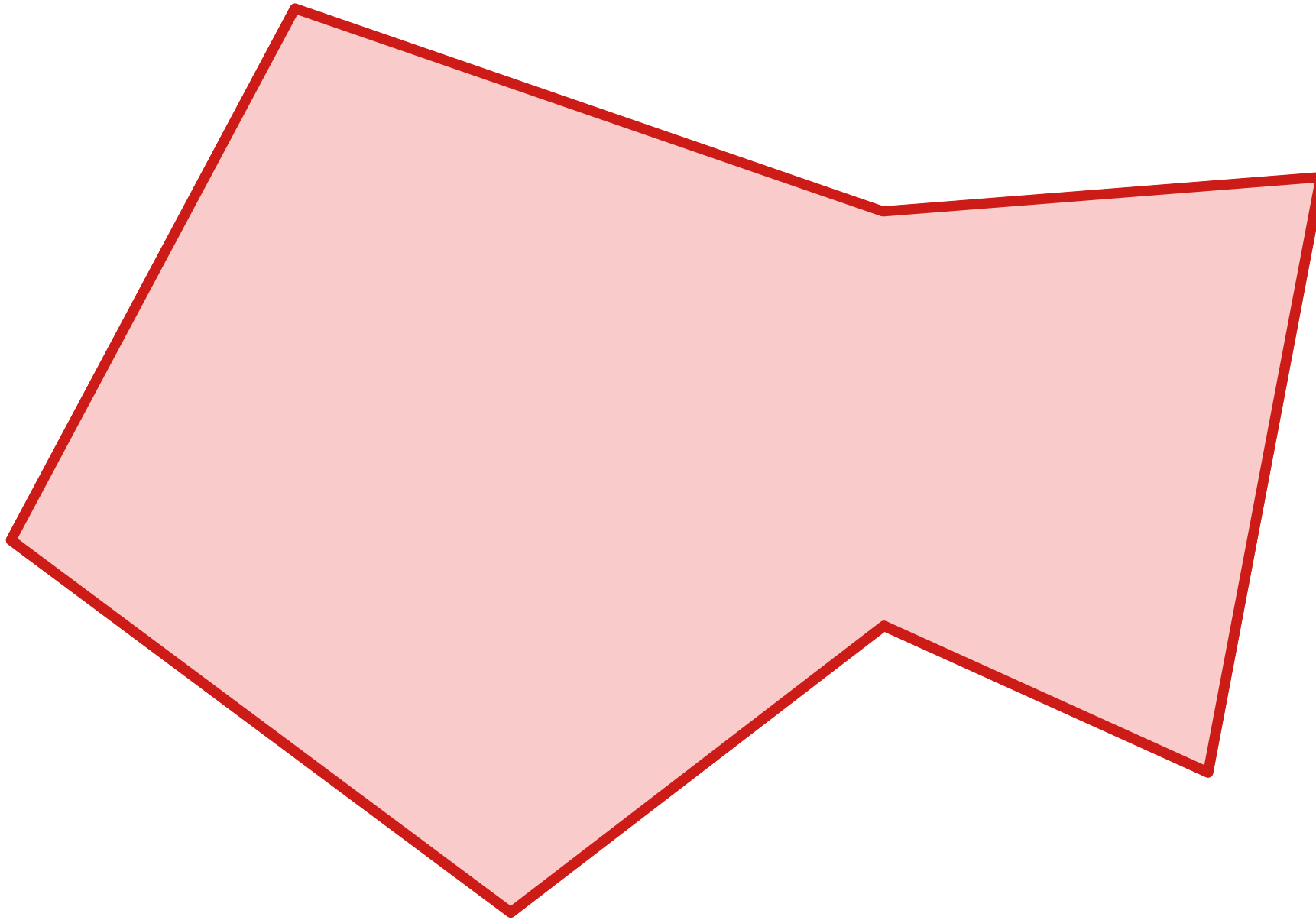
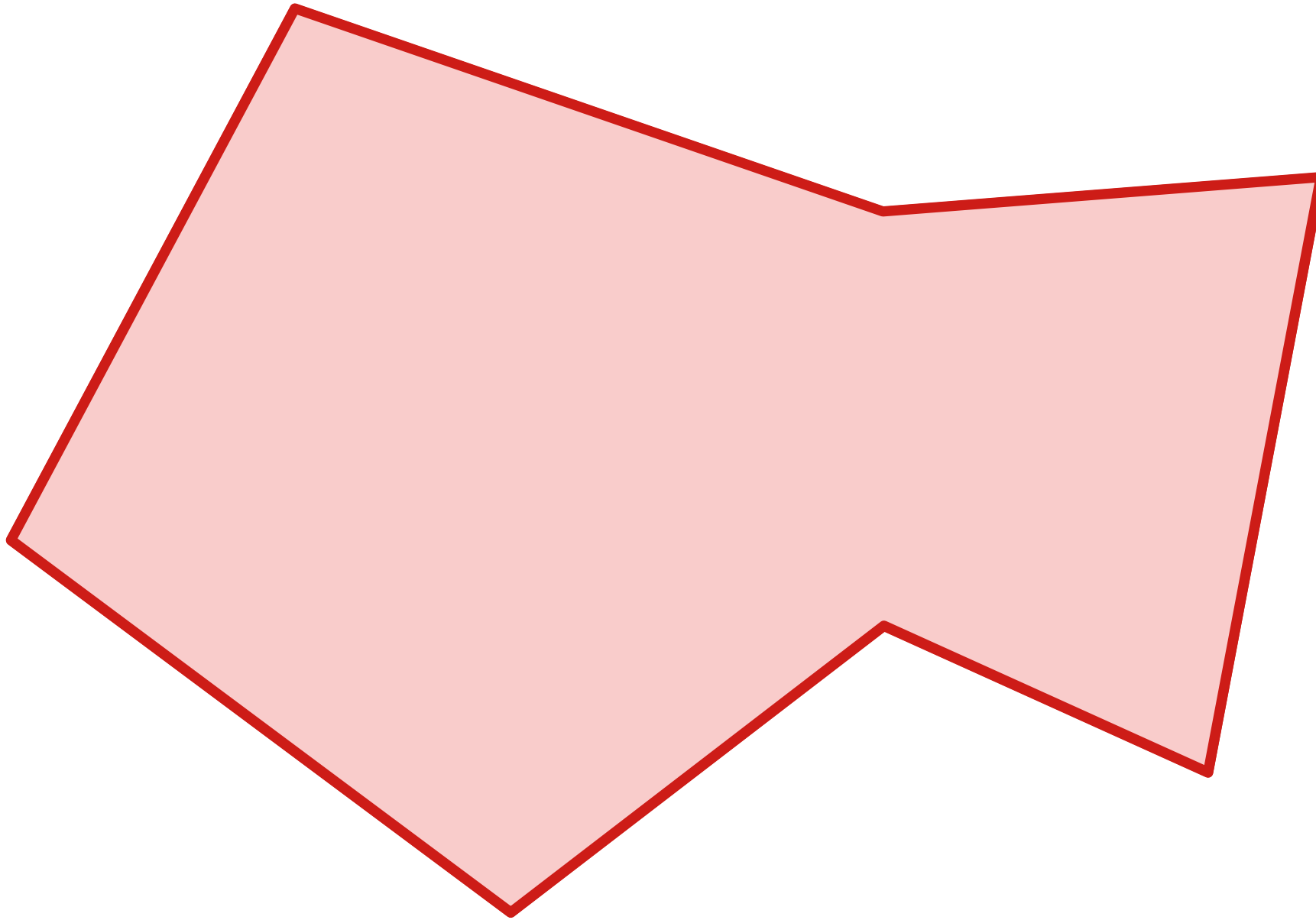


Figure 1: (a) Polygon hierarchy and (b) straight skeleton

↓
Market



↓
Market



```
static const int maxMarketPoints = 32;  
struct MarketRecord {  
    float2 points[maxMarketPoints];  
    ...  
};
```

```
[Shader("node")]  
[NodeLaunch("broadcasting")]  
[NodeDispatchGrid(1, 1, 1)]  
[NumThreads(maxMarketPoints, 1, 1)]  
...
```

```
void Market(  
    DispatchNodeInputRecord<MarketRecord> inputRecord,  
    uint gtid : SV_GroupThreadId,  
    ...
```

```
) {
```



```
static const int maxMarketPoints = 32;  
struct MarketRecord {  
    float2 points[maxMarketPoints];  
    ...  
};
```

```
[Shader("node")]  
[NodeLaunch("broadcasting")]  
[NodeDispatchGrid(1, 1, 1)]  
[NumThreads(maxMarketPoints, 1, 1)]  
...
```

```
void Market(  
    DispatchNodeInputRecord<MarketRecord> inputRecord,  
    uint gtid : SV_GroupThreadId,  
    ...  
    )
```

```
{
```



```
static const int maxMarketPoints = 32;  
struct MarketRecord {  
    float2 points[maxMarketPoints];  
    ...  
};
```

```
[Shader("node")]  
[NodeLaunch("broadcasting")]  
[NodeDispatchGrid(1, 1, 1)]  
[NumThreads(maxMarketPoints, 1, 1)]  
...
```

```
void Market(  
    DispatchNodeInputRecord<MarketRecord> inputRecord,  
    uint gtid : SV_GroupThreadId,  
    ...
```

```
) {
```




```
static const int maxMarketPoints = 32;  
struct MarketRecord {  
    float2 points[maxMarketPoints];  
    ...  
};
```

```
[Shader("node")]
```

```
[NodeLaunch("broadcasting")]
```

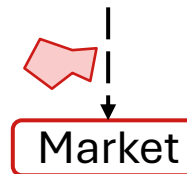
```
[NodeDispatchGrid(1, 1, 1)]
```

```
[NumThreads(maxMarketPoints, 1, 1)]
```

```
...
```

```
void Market(  
    DispatchNodeInputRecord<MarketRecord> inputRecord,  
    uint gtid : SV_GroupThreadId,  
    ...  
){
```

```
)
```



```
static const int maxMarketPoints = 32;  
struct MarketRecord {  
    float2 points[maxMarketPoints];  
    ...  
};
```

```
[Shader("node")]
```

```
[NodeLaunch("broadcasting")]
```

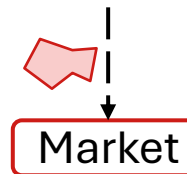
```
[NodeDispatchGrid(1, 1, 1)]
```

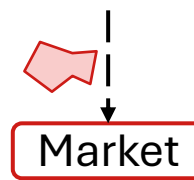
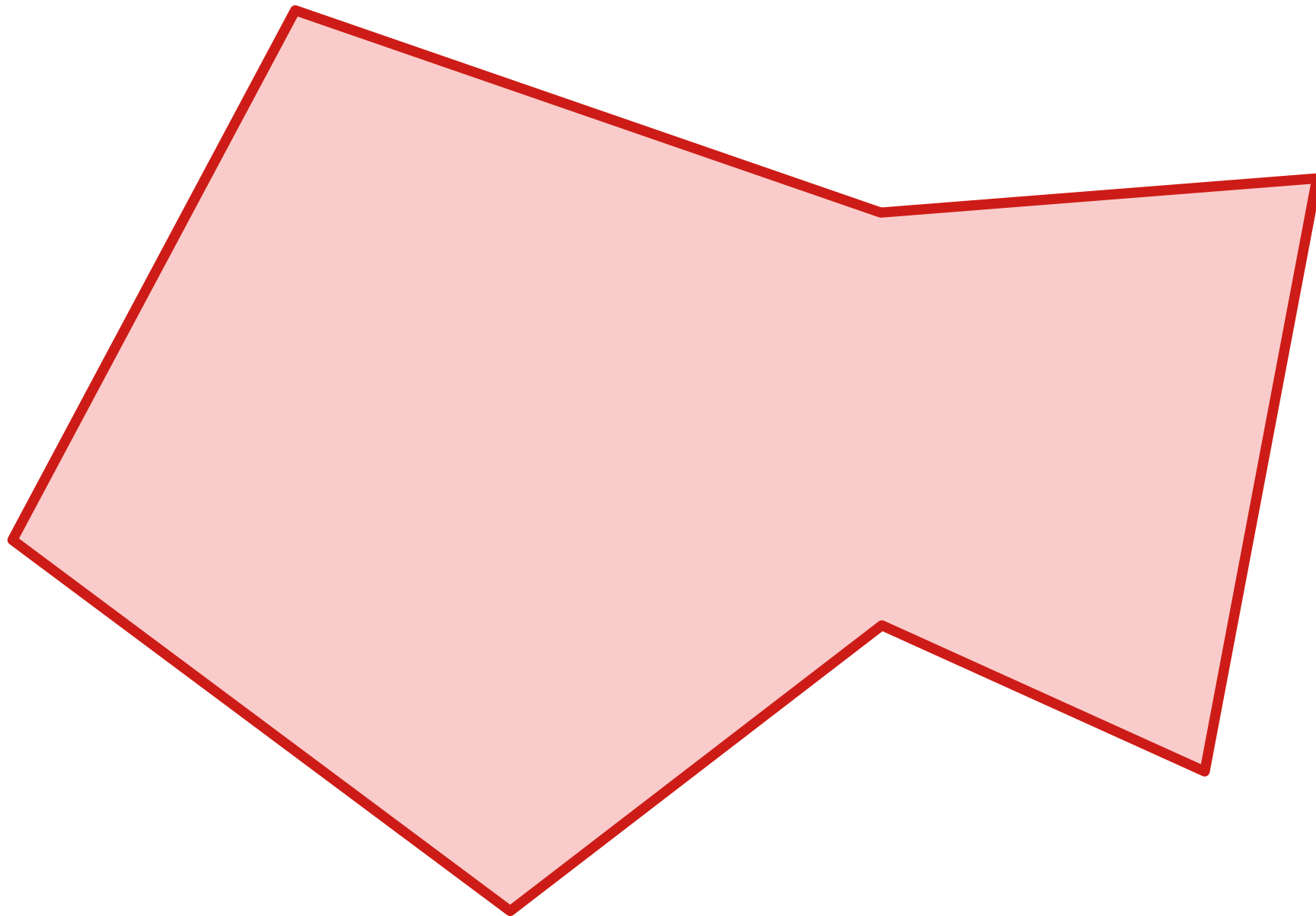
```
[NumThreads(maxMarketPoints, 1, 1)]
```

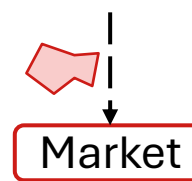
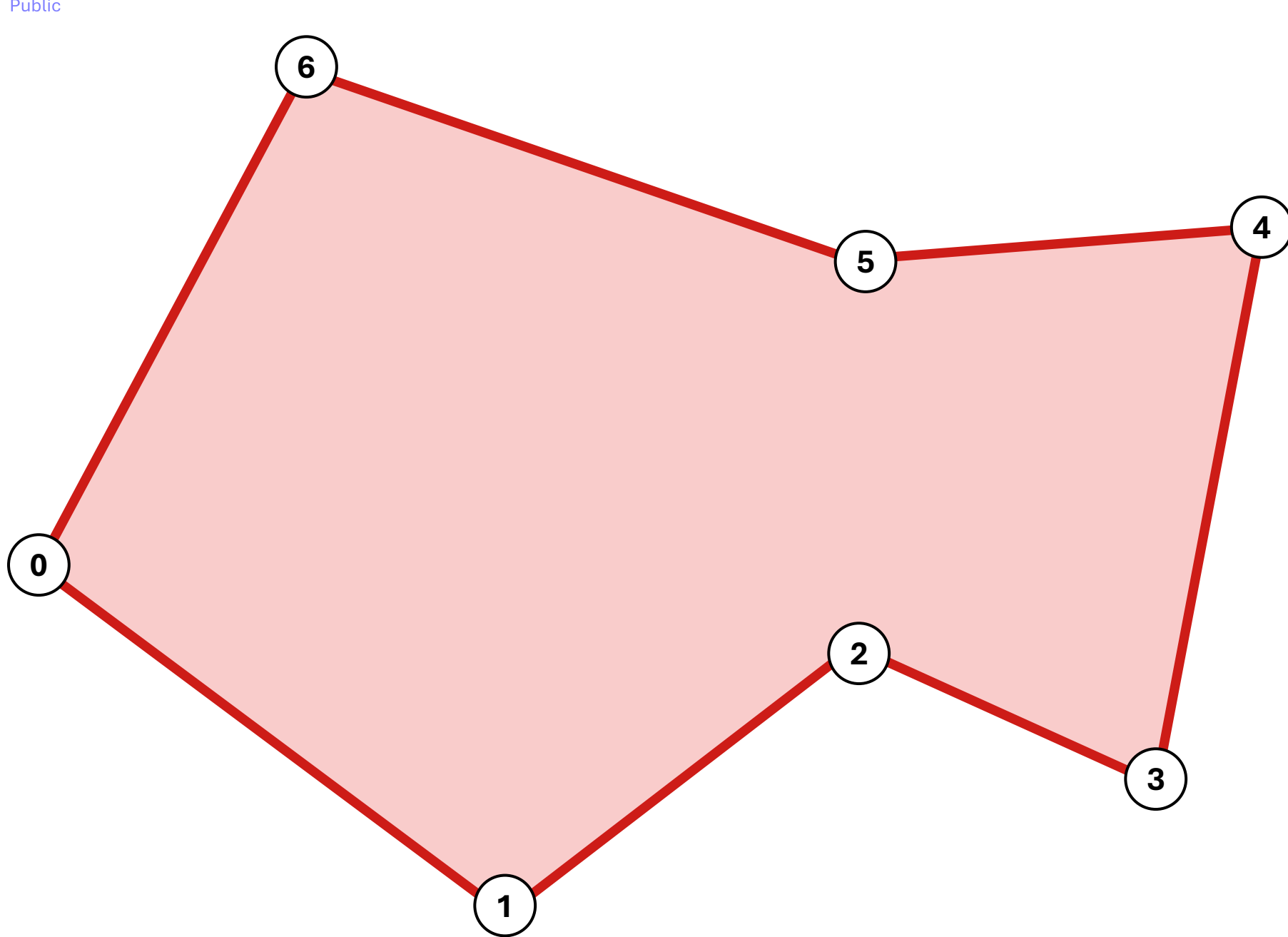
```
...
```

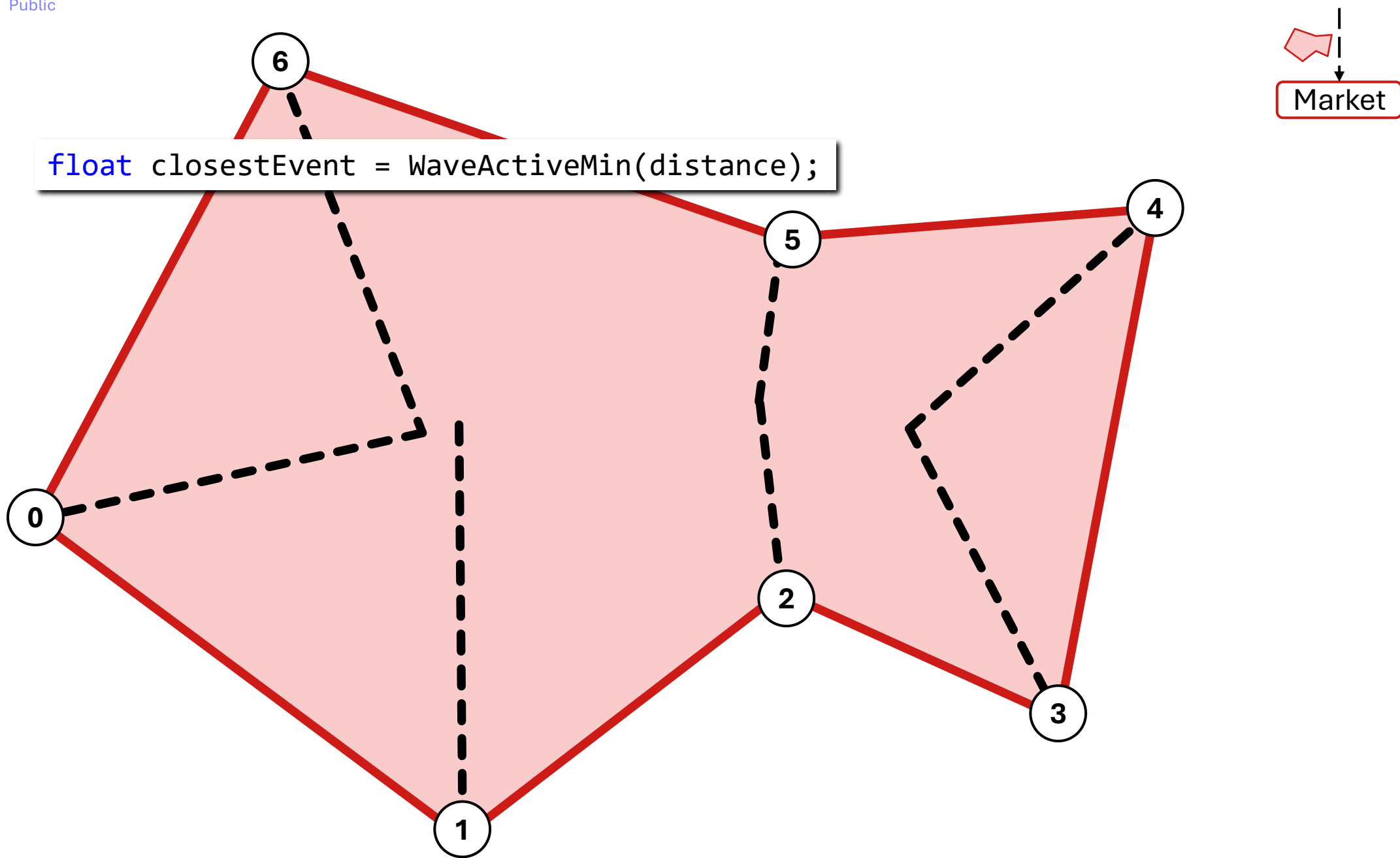
```
void Market(  
    DispatchNodeInputRecord<MarketRecord> inputRecord,  
    uint gtid : SV_GroupThreadId,  
    ...  
){
```

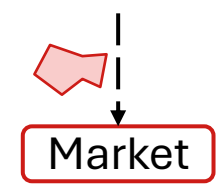
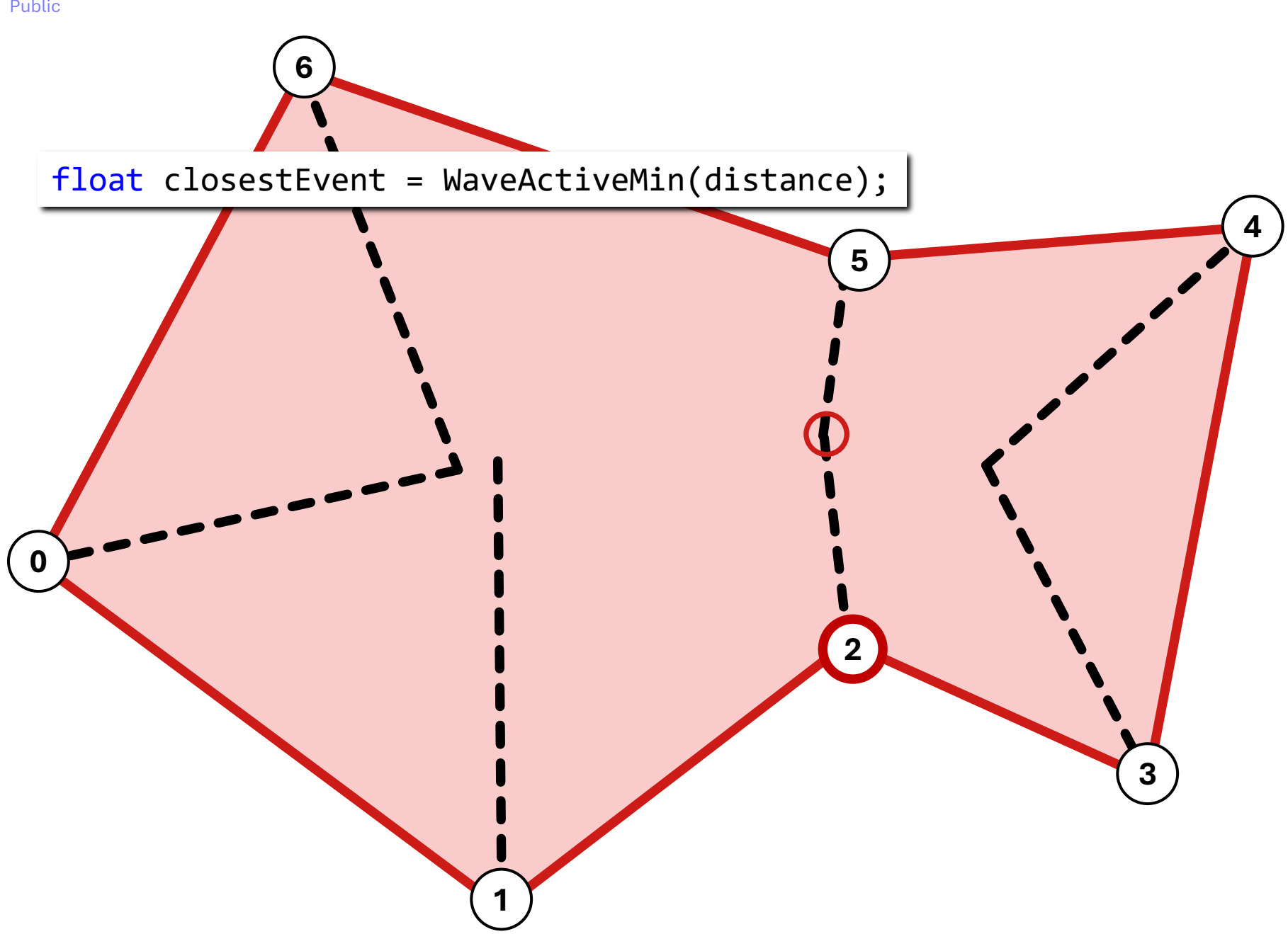
```
)
```

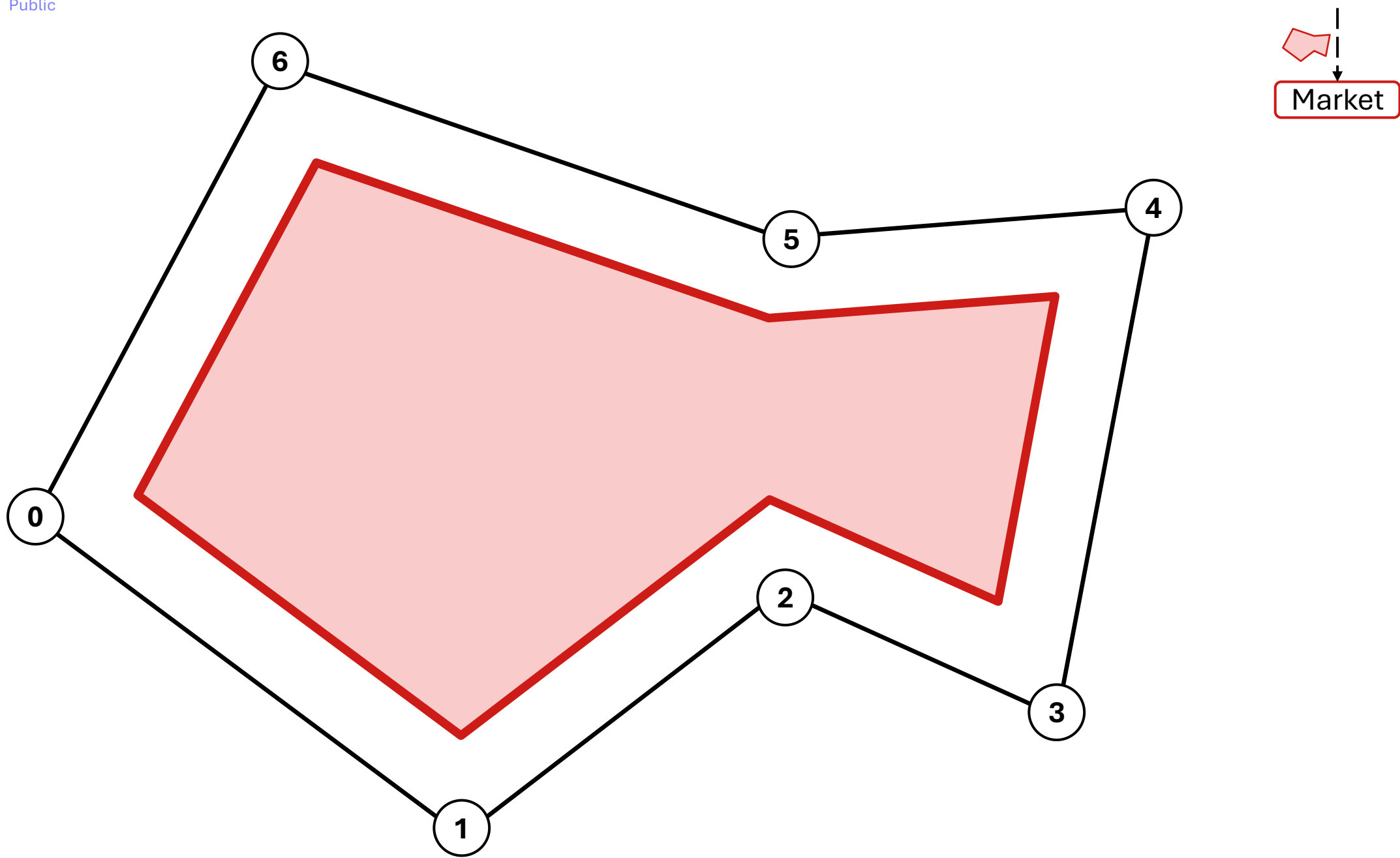


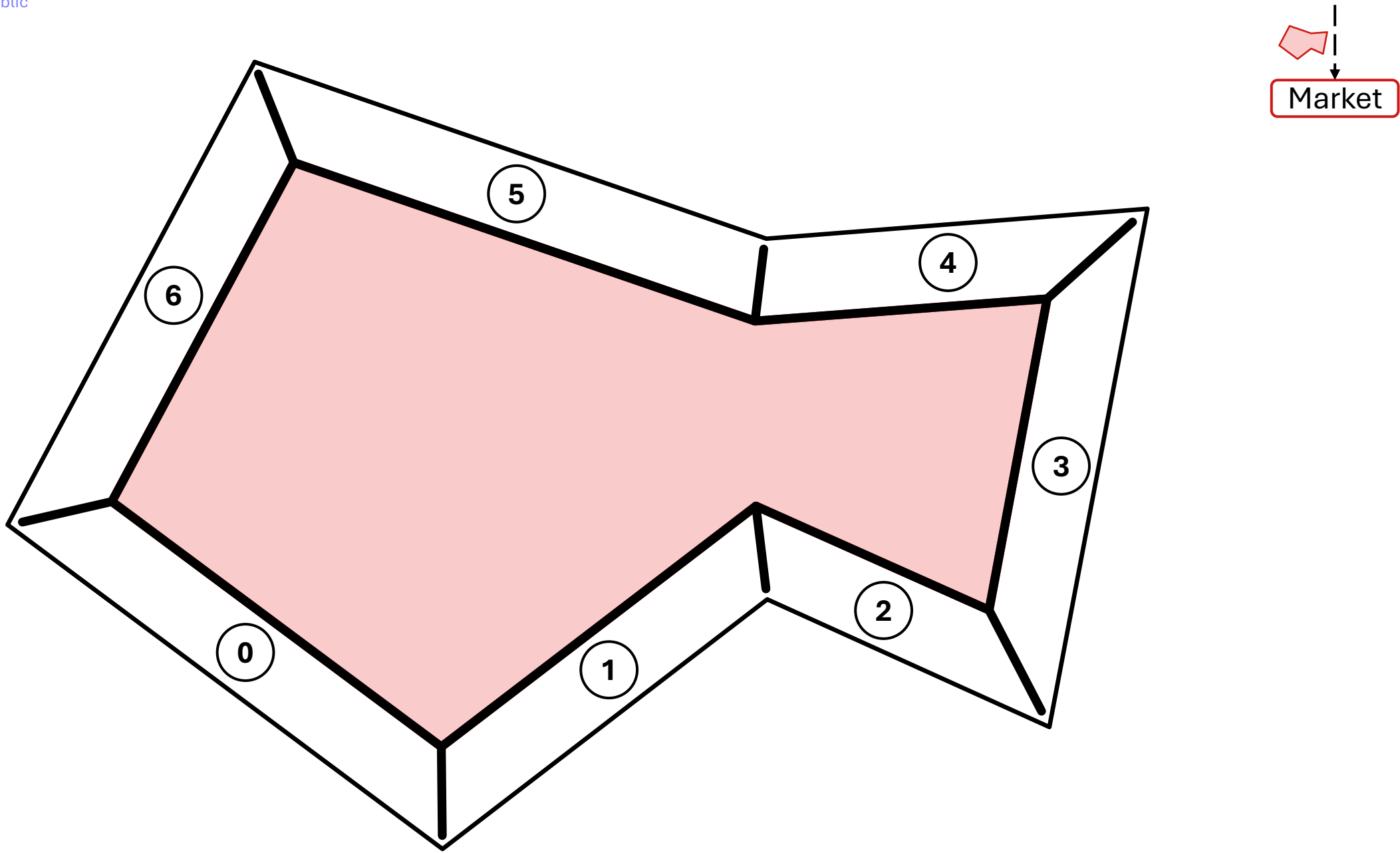


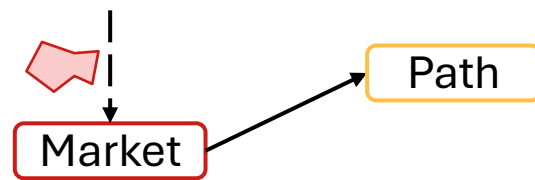
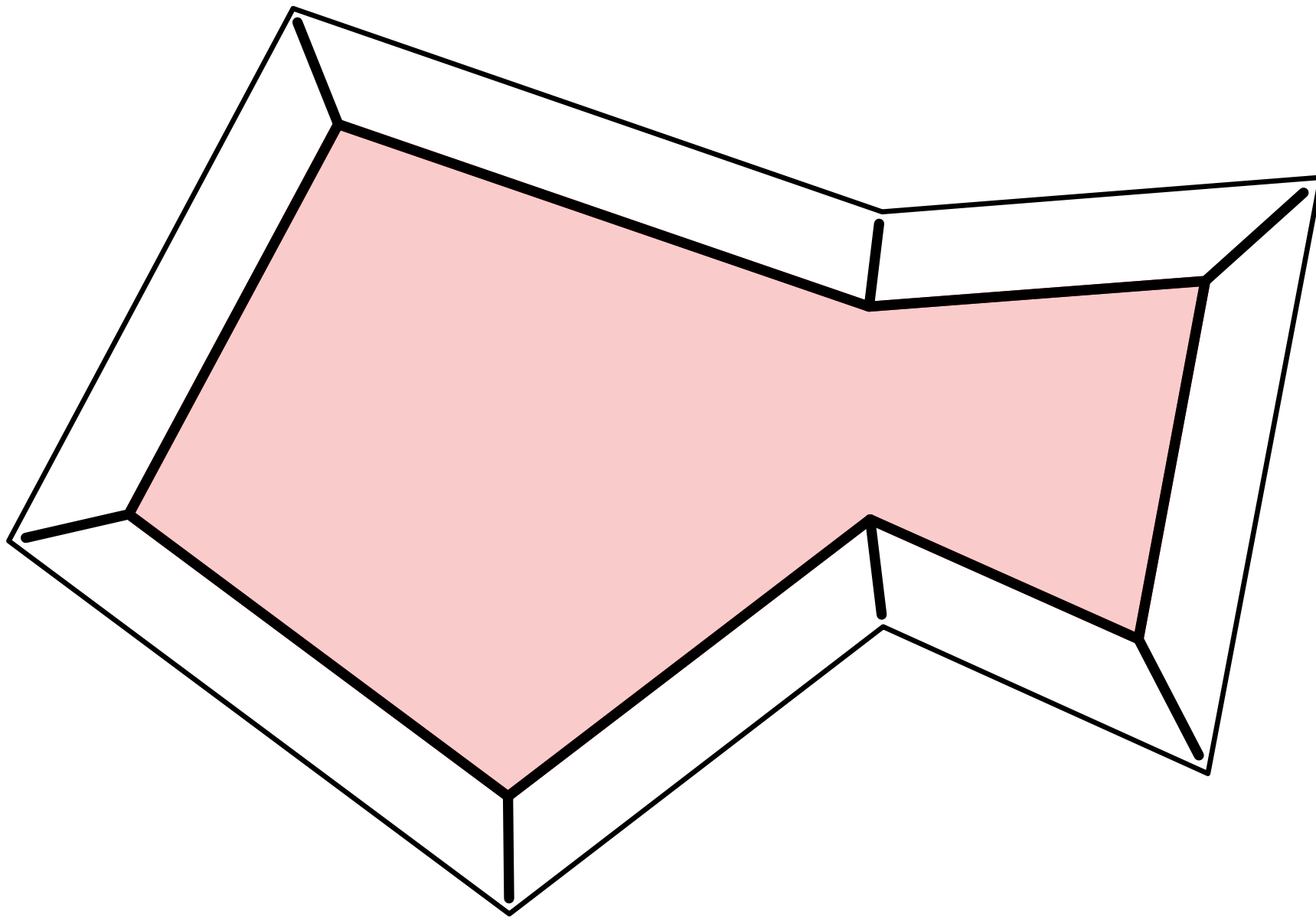


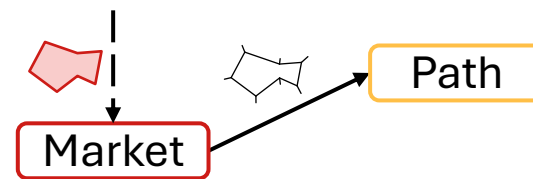
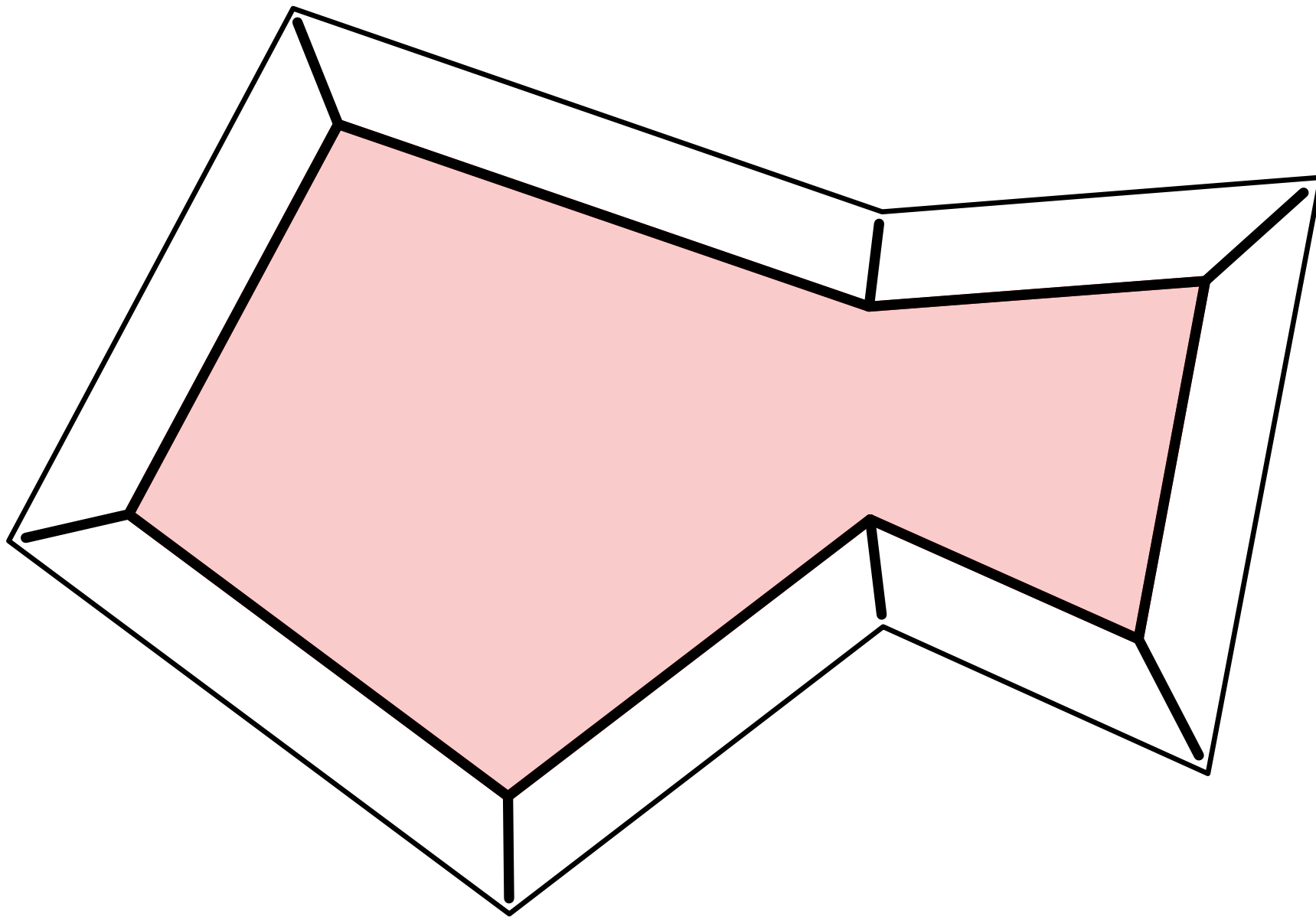


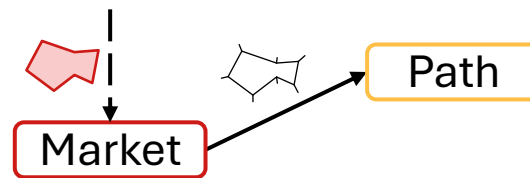
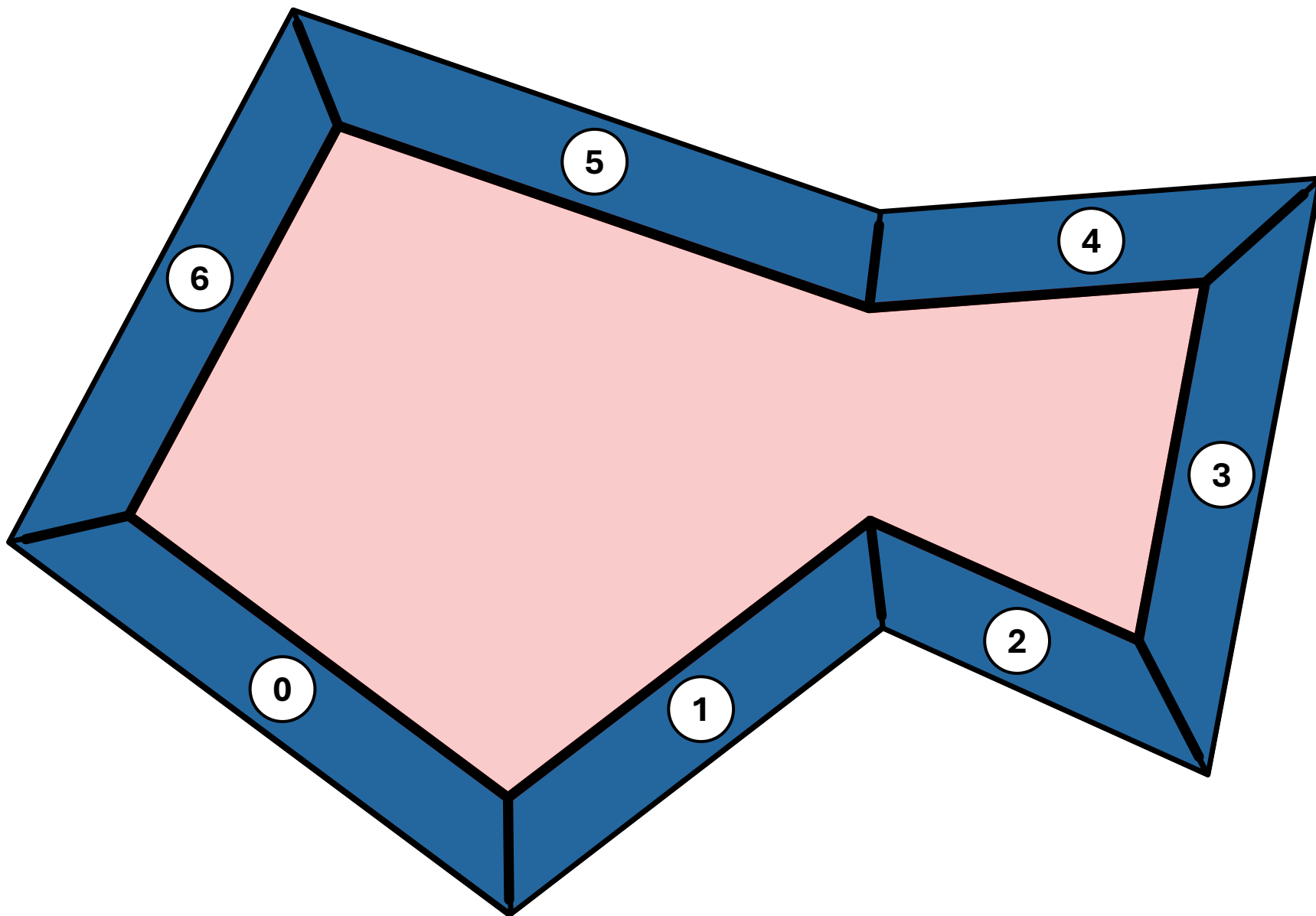


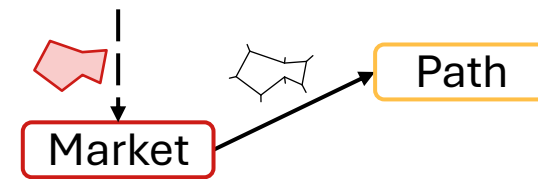
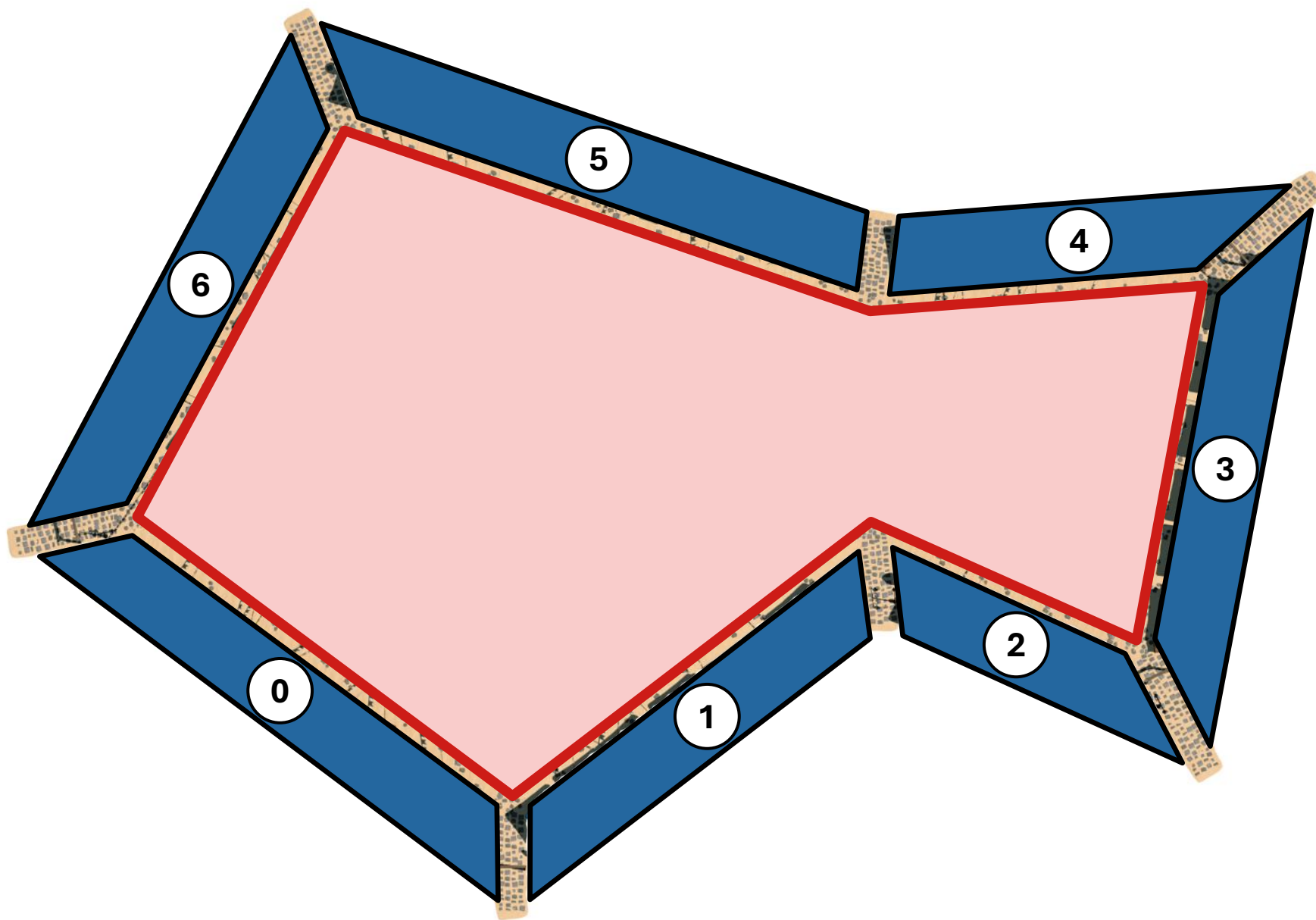


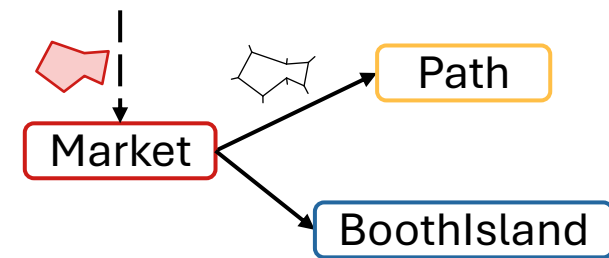
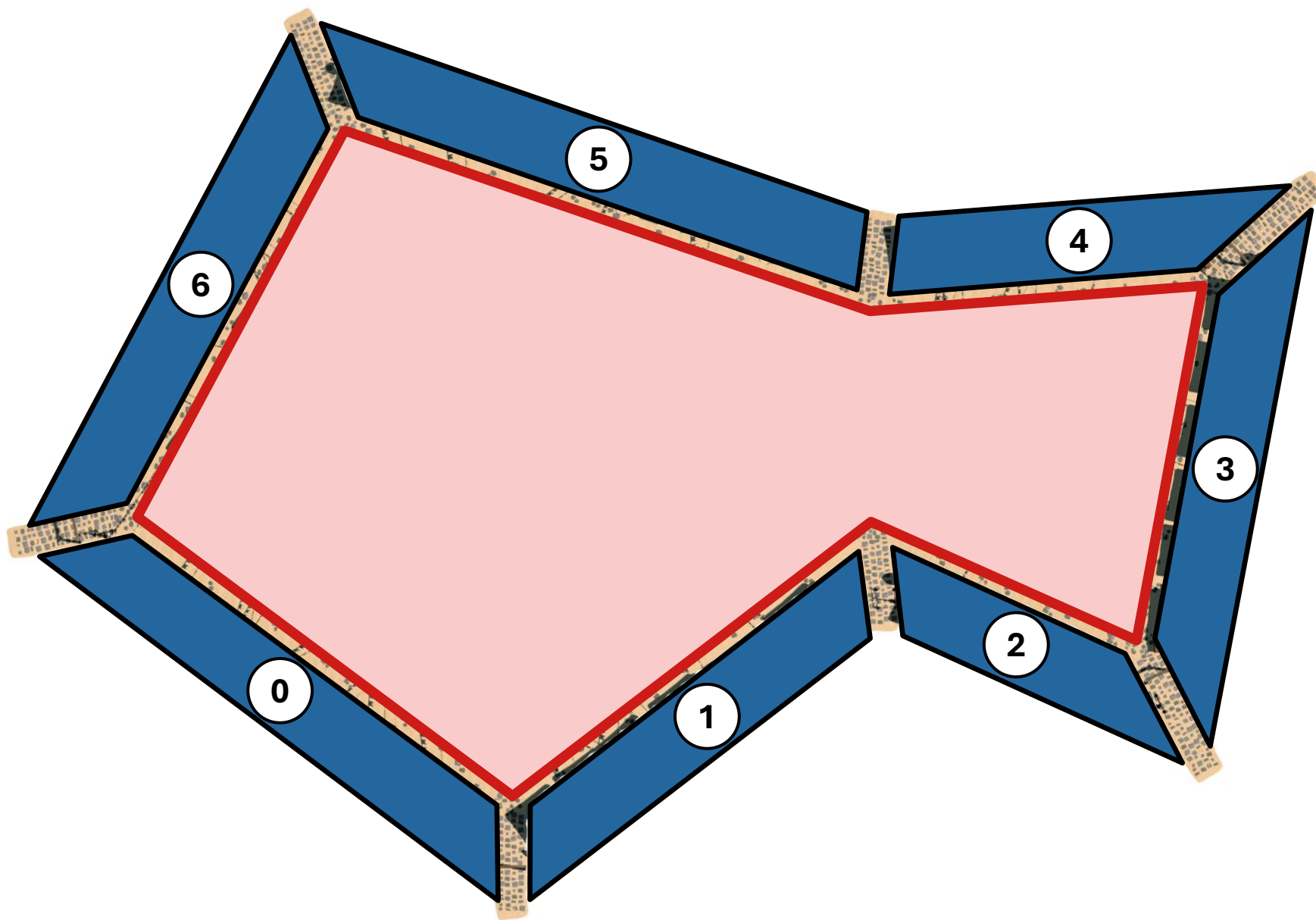










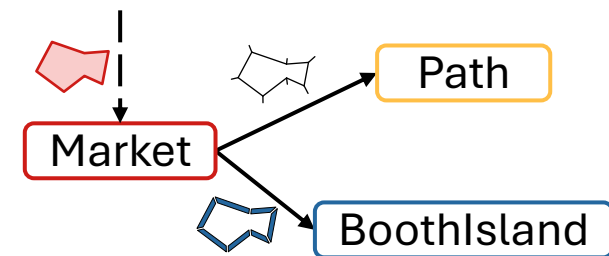



```
static const int maxMarketPoints = 32;  
struct MarketRecord {  
    float2 points[maxMarketPoints];  
    ...  
};
```

```
[Shader("node")]  
[NodeLaunch("broadcasting")]  
[NodeDispatchGrid(1, 1, 1)]  
[NumThreads(maxMarketPoints, 1, 1)]  
...
```

```
void Market(  
    DispatchNodeInputRecord<MarketRecord> inputRecord,  
    uint gtid : SV_GroupThreadId,  
    ...
```

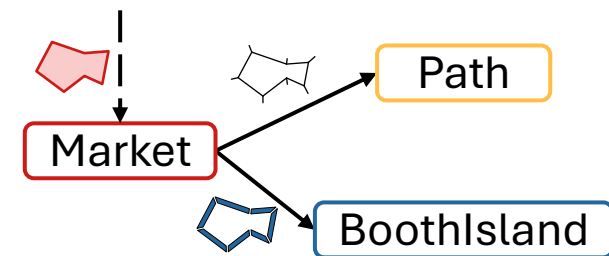
```
) {
```



```
static const int maxMarketPoints = 32;
struct MarketRecord {
    float2 points[maxMarketPoints];
    ...
};
```

```
[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeDispatchGrid(1, 1, 1)]
[NumThreads(maxMarketPoints, 1, 1)]
...
```

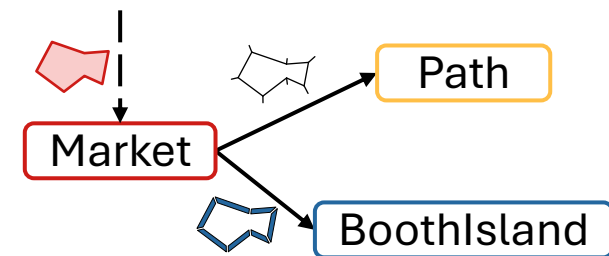
```
void Market(
    DispatchNodeInputRecord<MarketRecord> inputRecord,
    uint gtid : SV_GroupThreadId,
    [MaxRecords(maxMarketPoints)]
    NodeOutput<BoothIslandRecord> BoothIsland,
    [MaxRecords(2*maxMarketPoints)]
    NodeOutput<PathRecord> Path,
    ...
){
```

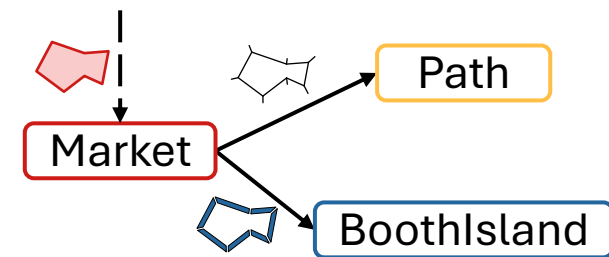
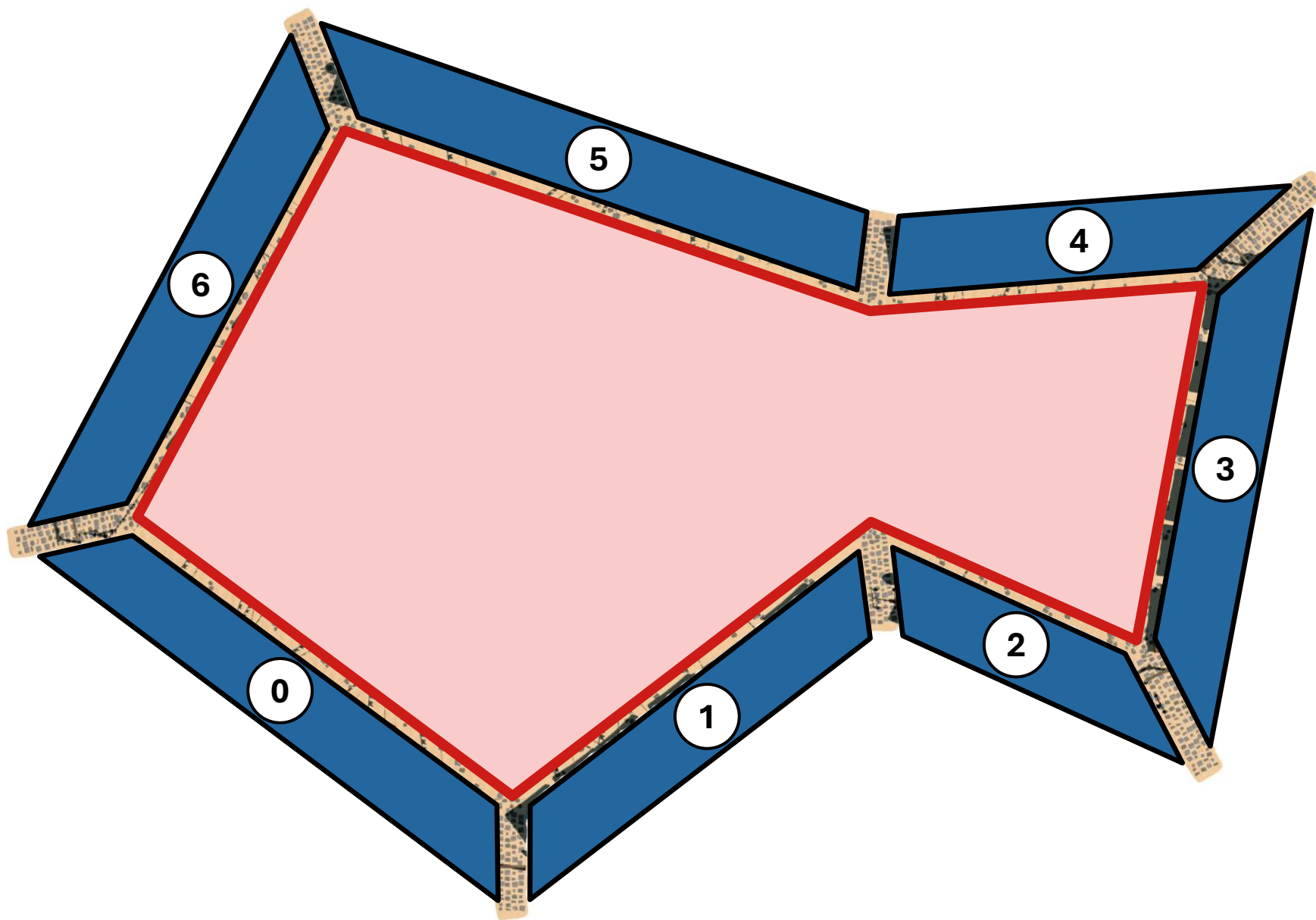


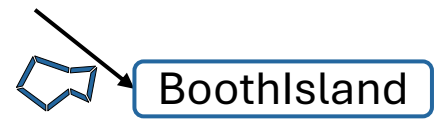
```
static const int maxMarketPoints = 32;
struct MarketRecord {
    float2 points[maxMarketPoints];
    ...
};
```

```
[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeDispatchGrid(1, 1, 1)]
[NumThreads(maxMarketPoints, 1, 1)]
...
```

```
void Market(
    DispatchNodeInputRecord<MarketRecord> inputRecord,
    uint gtid : SV_GroupThreadId,
    [MaxRecords(maxMarketPoints)]
    NodeOutput<BoothIslandRecord> BoothIsland,
    [MaxRecords(2*maxMarketPoints)]
    NodeOutput<PathRecord> Path,
    ...
){
```









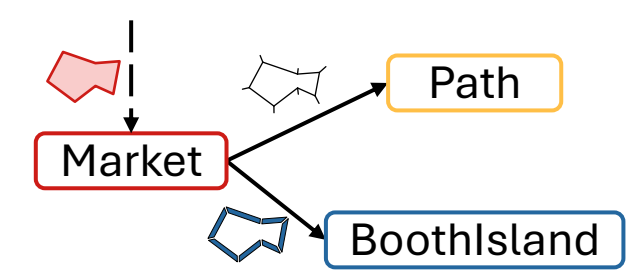
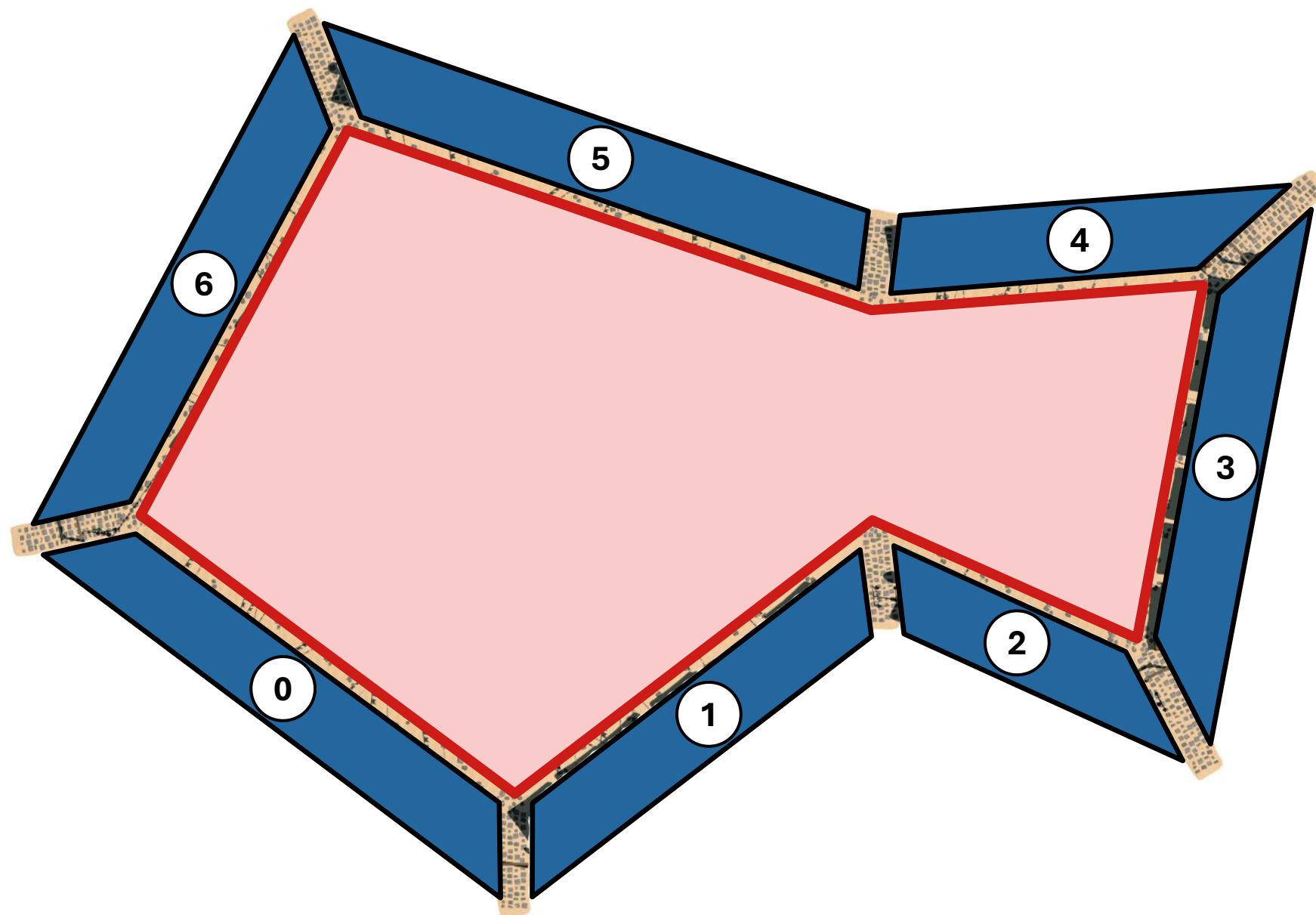
Draw List

```
int index;  
InterlockedAdd(drawMeshArgumentCounter, 1, index);  
drawMeshArguments[index] = args;
```

BVH Instance List

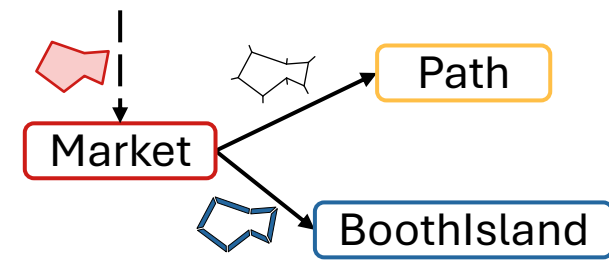
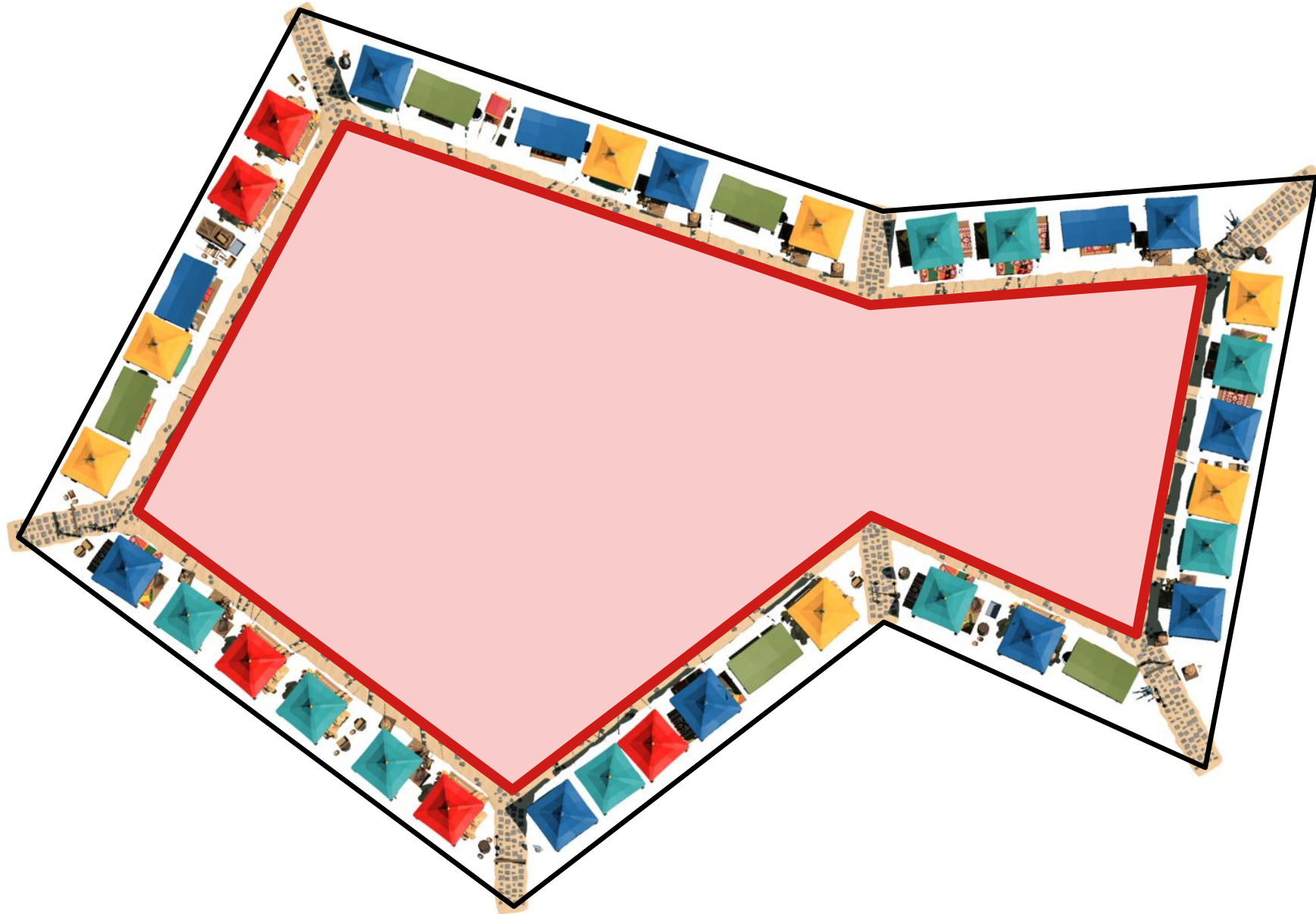
Mesh Nodes

```
[Shader("node")]  
[NodeLaunch("mesh")]  
[OutputTopology("triangle")]  
...  
void MeshShader(...)
```



Draw List

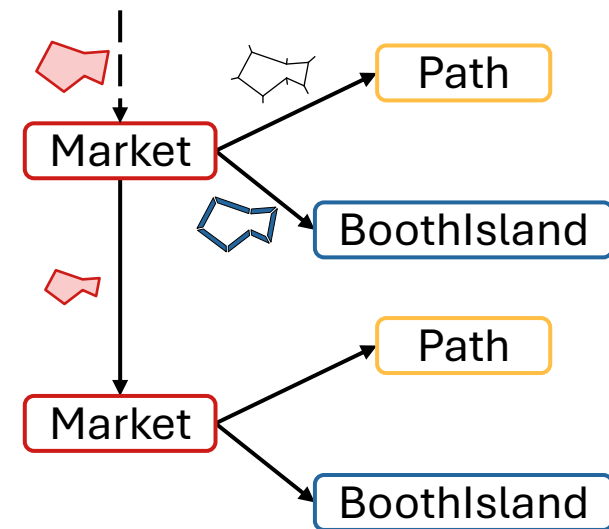
```
int index;  
InterlockedAdd(  
drawMeshArgument
```



```
static const int maxMarketPoints = 32;
struct MarketRecord {
    float2 points[maxMarketPoints];
    ...
};
```

```
[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeDispatchGrid(1, 1, 1)]
[NumThreads(maxMarketPoints, 1, 1)]
...
```

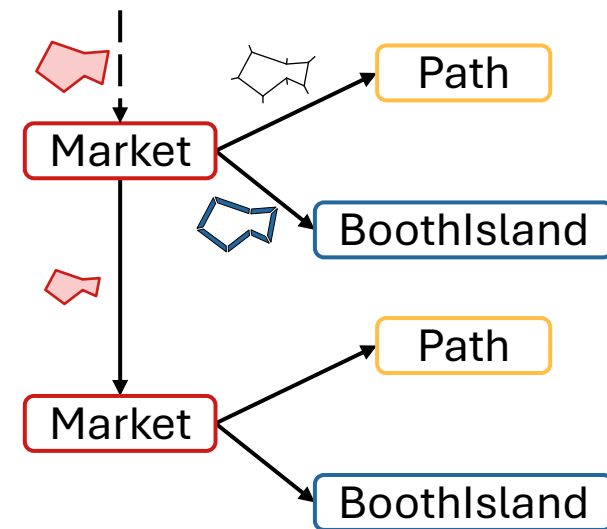
```
void Market(
    DispatchNodeInputRecord<MarketRecord> inputRecord,
    uint gtid : SV_GroupThreadId,
    [MaxRecords(maxMarketPoints)]
    NodeOutput<BoothIslandRecord> BoothIsland,
    [MaxRecords(2*maxMarketPoints)]
    NodeOutput<PathRecord> Path,
    ...
){
```

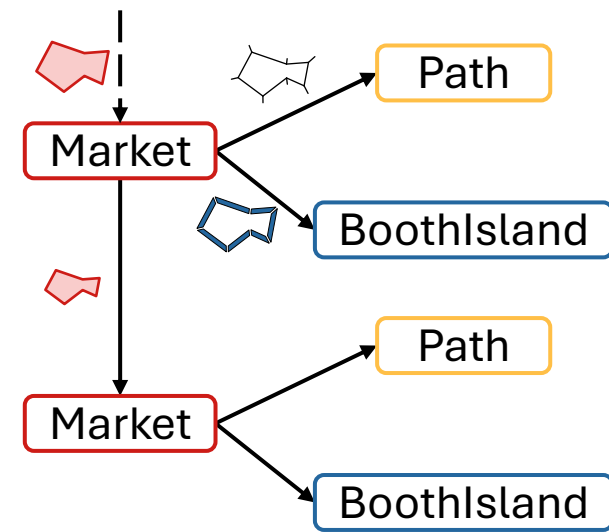
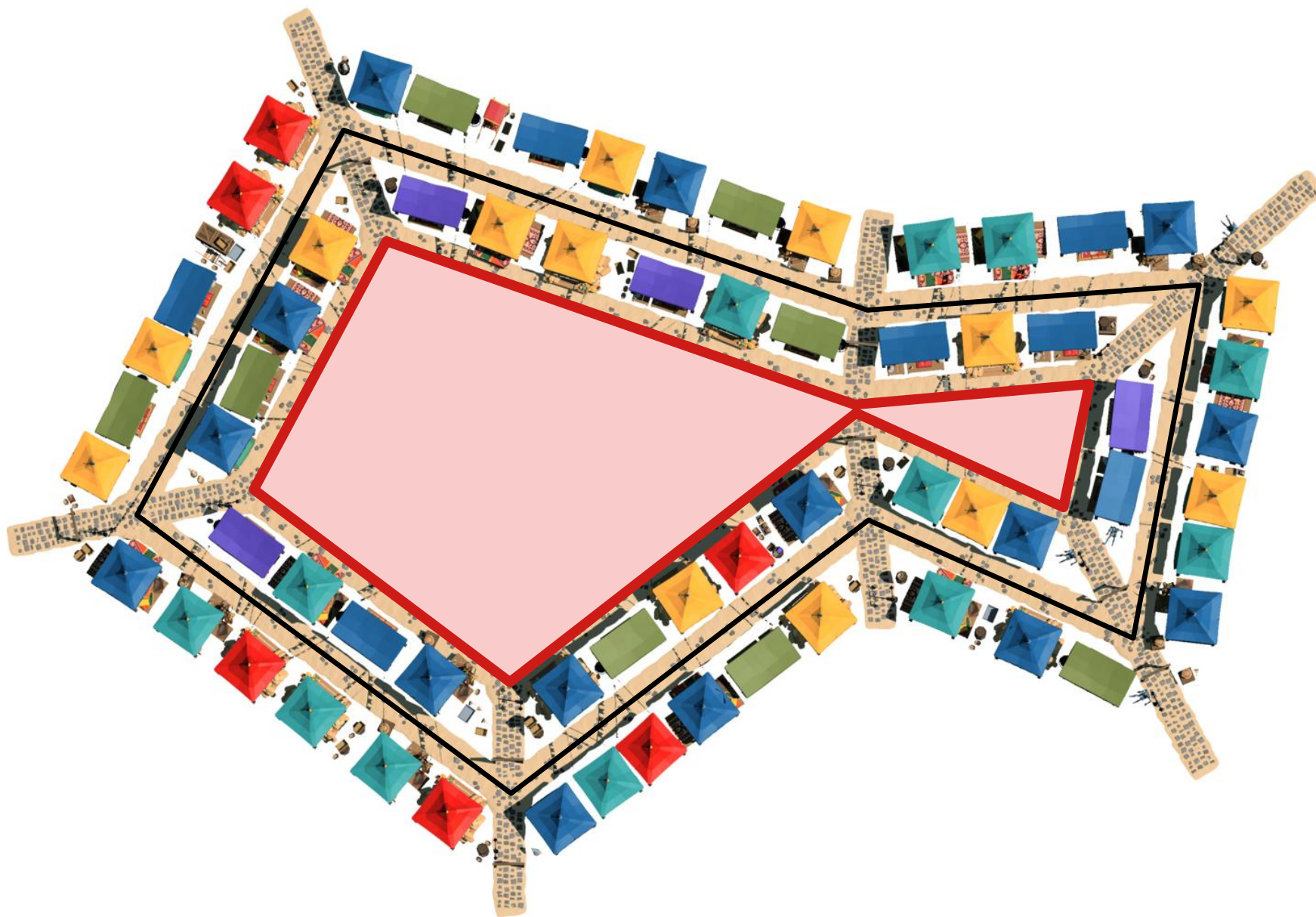


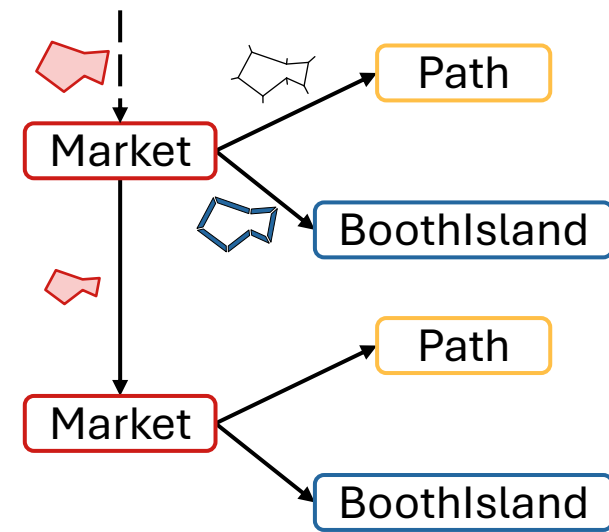
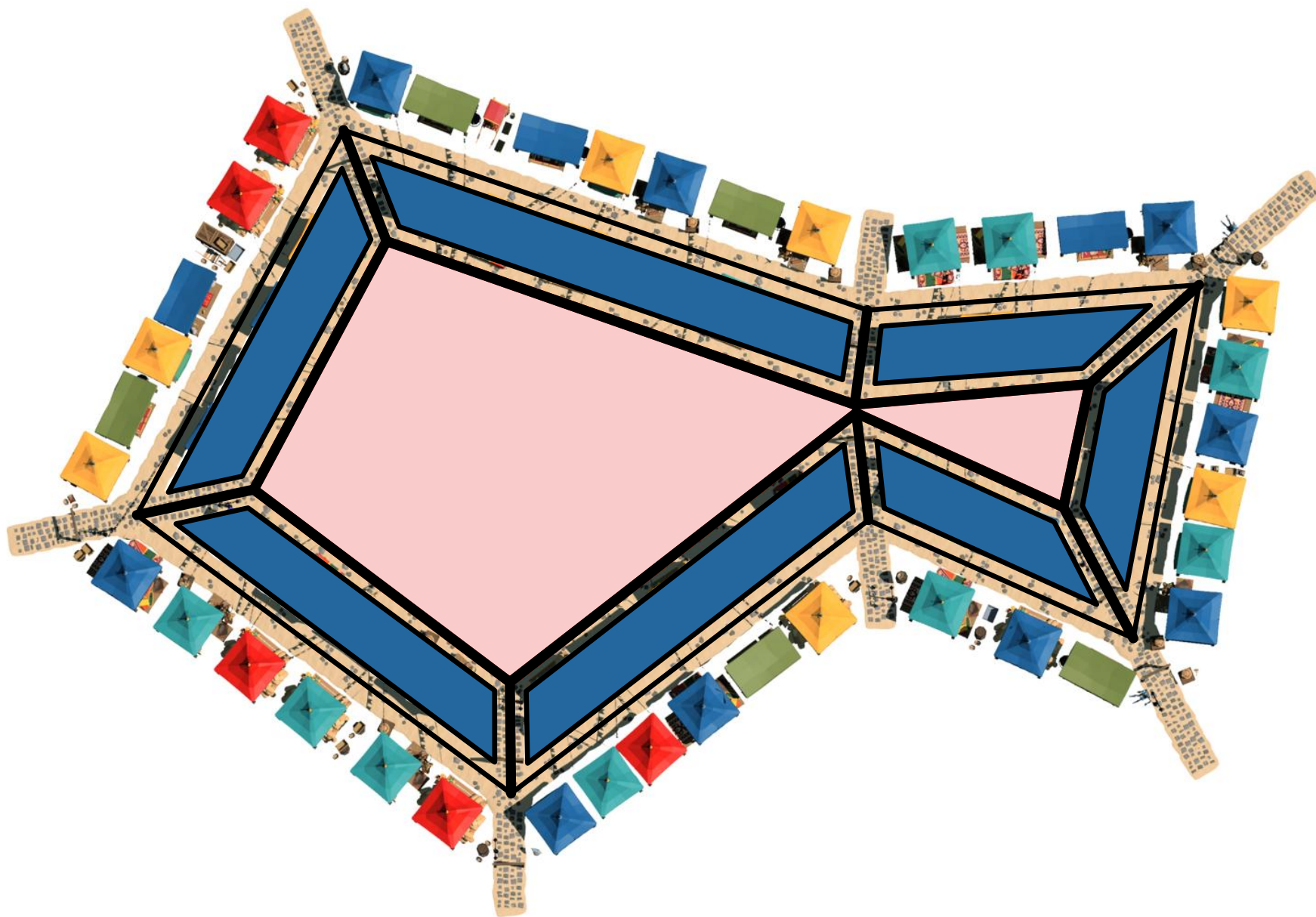
```
static const int maxMarketPoints = 32;
struct MarketRecord {
    float2 points[maxMarketPoints];
    ...
};
```

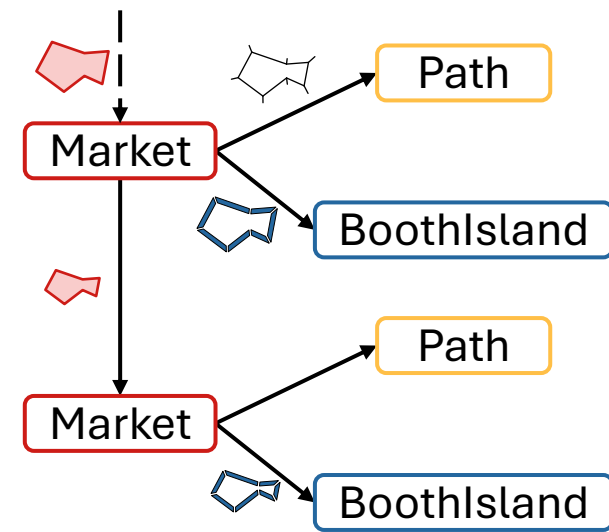
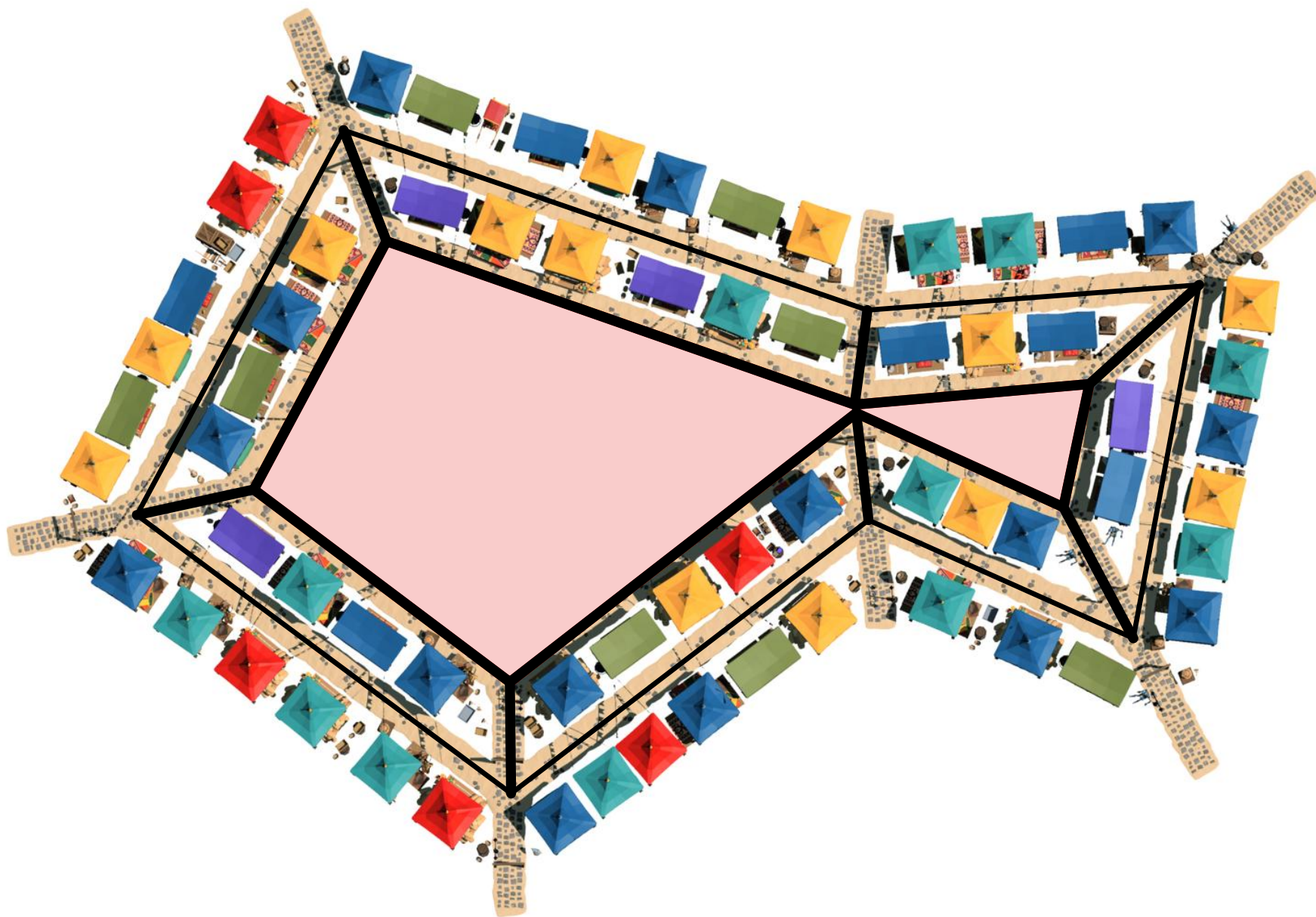
```
[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeDispatchGrid(1, 1, 1)]
[NumThreads(maxMarketPoints, 1, 1)]
[NodeMaxRecursionDepth(12)]
```

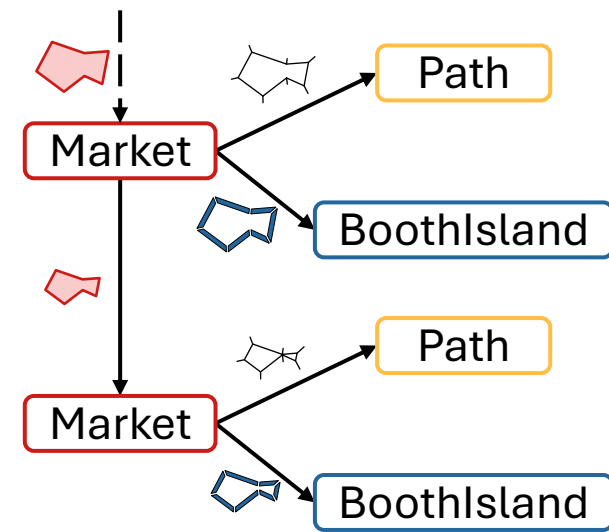
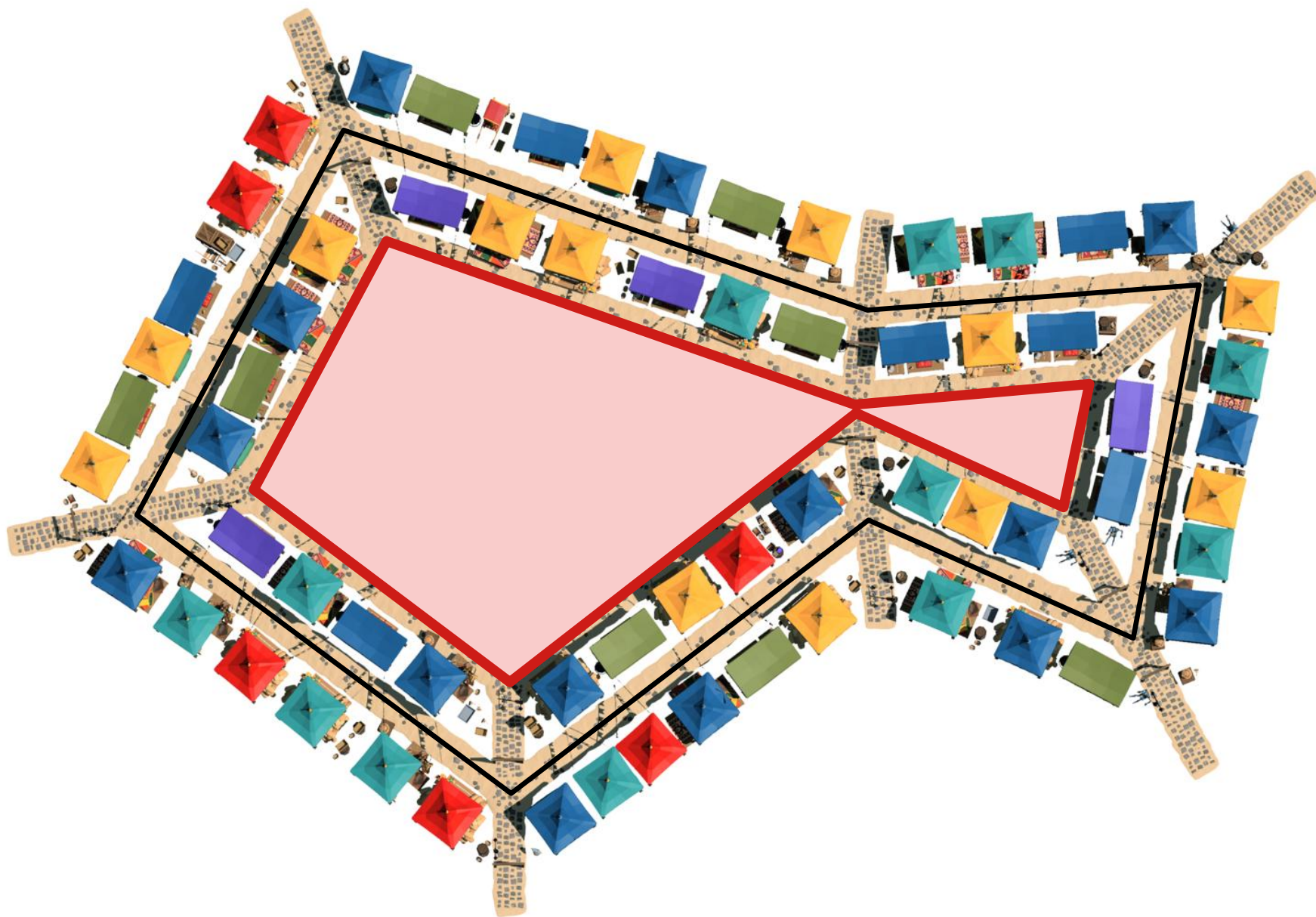
```
void Market(
    DispatchNodeInputRecord<MarketRecord> inputRecord,
    uint gtid : SV_GroupThreadId,
    [MaxRecords(maxMarketPoints)]
    NodeOutput<BoothIslandRecord> BoothIsland,
    [MaxRecords(2*maxMarketPoints)]
    NodeOutput<PathRecord> Path,
    [MaxRecords(2)] NodeOutput<MarketRecord> Market,
    ...
){
```

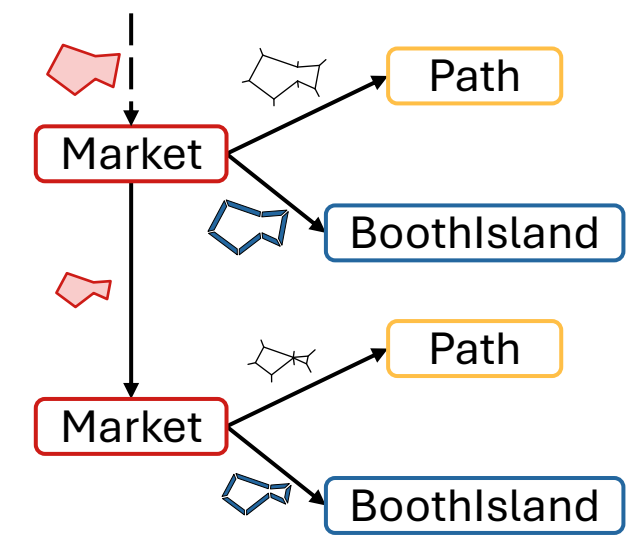
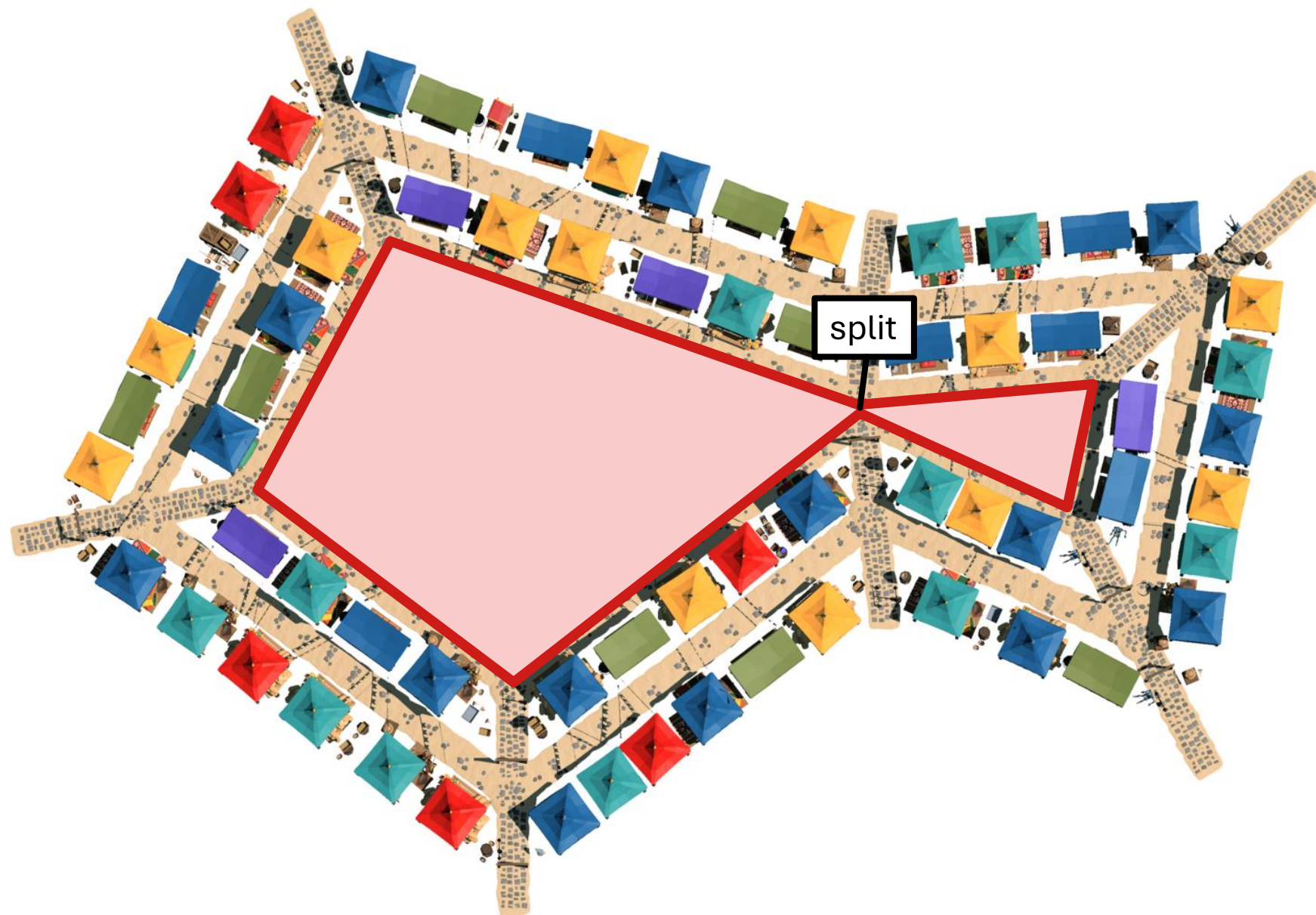


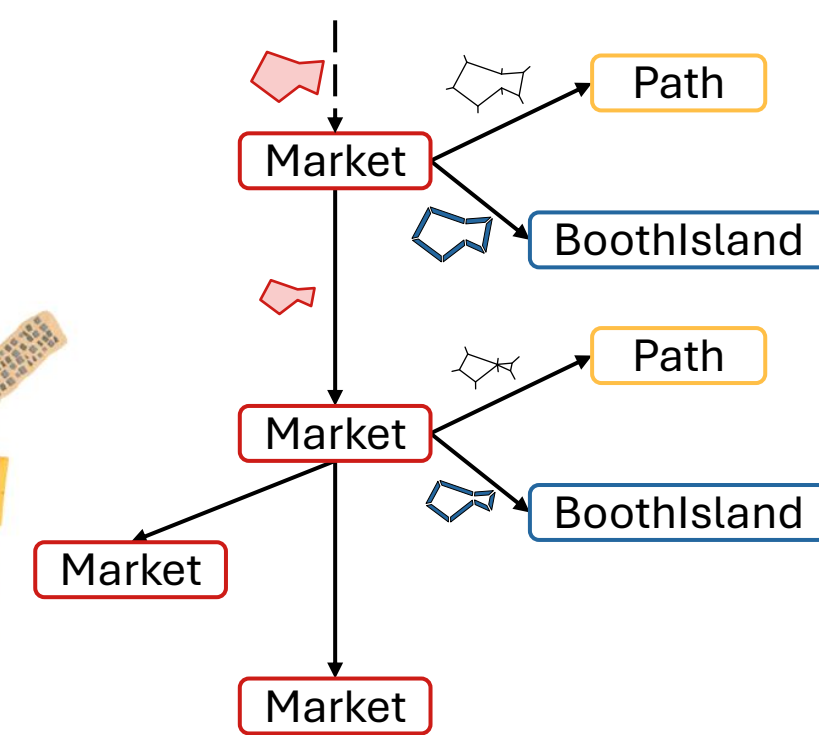
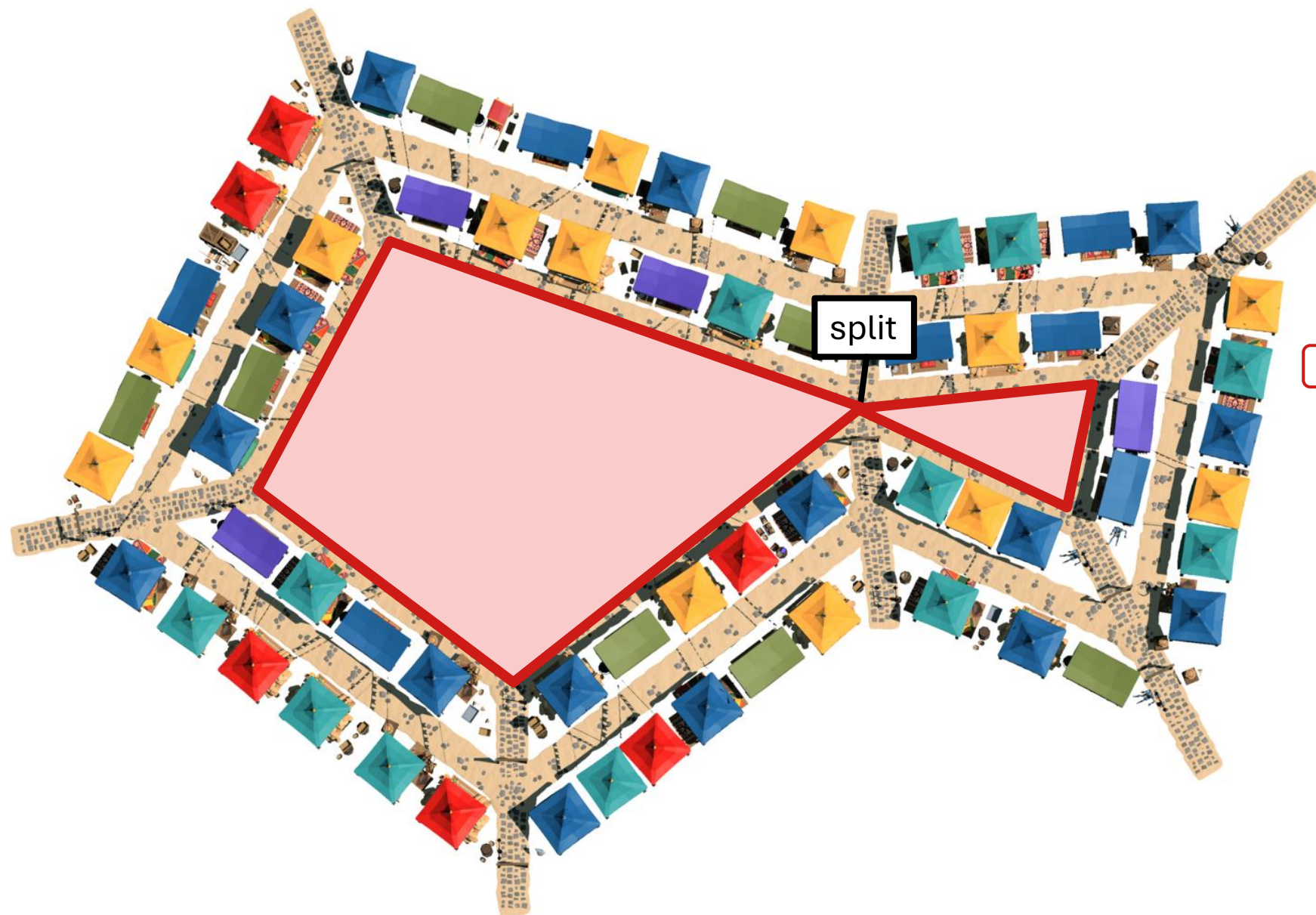


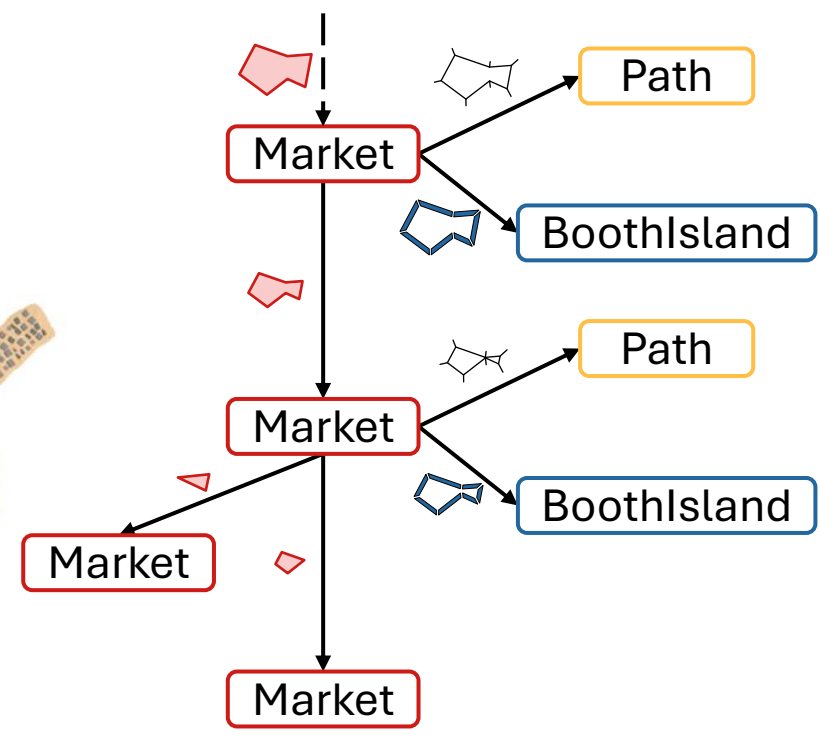
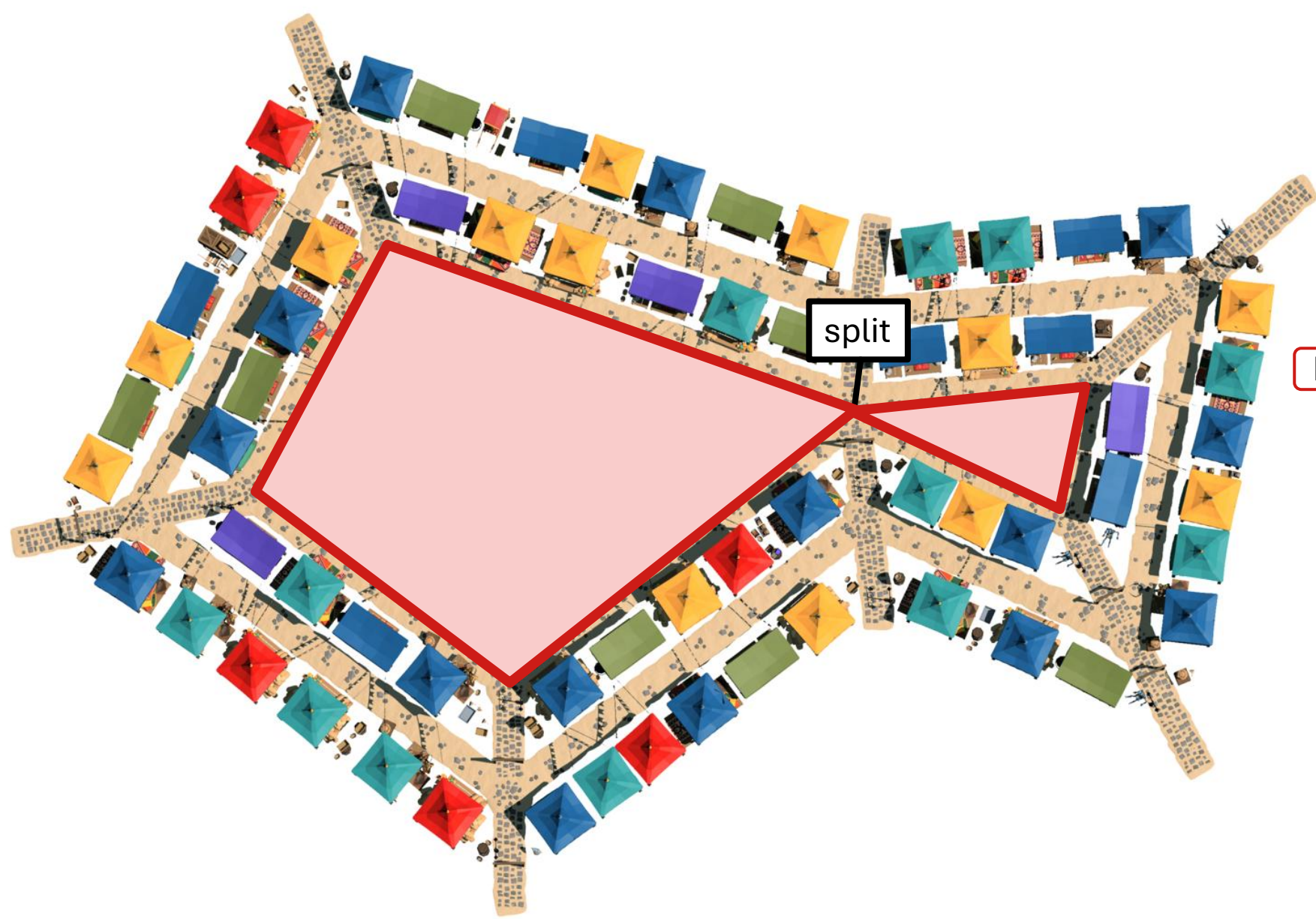


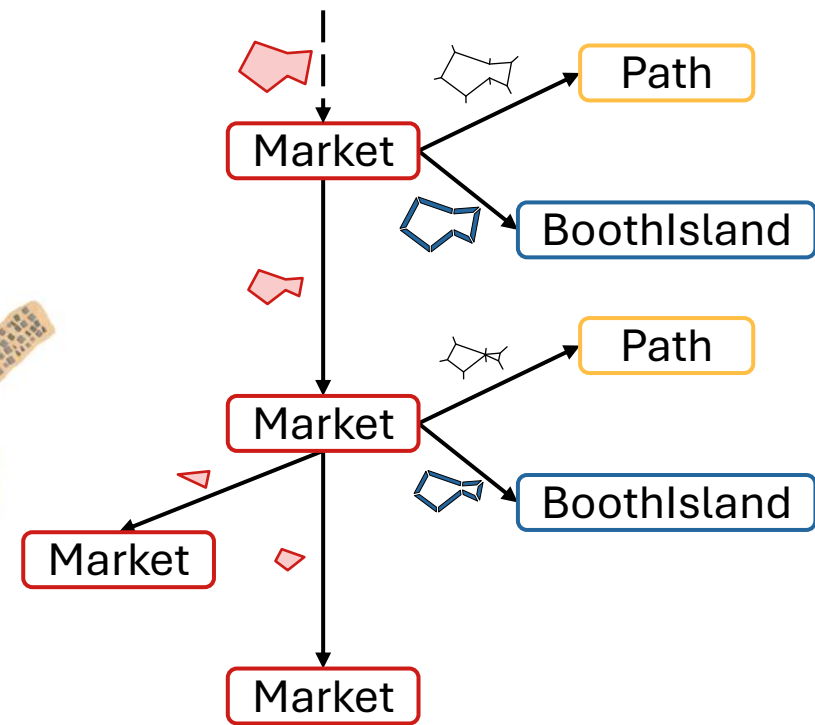
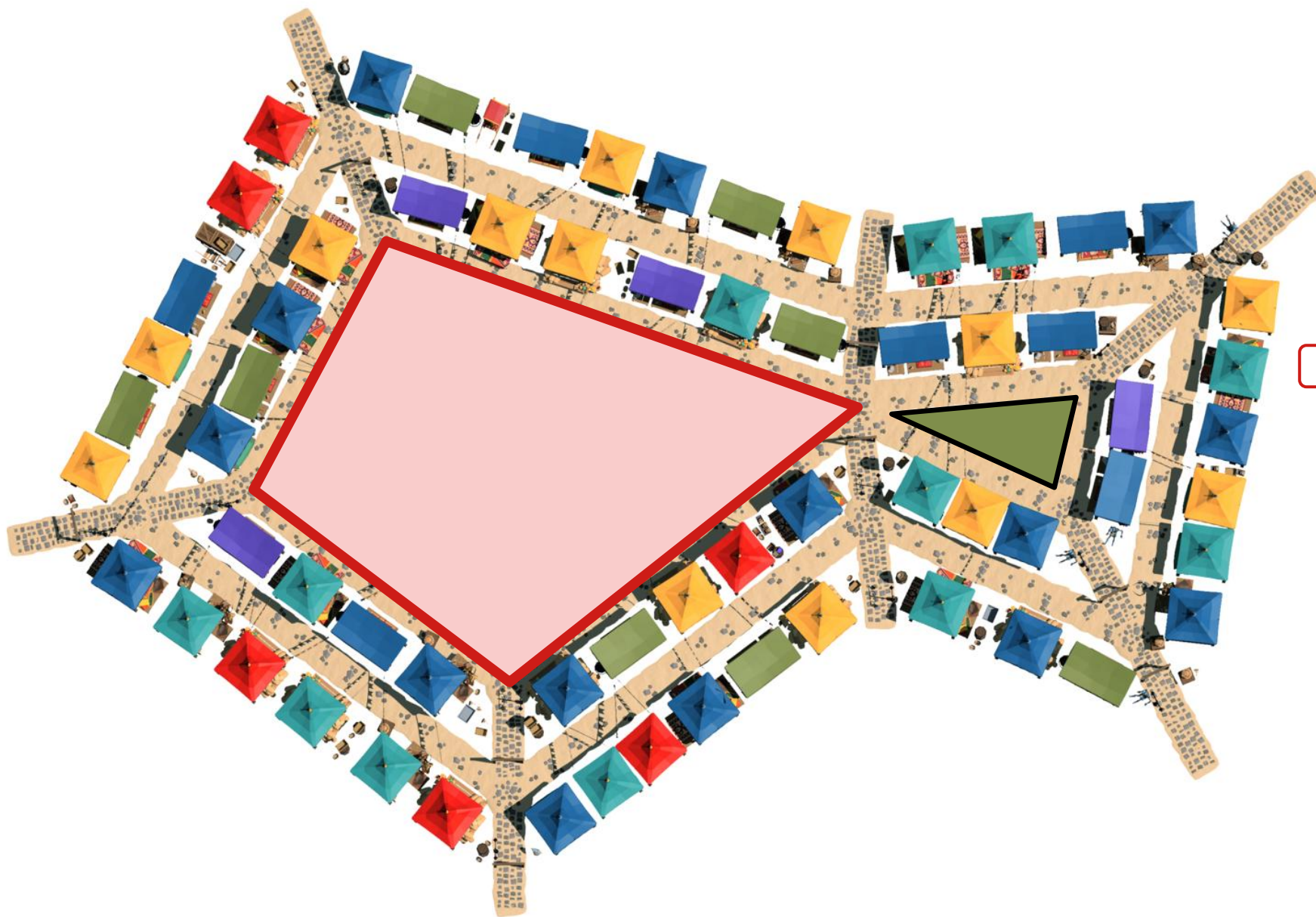


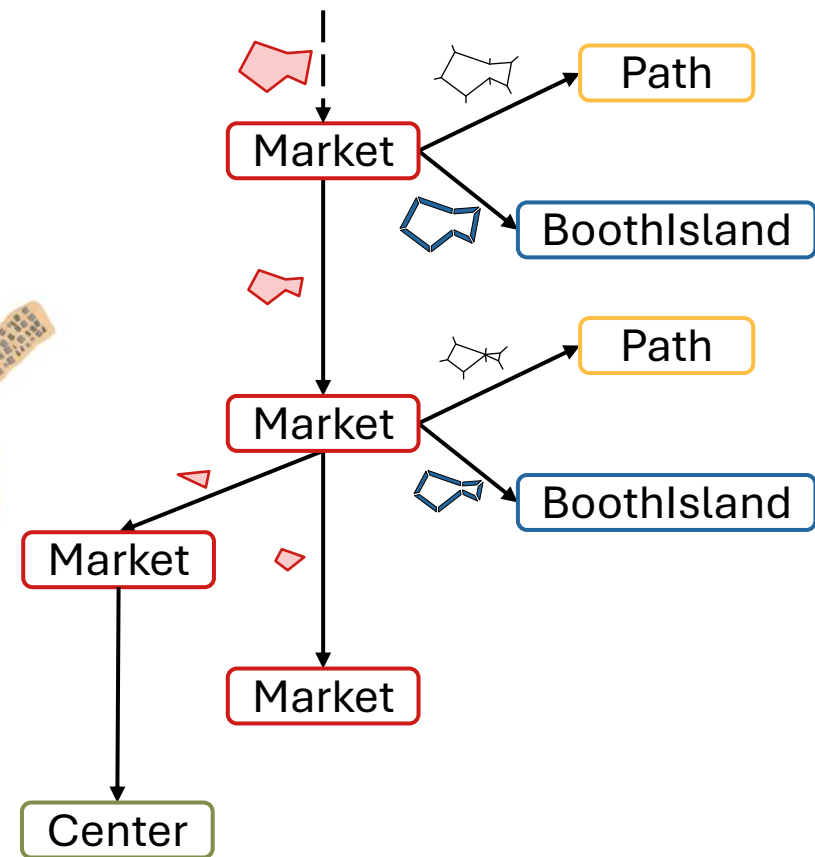
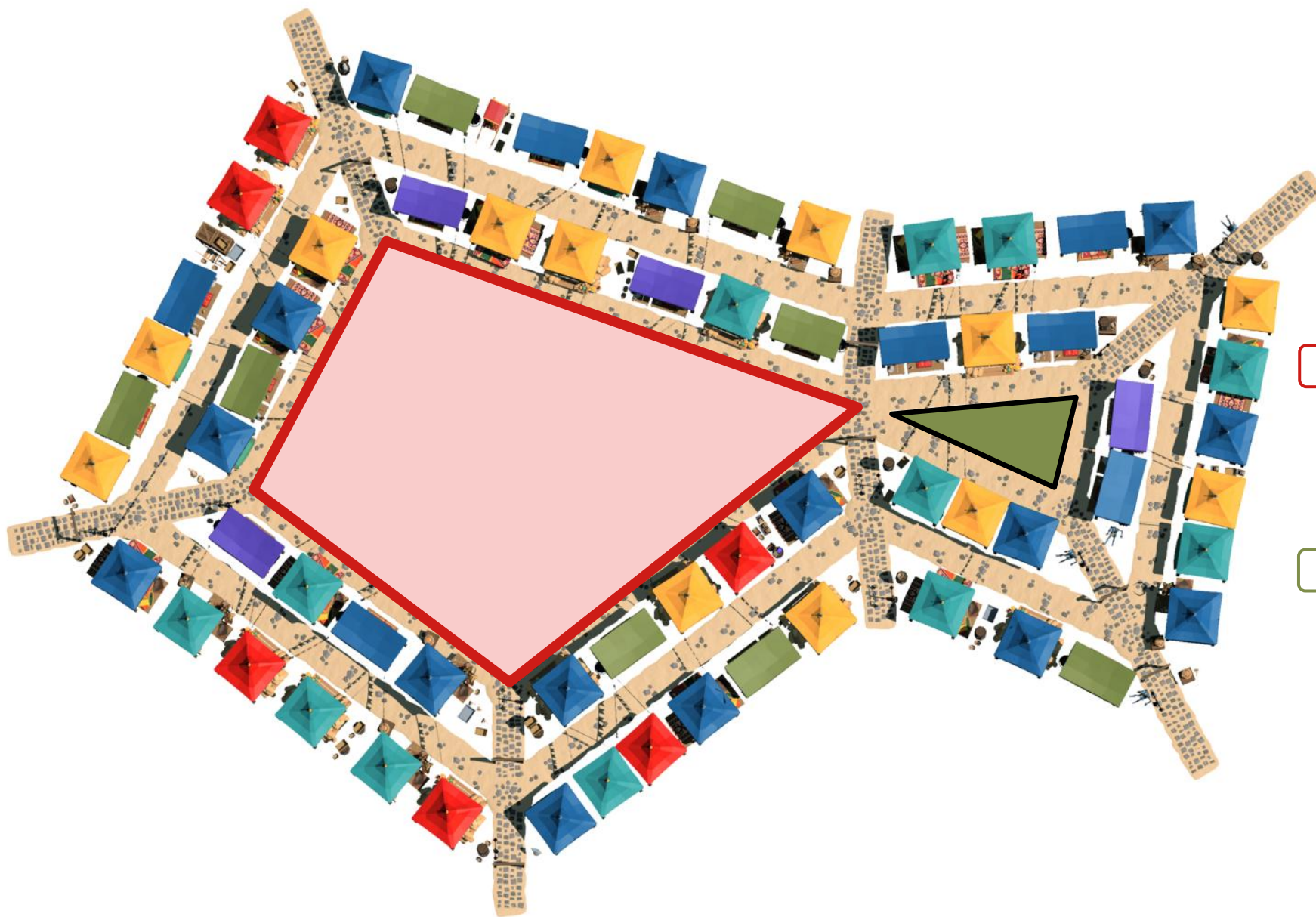


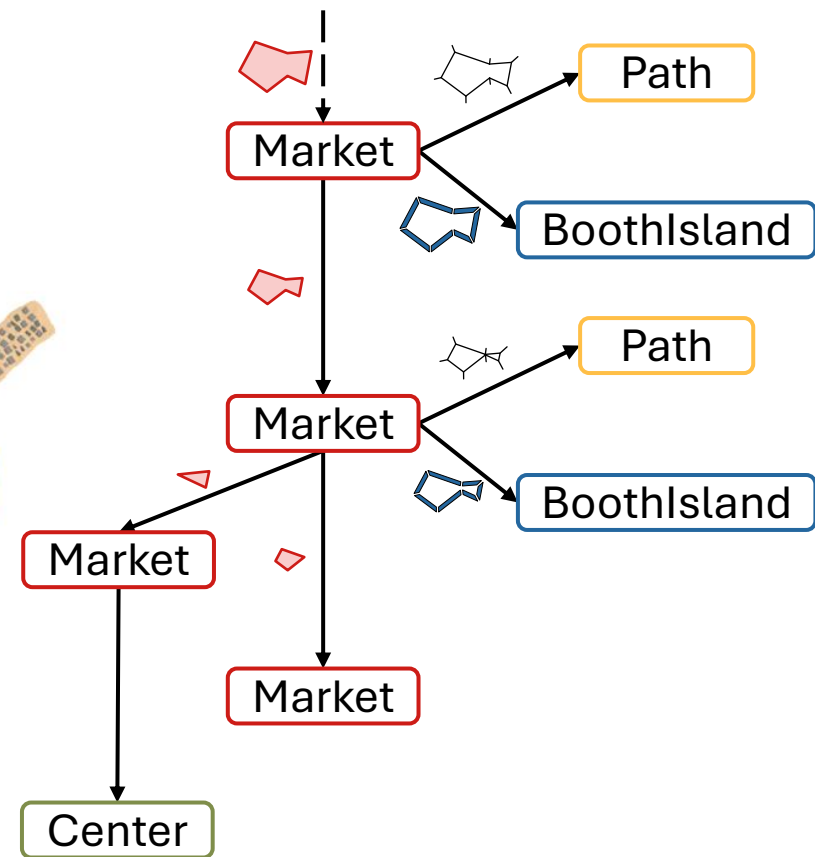
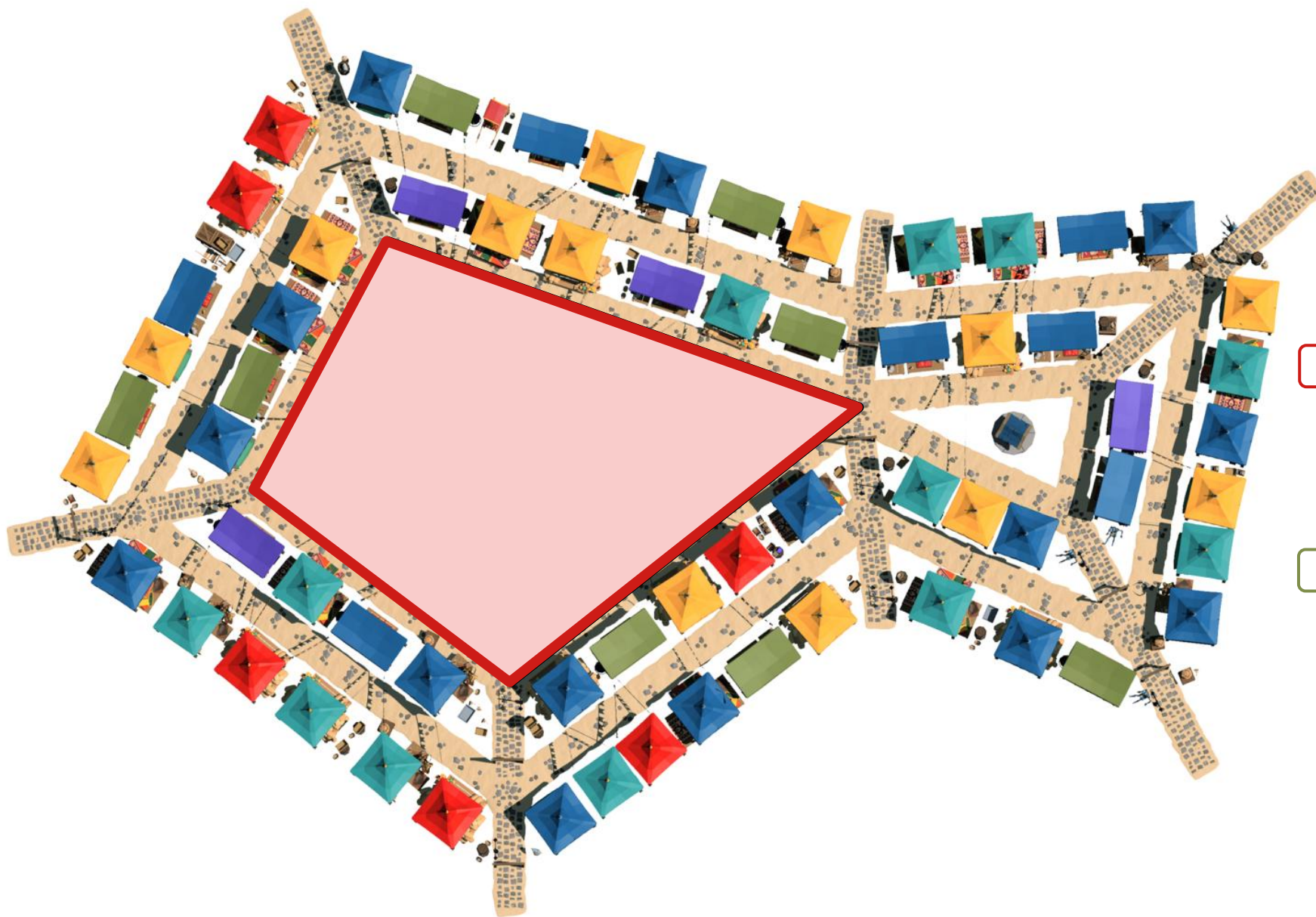


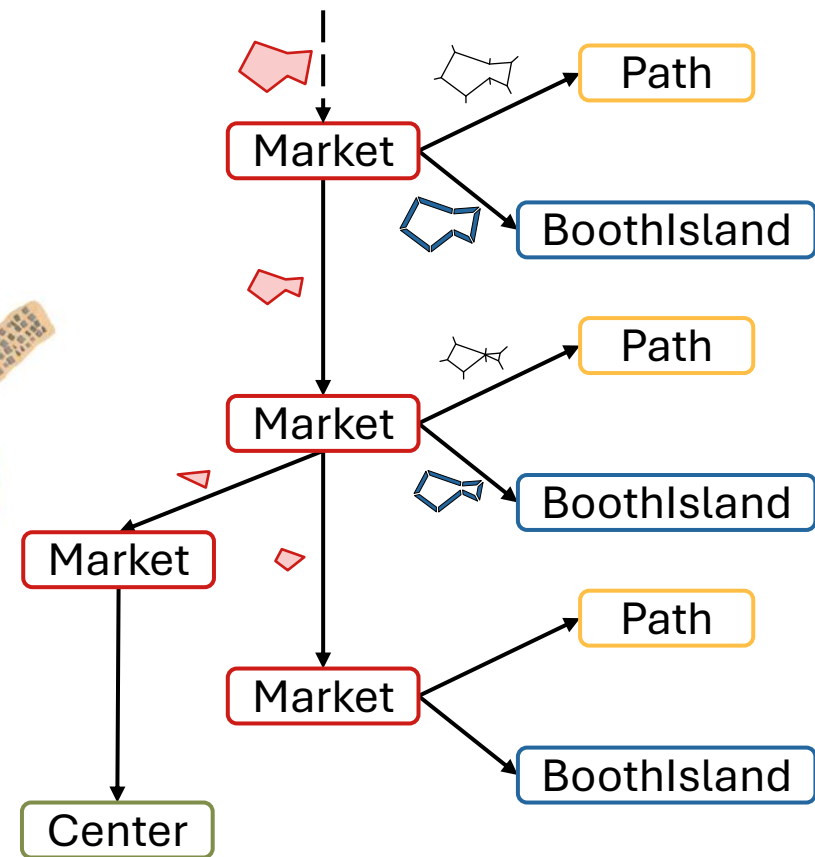
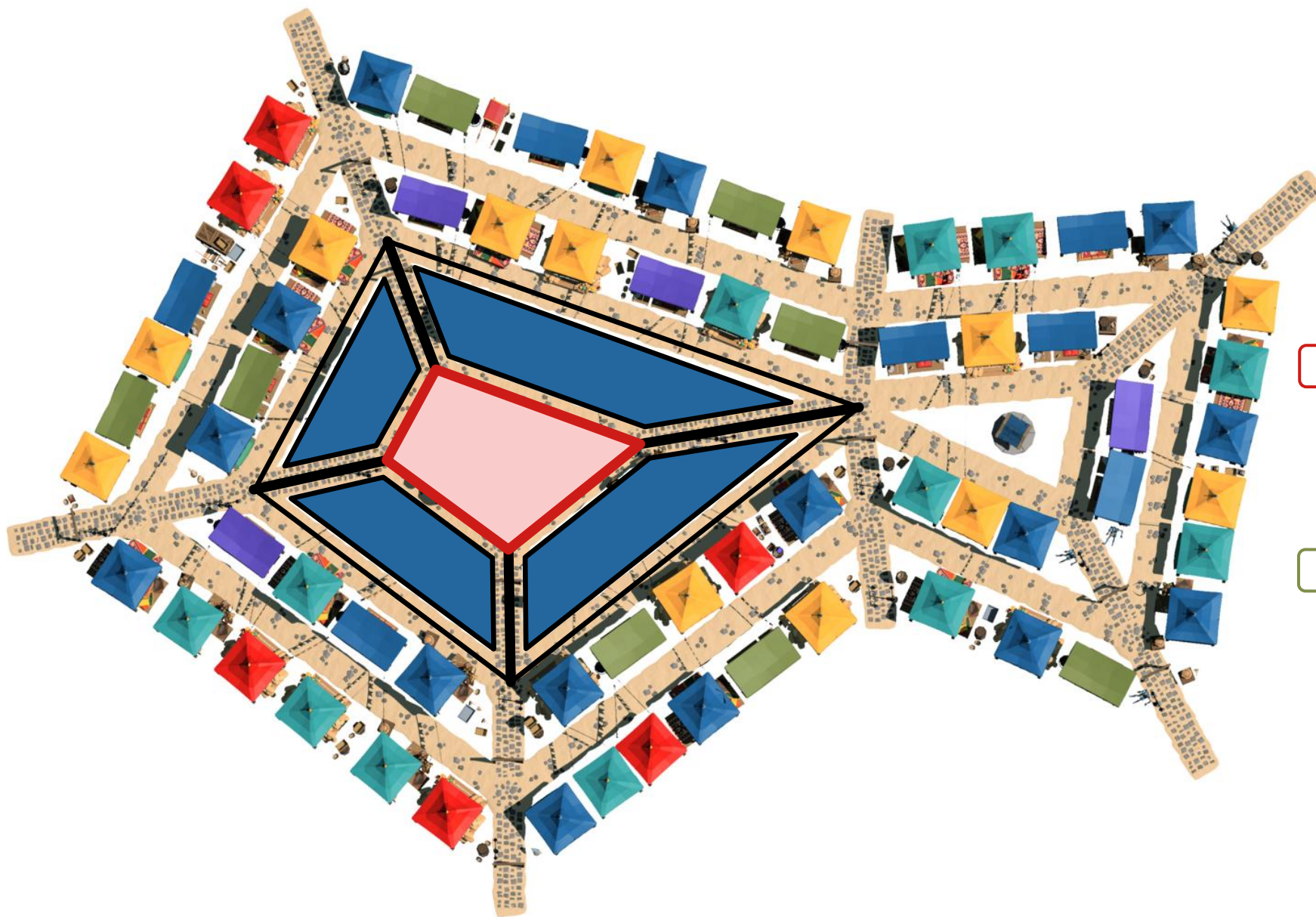


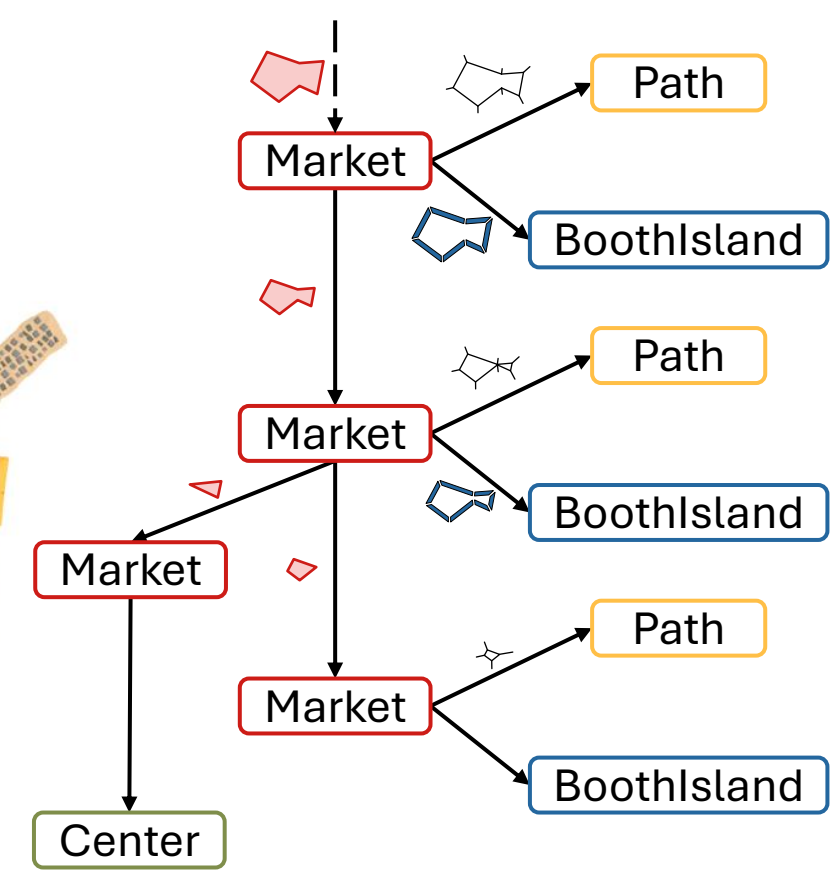
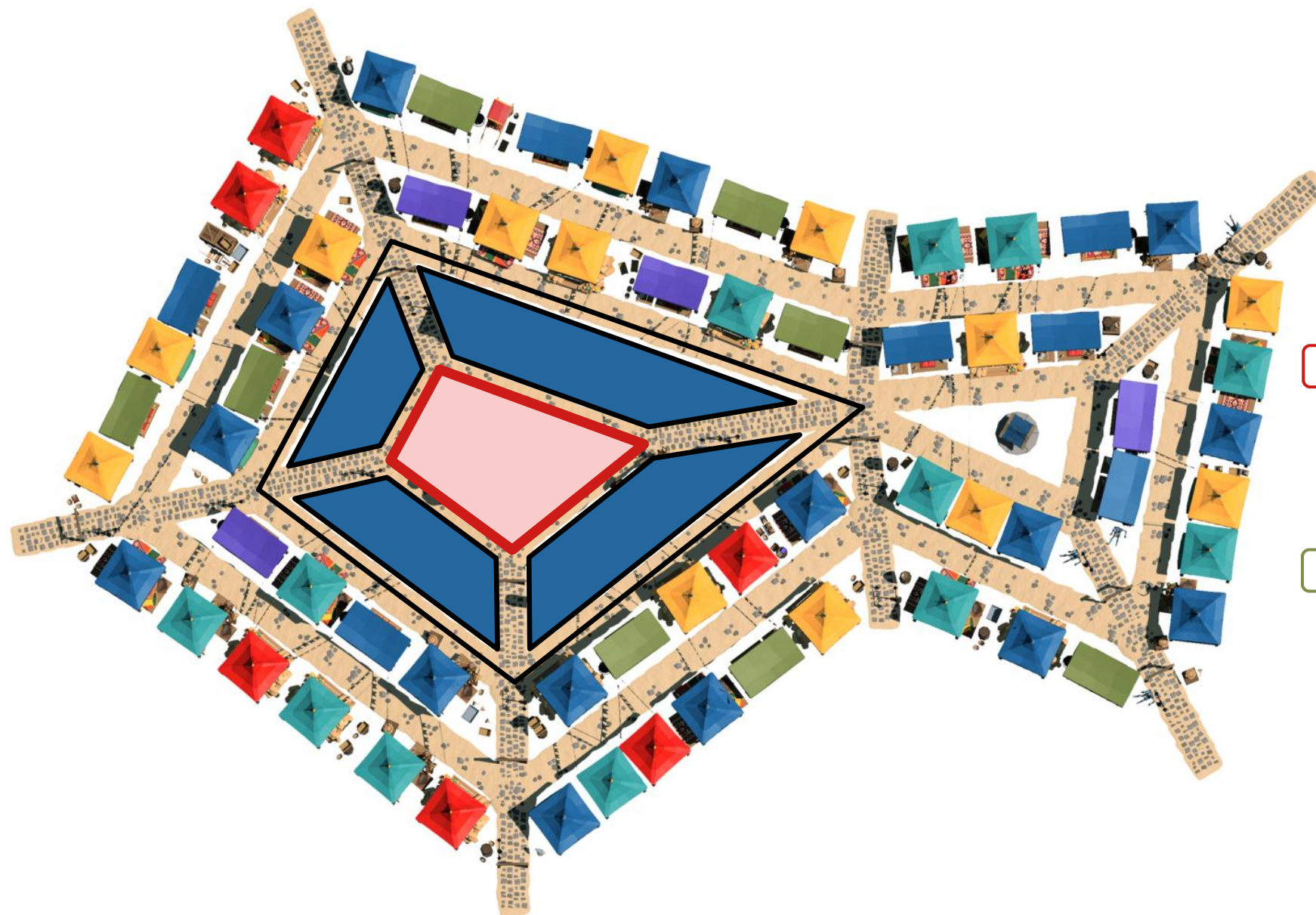


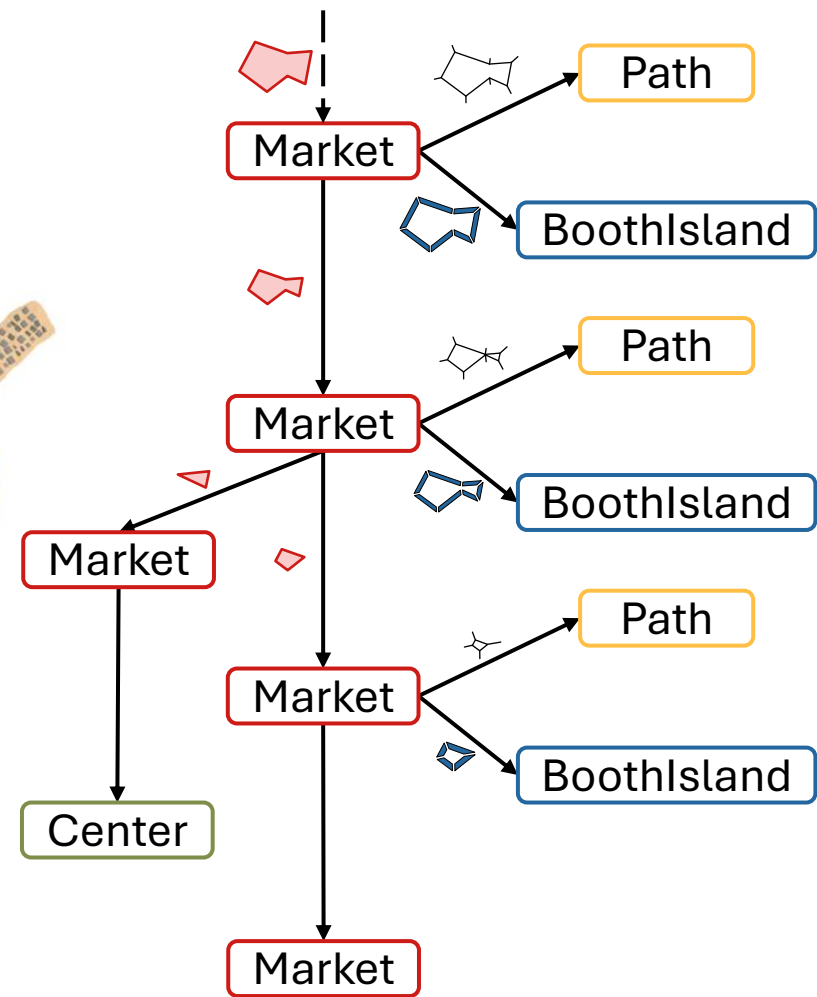
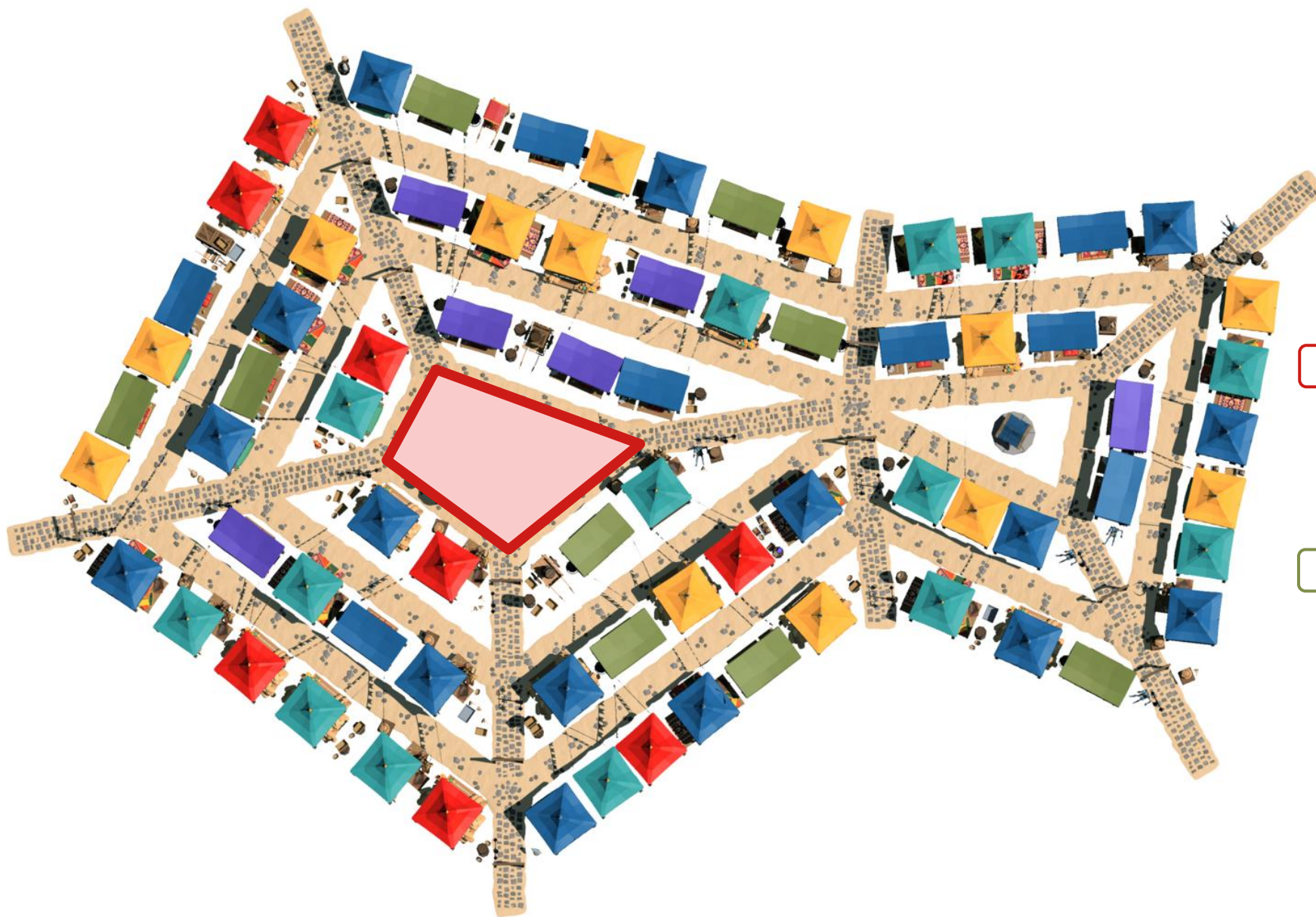


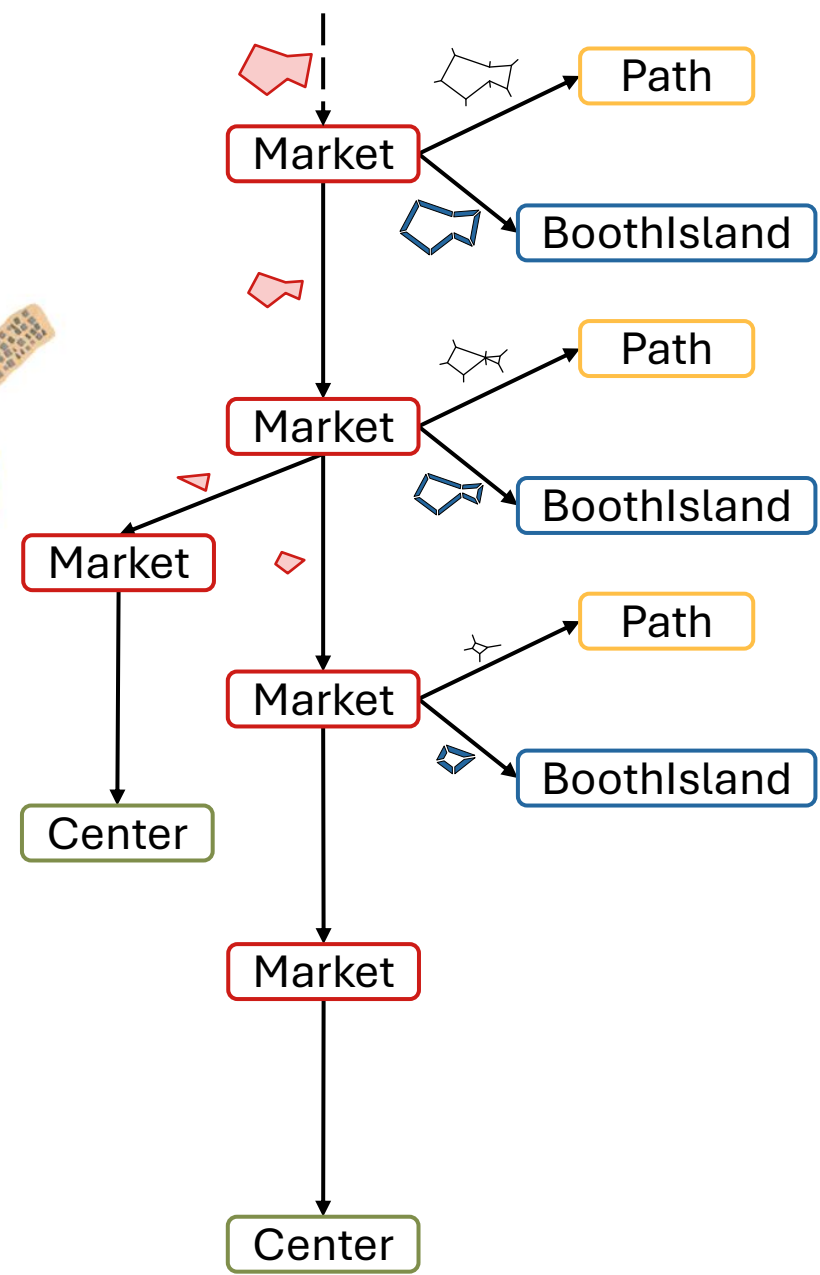


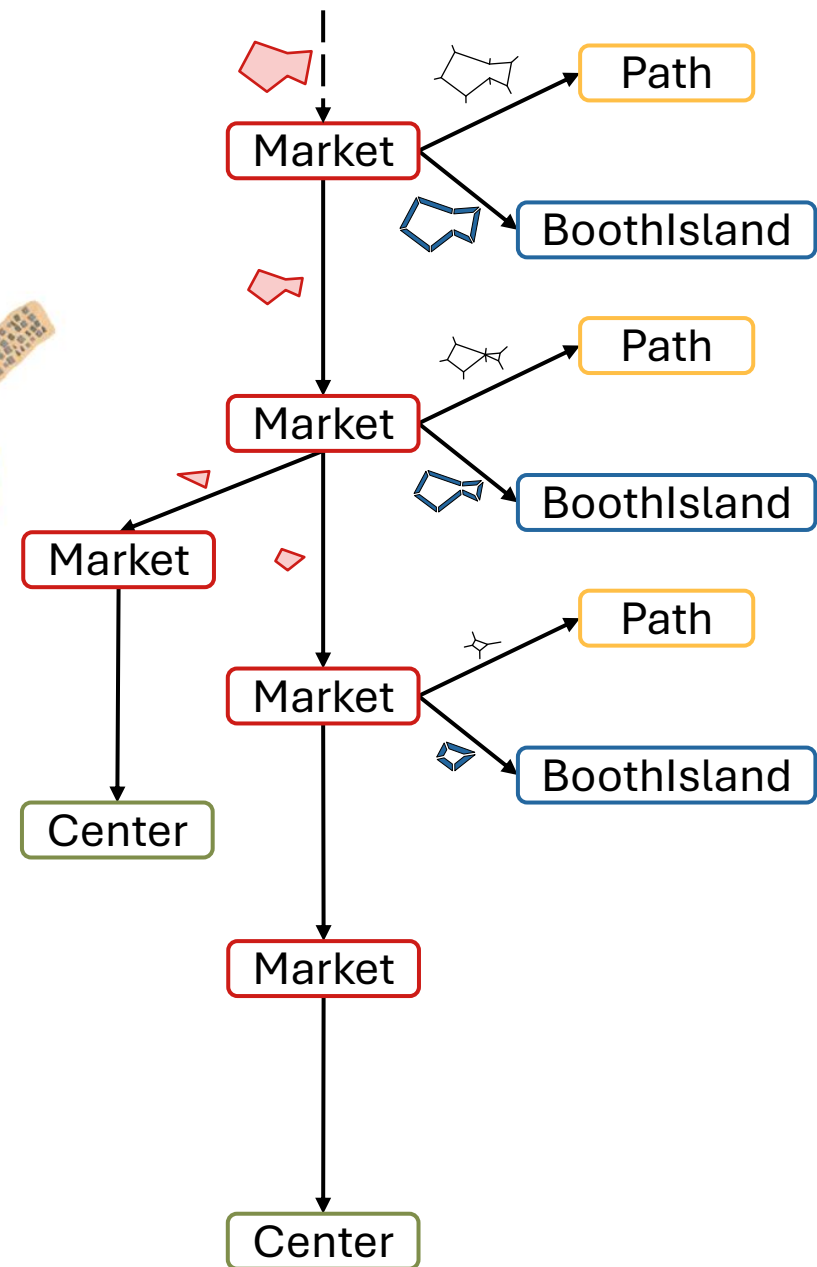


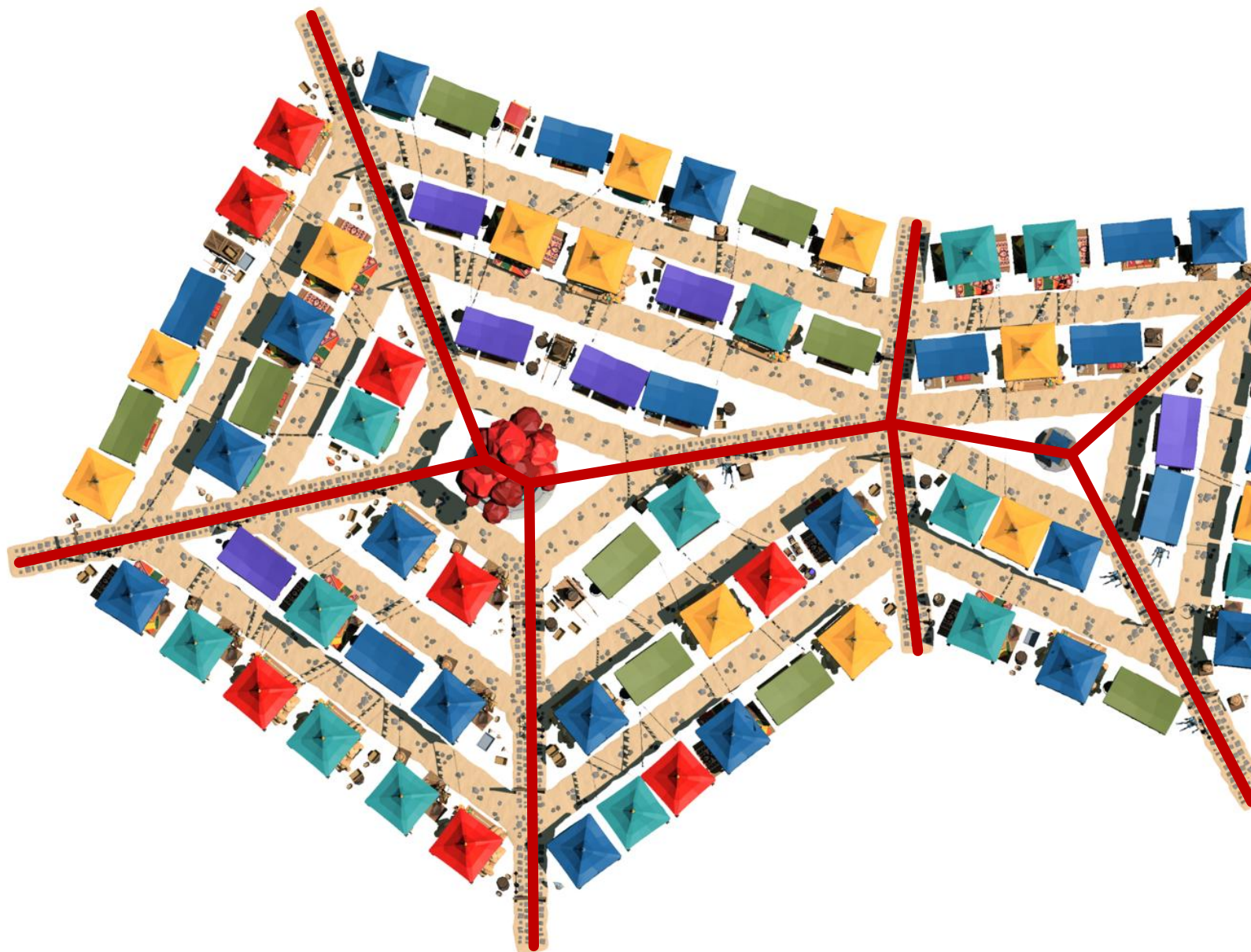












Journal of Universal Computer Science, vol. 1, no. 12 (1995), 752-761
 submitted: 17/11/95, accepted: 21/12/95, appeared: 28/12/95 © Springer Pub. Co.

A Novel Type of Skeleton for Polygons

Oswin Aichholzer
 Franz Aurenhammer
 Institute for Theoretical Computer Science
 Graz University of Technology
 Klosterwiesgasse 32/2, A-8010 Graz, Austria
 {oaich,auren}@igi.tu-graz.ac.at

David Alberts
 Bernd Gärtner
 Institut für Informatik
 Freie Universität Berlin
 Takustraße 9, D-14195 Berlin, Germany
 {alberts,gaertner}@inf.fu-berlin.de

Abstract A new internal structure for simple polygons, the straight skeleton, is introduced and discussed. It is composed of pieces of angular bisectors which partition the interior of a given n -gon P in a tree-like fashion into n monotone polygons. Its straight-line structure and its lower combinatorial complexity may make the straight skeleton preferable to the widely used medial axis of a polygon. As a seemingly unrelated application, the straight skeleton provides a canonical way of constructing a polygonal roof above a general layout of ground walls.

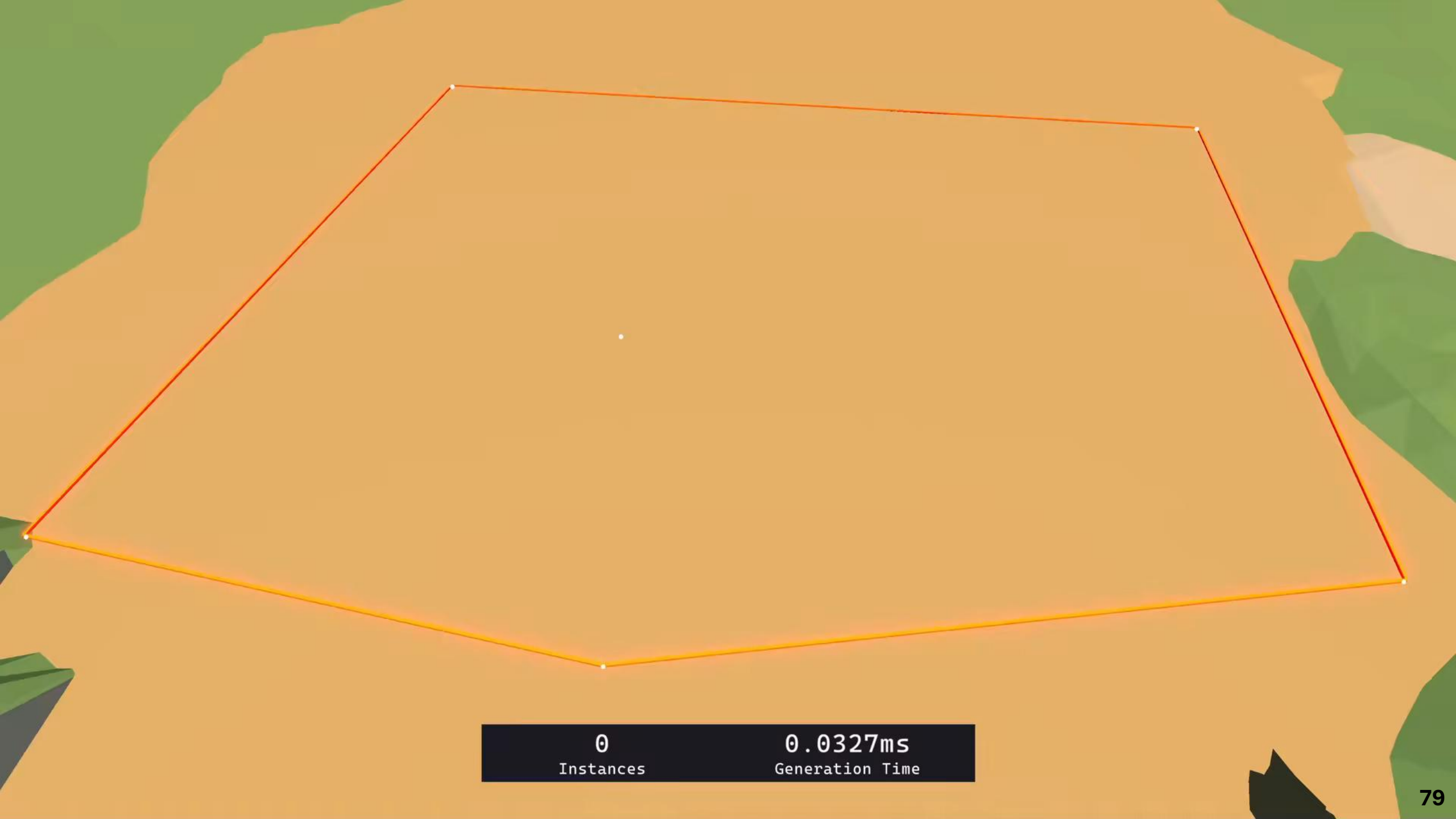
Keywords: Simple polygon, angular bisectors, internal skeleton, roof construction

1 Introduction and basic properties

The purpose of this paper is to introduce and discuss a new and interesting internal structure for simple polygons in the plane. The new structure, called the *straight skeleton*, is solely made up of straight line segments which are pieces of angular bisectors of polygon edges. It uniquely partitions the interior of a given n -gon P into n monotone polygons, one for each edge of P .

The straight skeleton, in general, differs from the well-known *medial axis* of P which consists of all interior points whose closest point on P 's boundary is not unique; see e.g. Lee [L]. If P is convex then both structures are identical. Otherwise, the medial axis contains parabolically curved segments in the neighborhood of reflex vertices of P which are avoided by the straight skeleton. If P is rectilinear then the straight skeleton is the medial axis of P for the L_∞ -metric. Skeletons have numerous applications inside and outside computer science as is documented e.g. in Kirkpatrick [K].

While the medial axis is a Voronoi-diagram-like concept, the straight skeleton is not defined using a distance function but rather by an appropriate *shrinking process* for P . Imagine that the boundary of P is contracted towards P 's interior, in a self-parallel manner and at the same speed for all edges. Lengths of edges might decrease or increase in this process. Each vertex of P moves along the angular bisector of its incident edges. This situation continues as long as the boundary does not change topologically. There are two possible types of changes:



0	0.0327ms
Instances	Generation Time



Requirements for our system:

- ☑ GPU only
- ☑ instant updates when editing
- ☑ processes control parameters
- ☑ hierarchy of reusable nodes

631

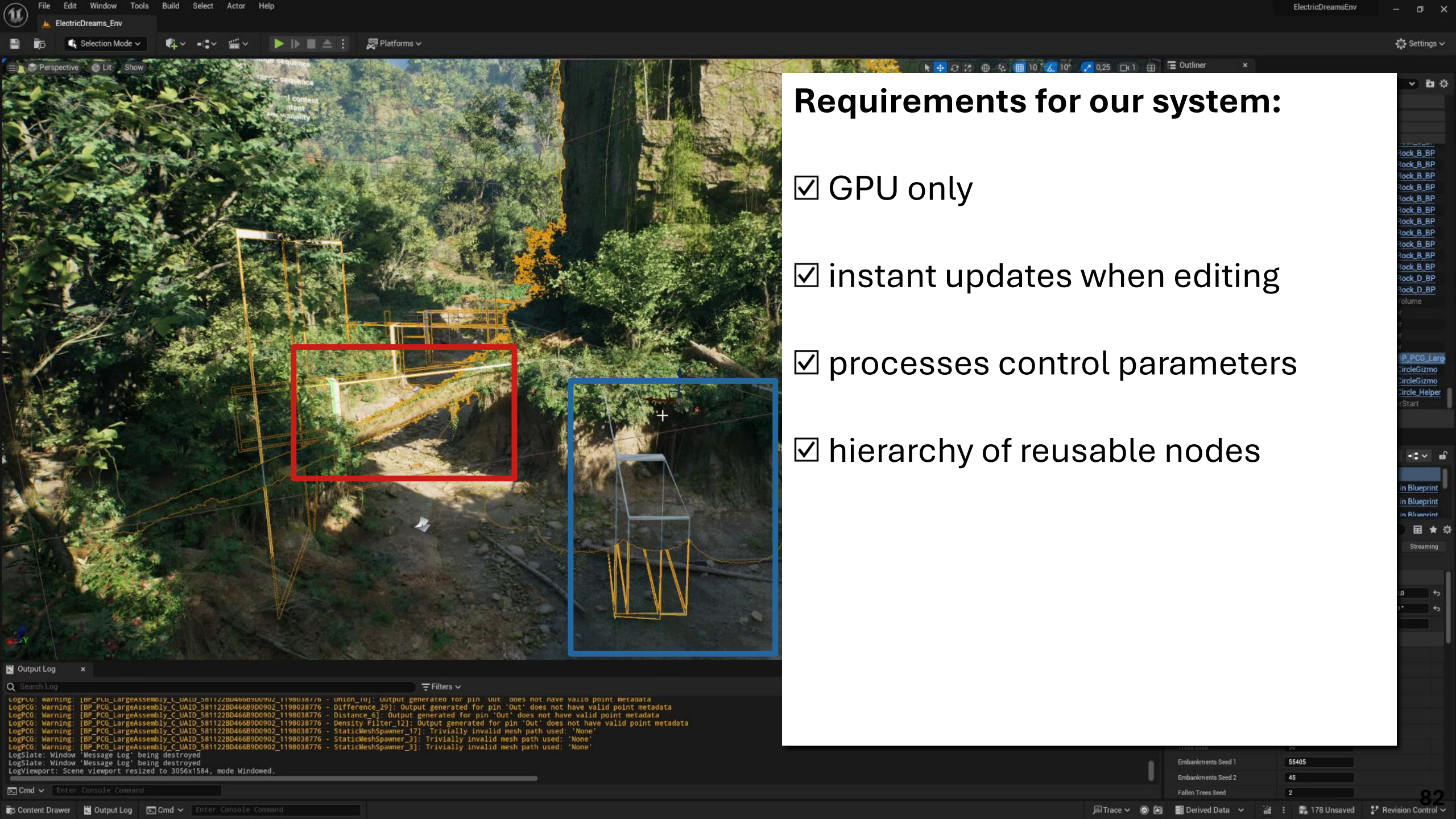
Instances

0.50ms

Generation Time

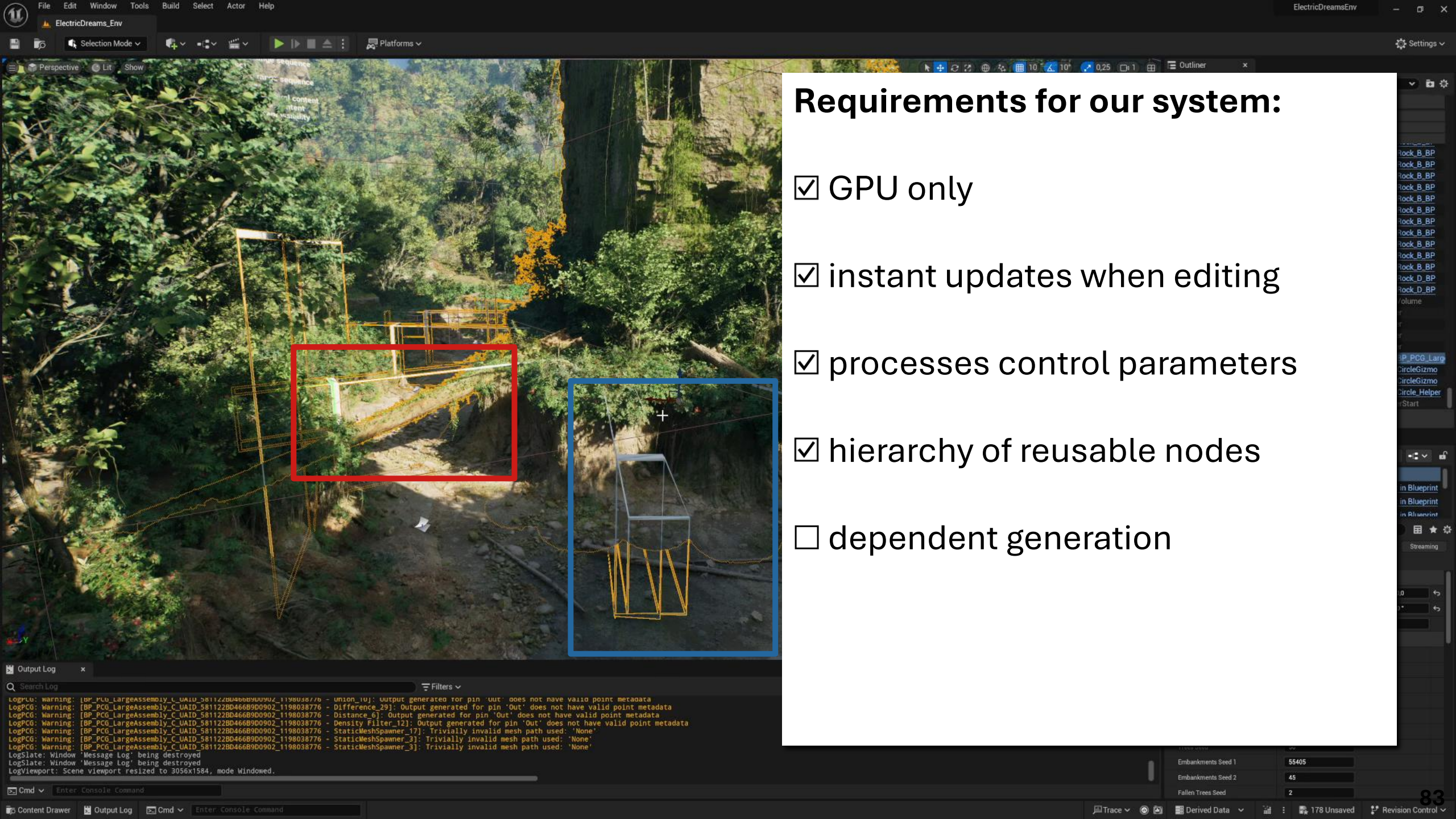


631	0.95ms
Instances	Generation Time



Requirements for our system:

- ☑ GPU only
- ☑ instant updates when editing
- ☑ processes control parameters
- ☑ hierarchy of reusable nodes



Requirements for our system:

- ☒ GPU only
- ☒ instant updates when editing
- ☒ processes control parameters
- ☒ hierarchy of reusable nodes
- ☐ dependent generation

A spatial GPU data-structure... ...fast creation and update time... ...and fast access



Raytracing BVH!

```
commandList->BuildRaytracingAccelerationStructure(&buildDesc, 0, nullptr);
```

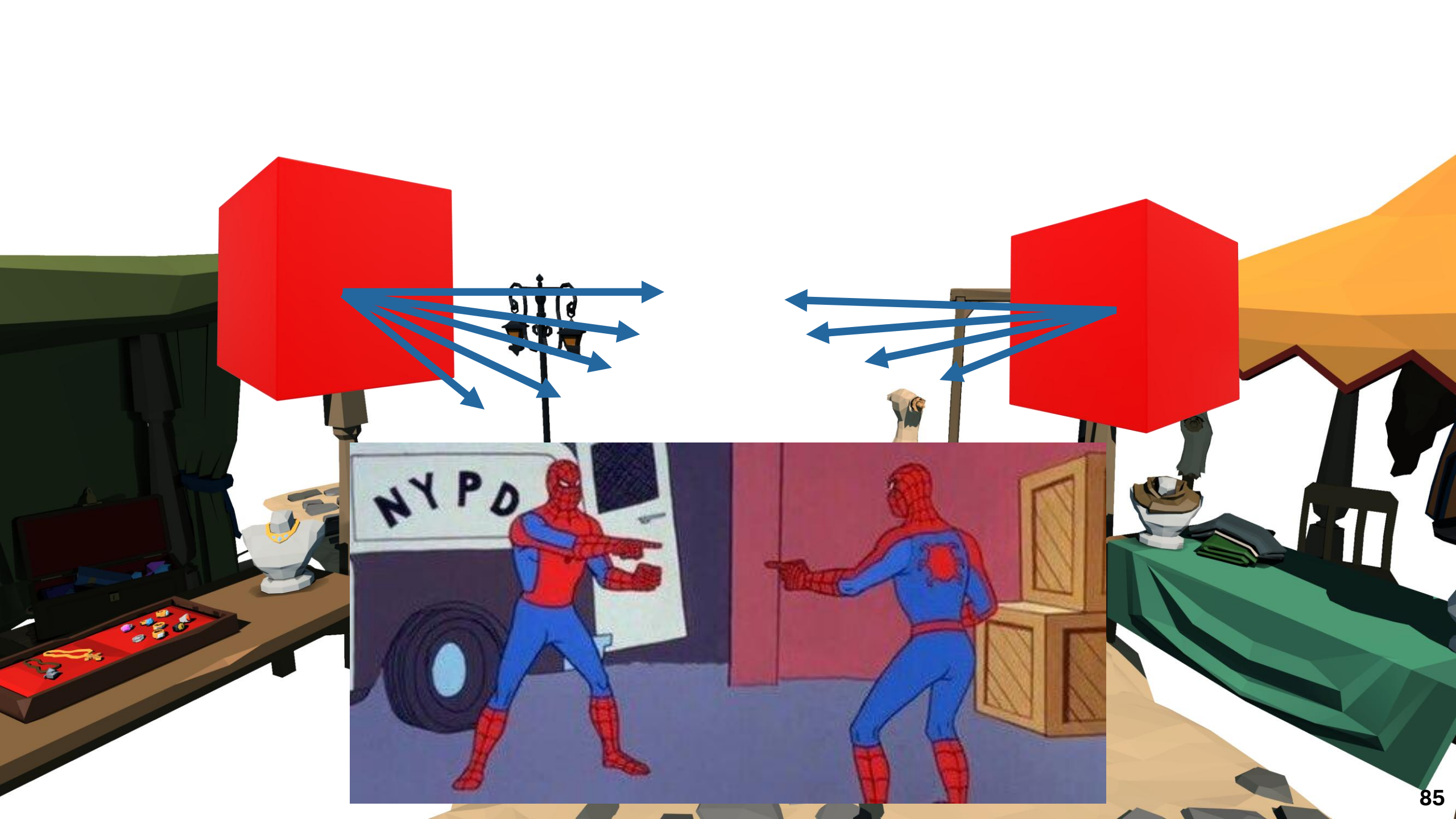
Draw List

```
int index;  
InterlockedAdd(drawMeshArgumentCounter, 1, index);  
drawMeshArguments[index] = args;
```

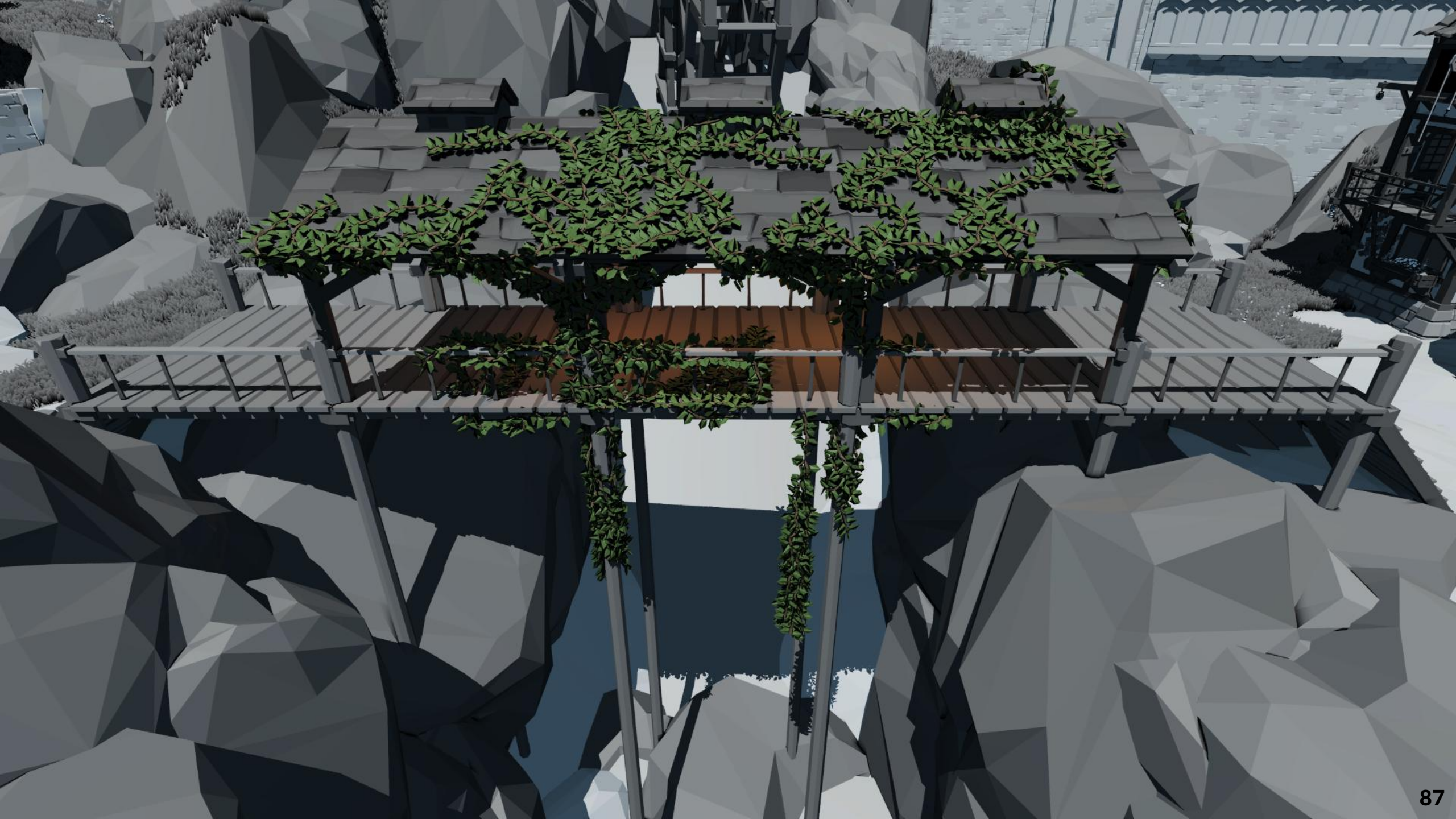
BVH Instance List

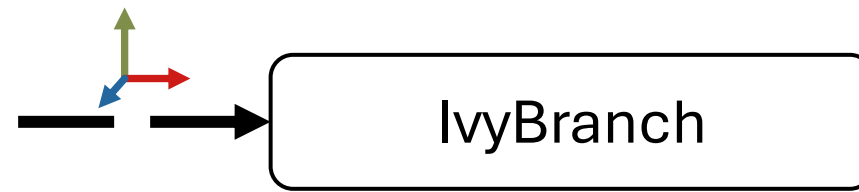
Mesh Nodes

```
[Shader("node")]  
[NodeLaunch("mesh")]  
[OutputTopology("triangle")]  
...  
void MeshShader(...)
```

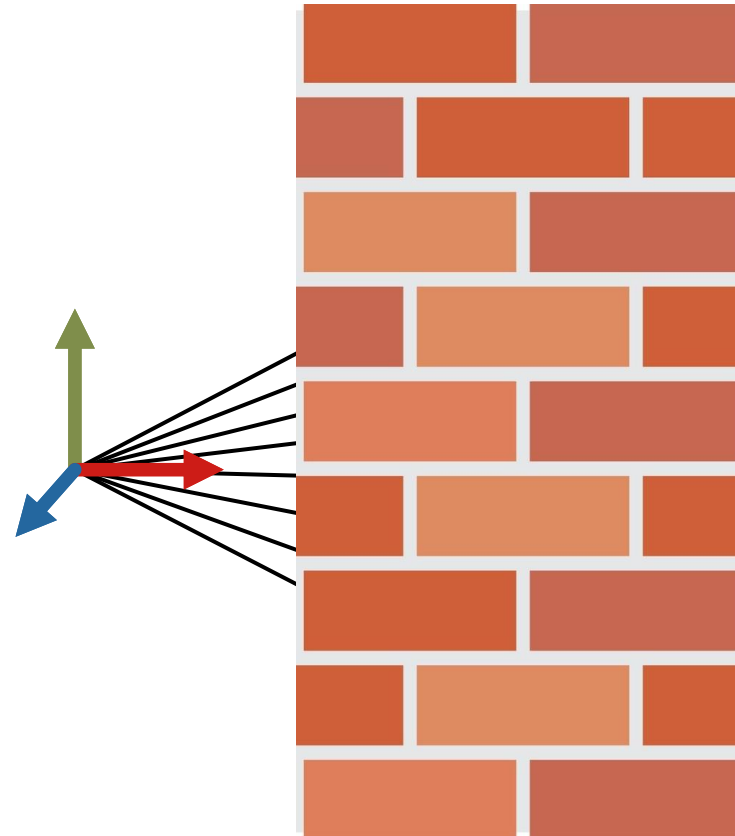


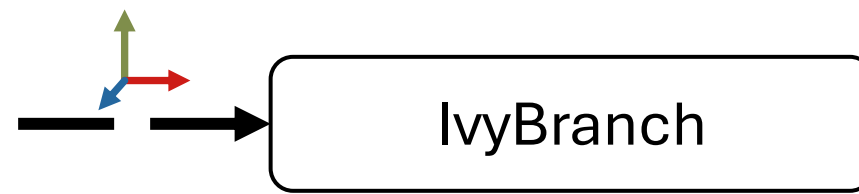




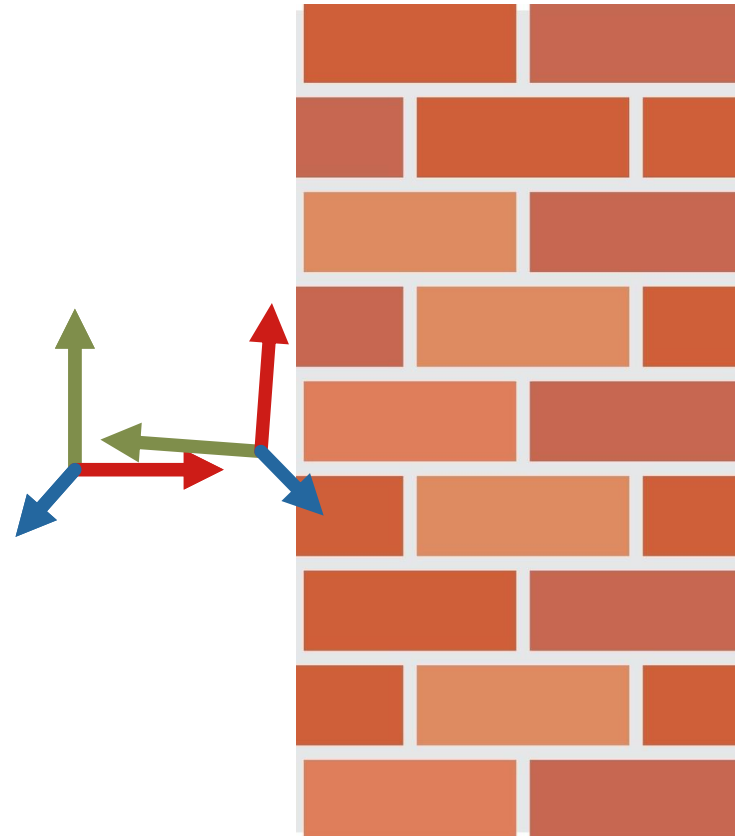


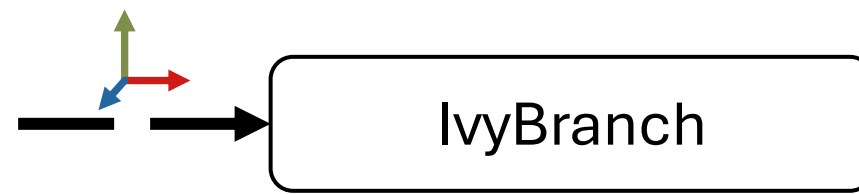
```
while(growing){  
    shootRays();  
    updatePosition();  
    drawAssets();  
    if(random) split();  
}
```



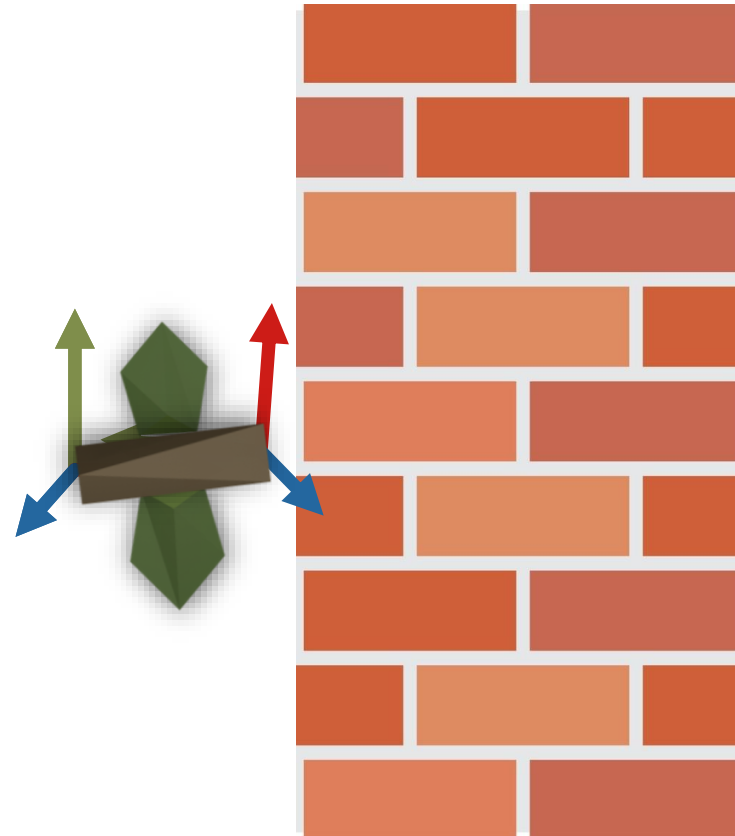


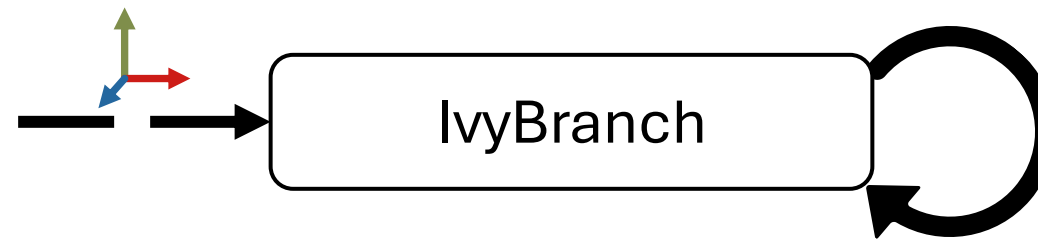
```
while(growing){  
    shootRays();  
    updatePosition();  
    drawAssets();  
    if(random) split();  
}
```



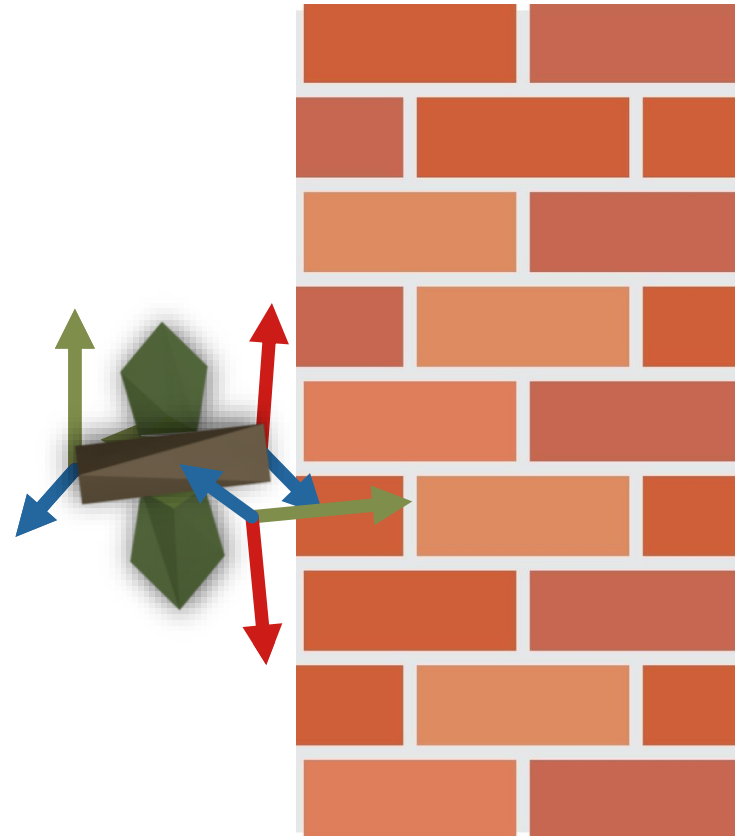


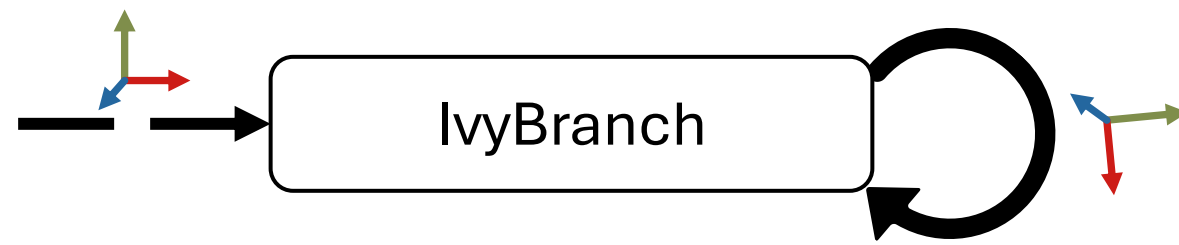
```
while(growing){  
    shootRays();  
    updatePosition();  
    drawAssets();  
    if(random) split();  
}
```



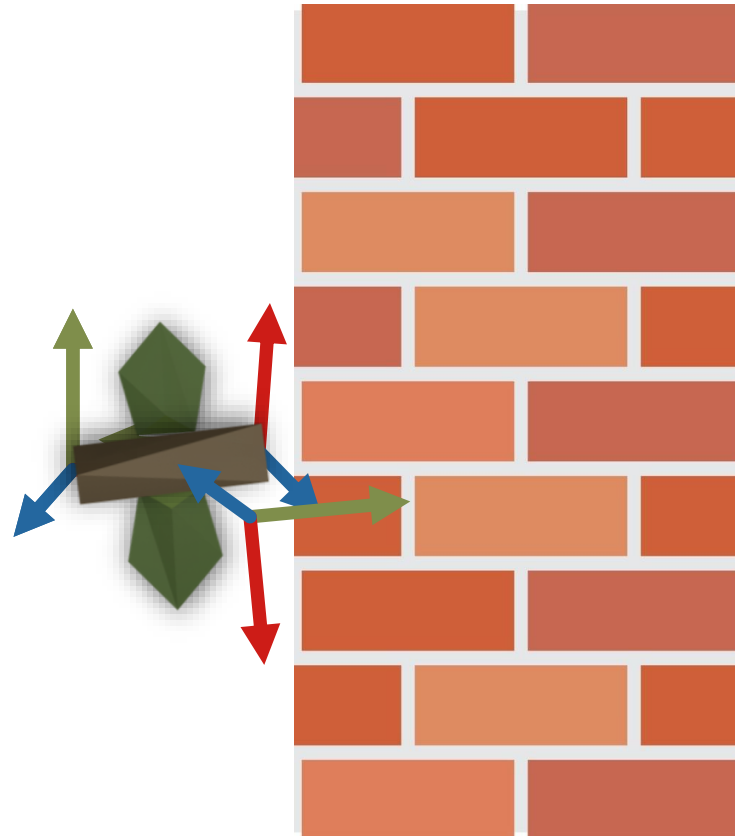


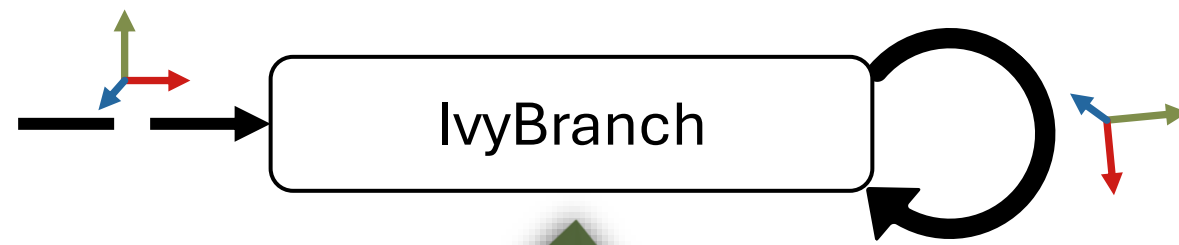
```
while(growing){  
    shootRays();  
    updatePosition();  
    drawAssets();  
    if(random) split();  
}
```



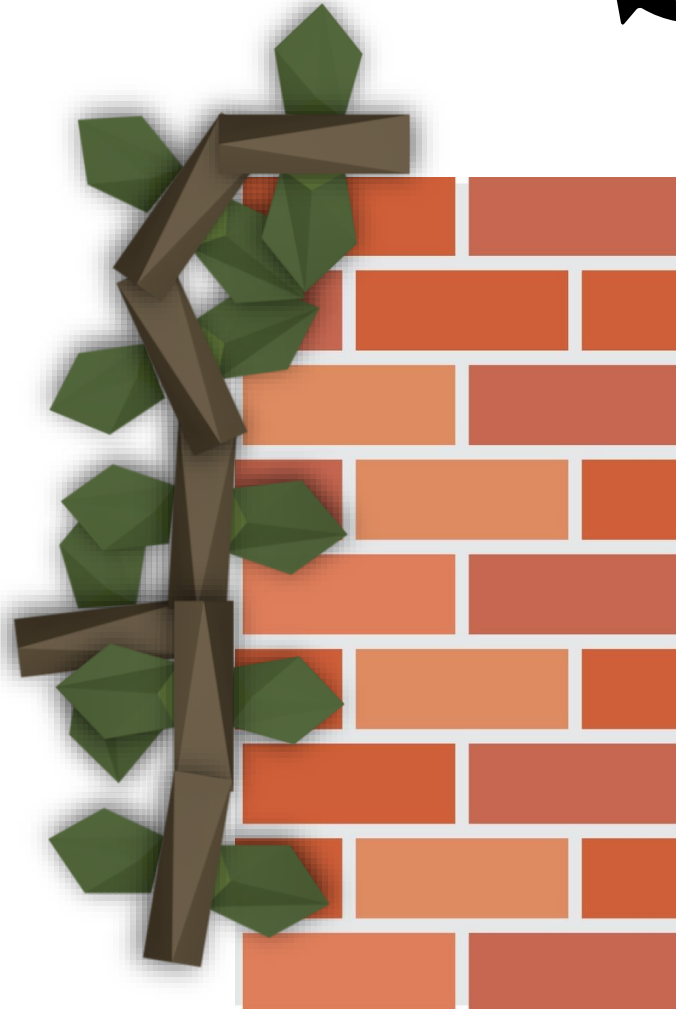


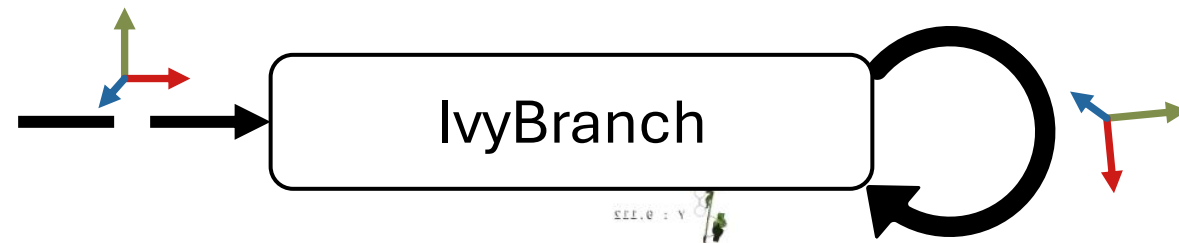
```
while(growing){  
    shootRays();  
    updatePosition();  
    drawAssets();  
    if(random) split();  
}
```





```
while(growing){  
    shootRays();  
    updatePosition();  
    drawAssets();  
    if(random) split();  
}
```

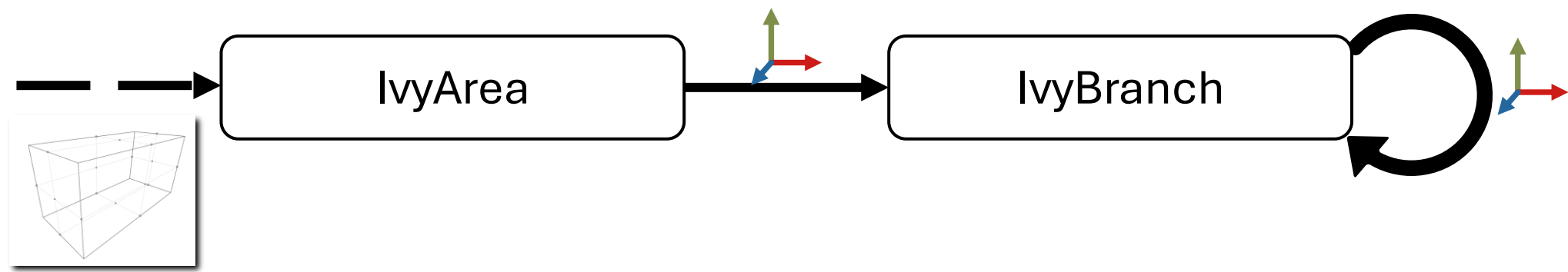


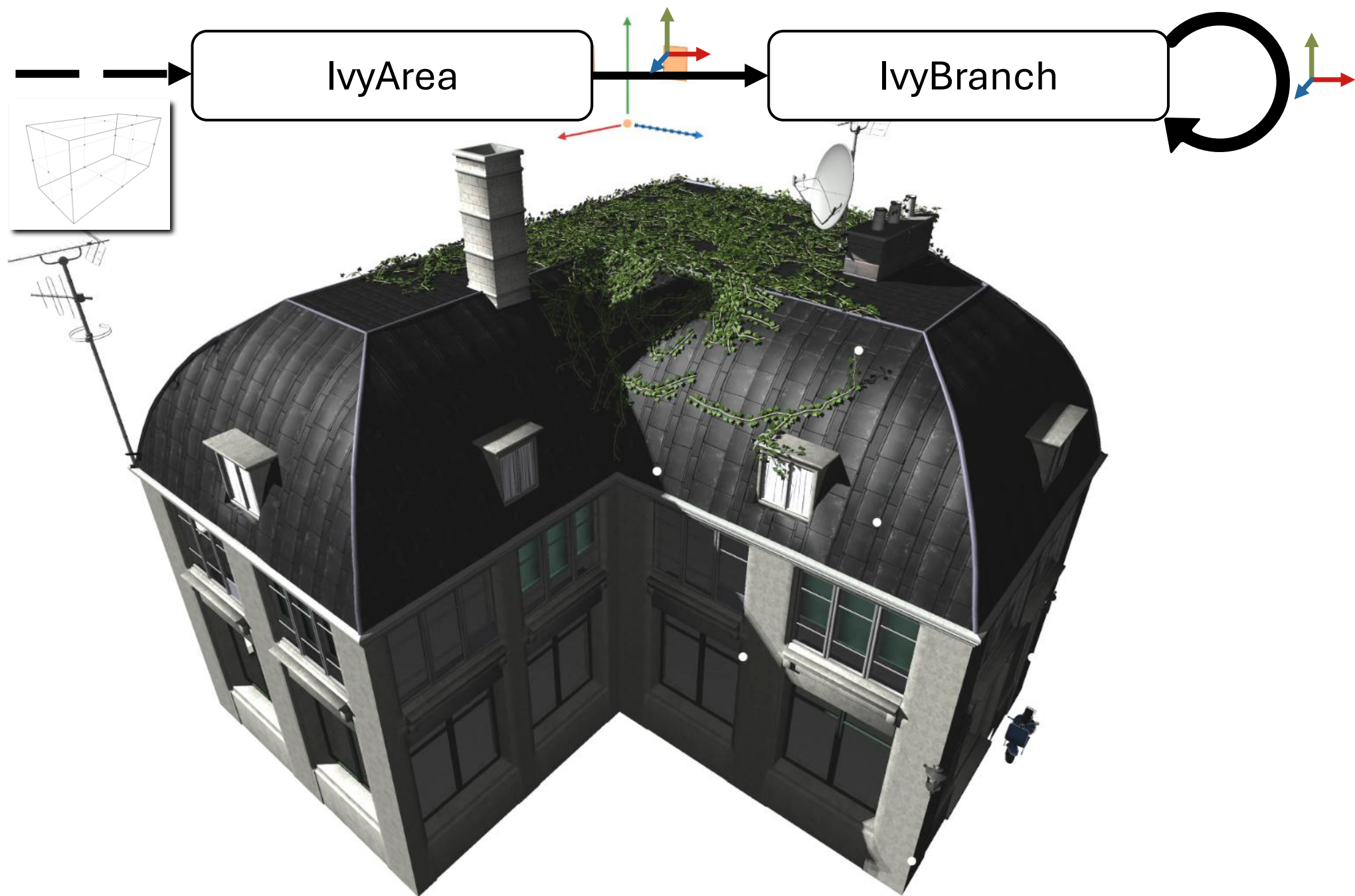


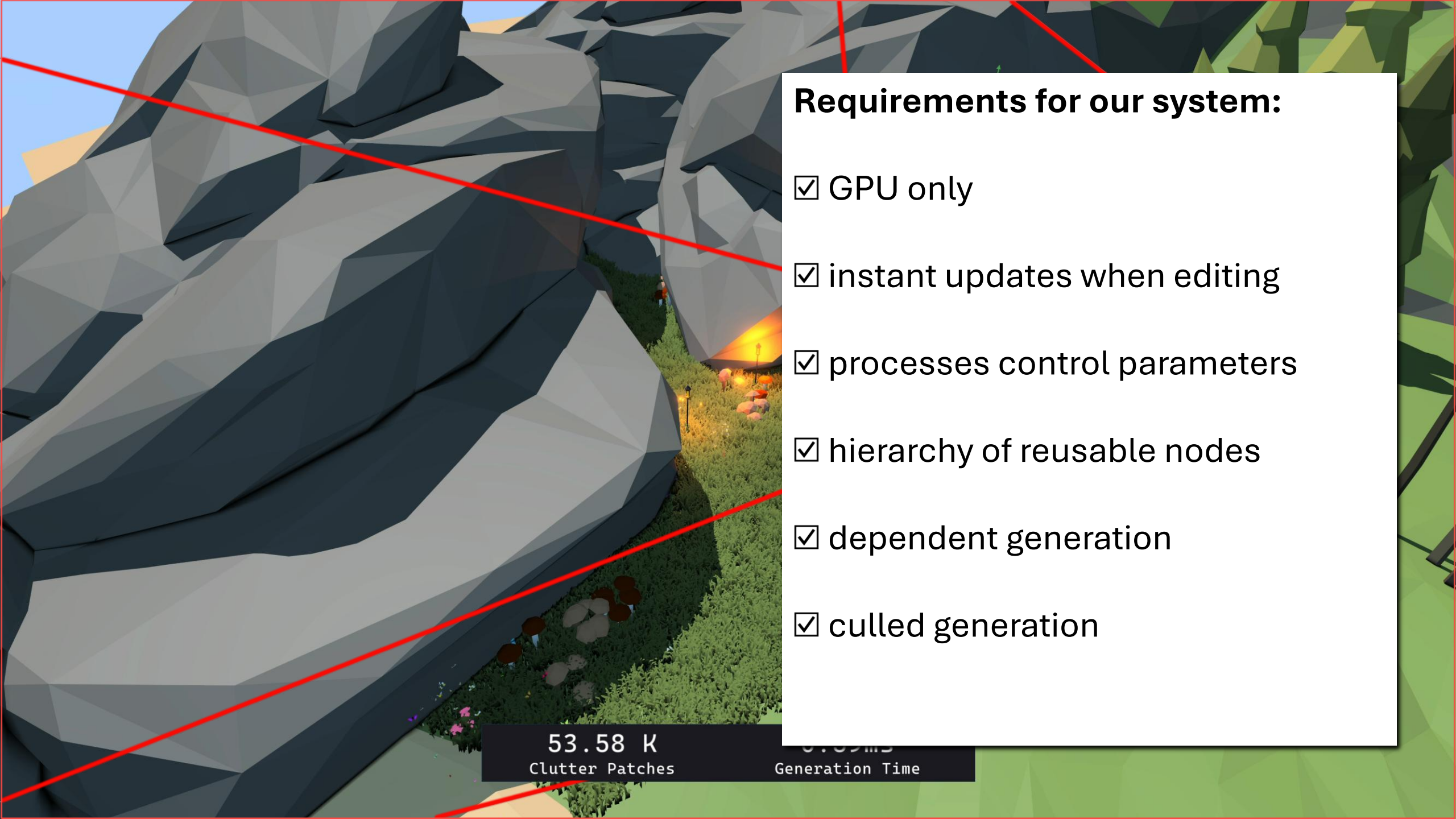
```
while(growing){  
  shootRays();  
  updatePosition();  
  drawAssets();  
  if(random) split();  
}
```









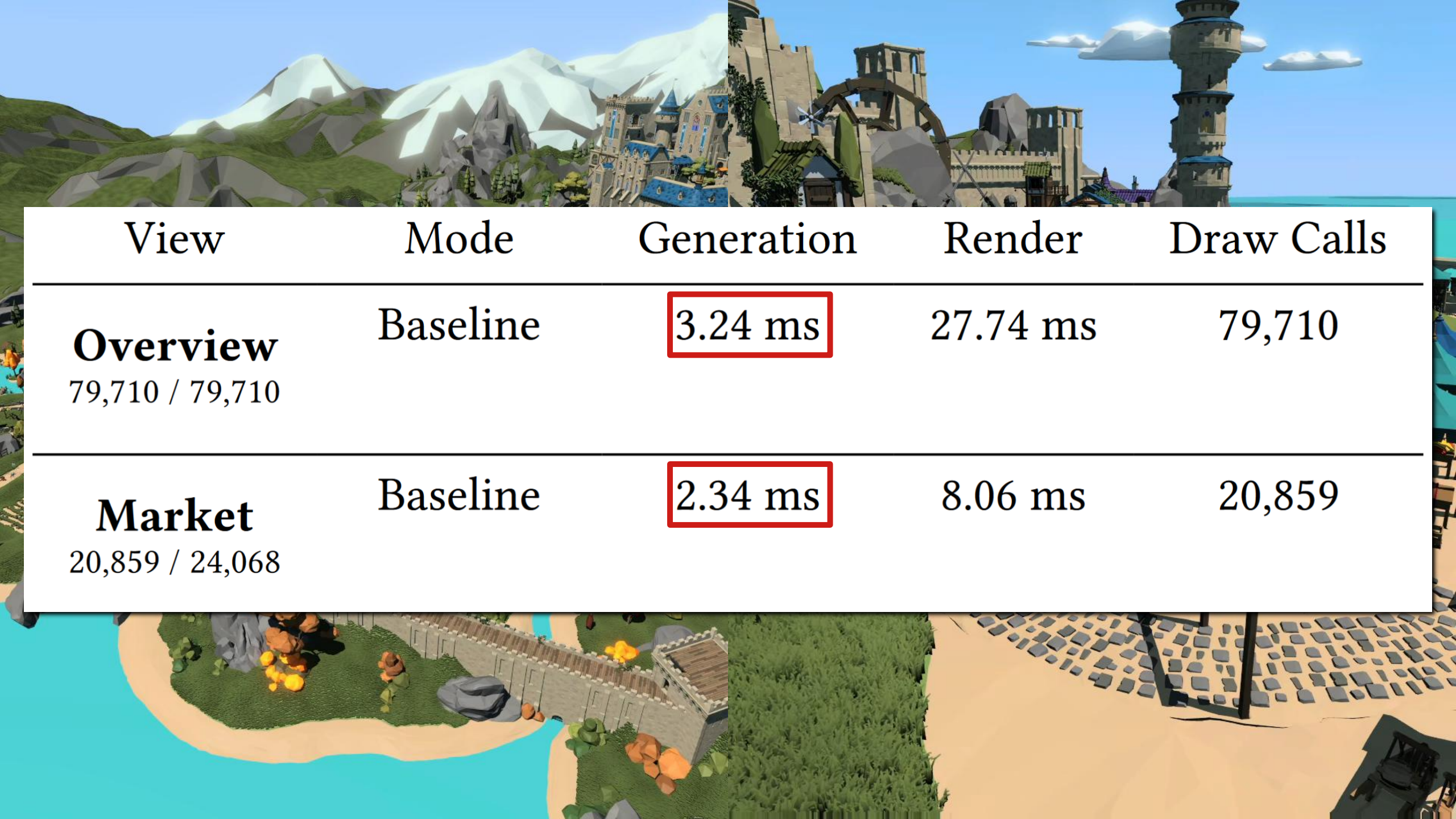


Requirements for our system:

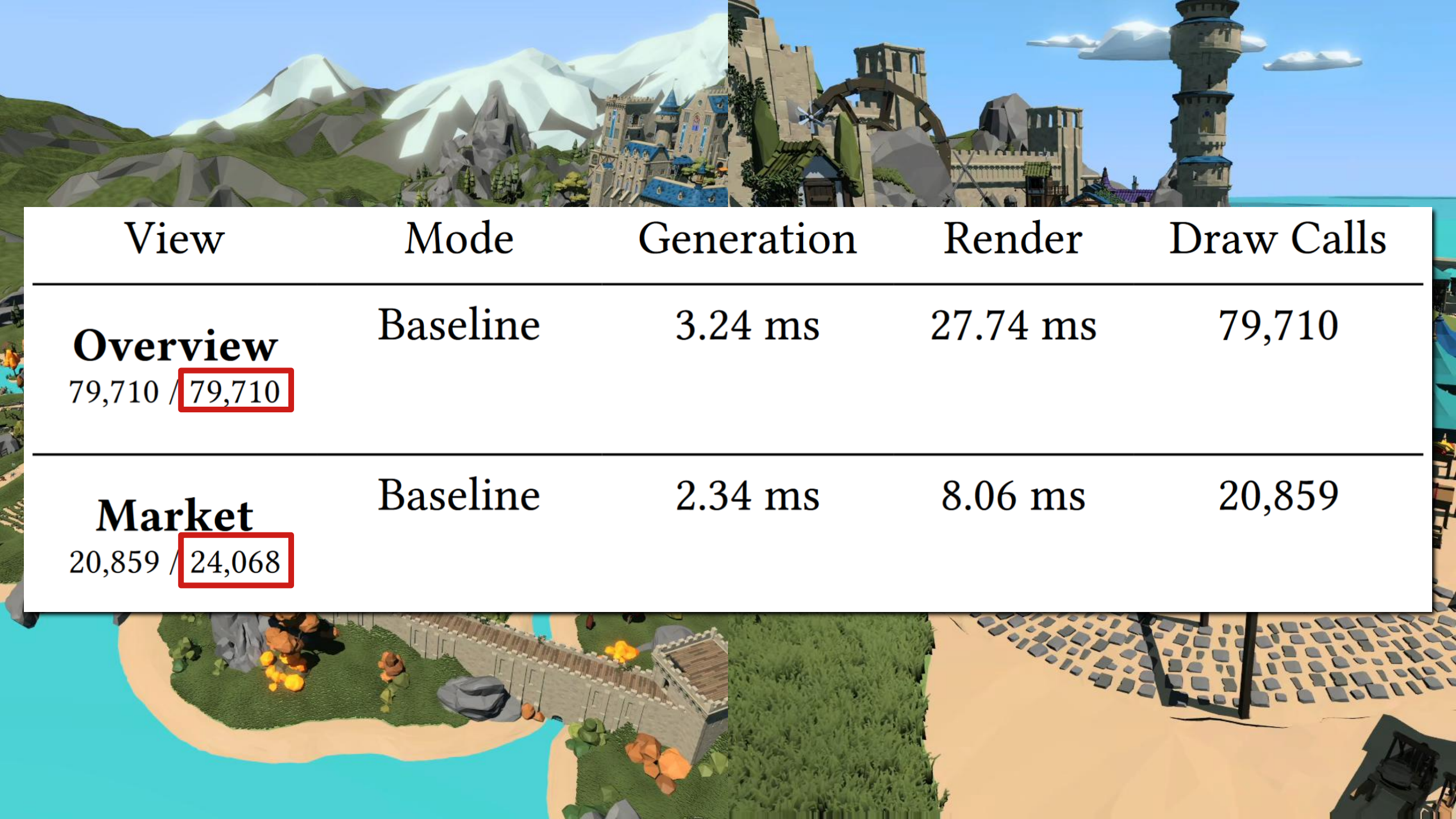
- ☒ GPU only
- ☒ instant updates when editing
- ☒ processes control parameters
- ☒ hierarchy of reusable nodes
- ☒ dependent generation
- ☒ culled generation

53.58 K
Clutter Patches

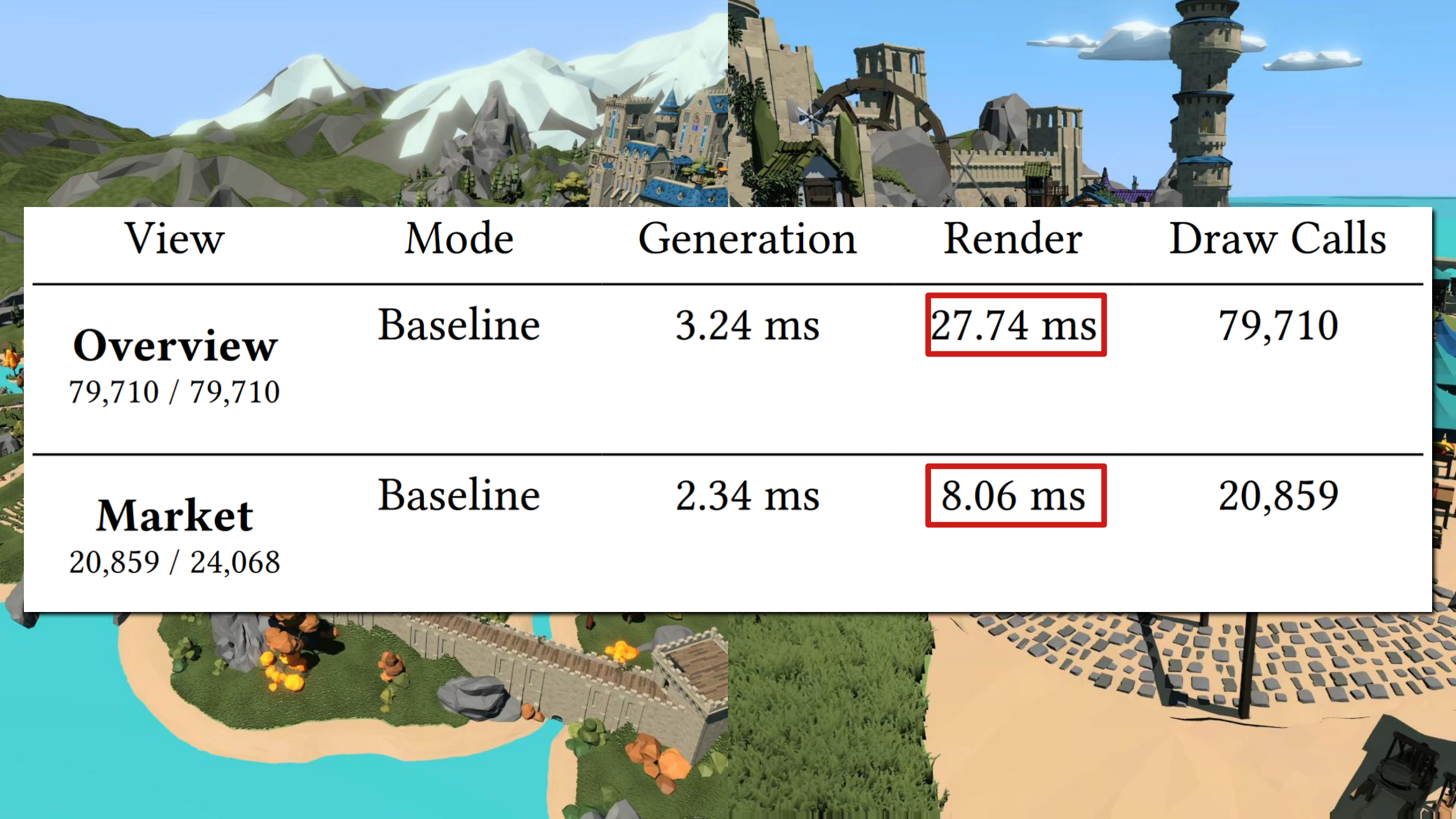
0.00ms
Generation Time



View	Mode	Generation	Render	Draw Calls
Overview 79,710 / 79,710	Baseline	3.24 ms	27.74 ms	79,710
Market 20,859 / 24,068	Baseline	2.34 ms	8.06 ms	20,859



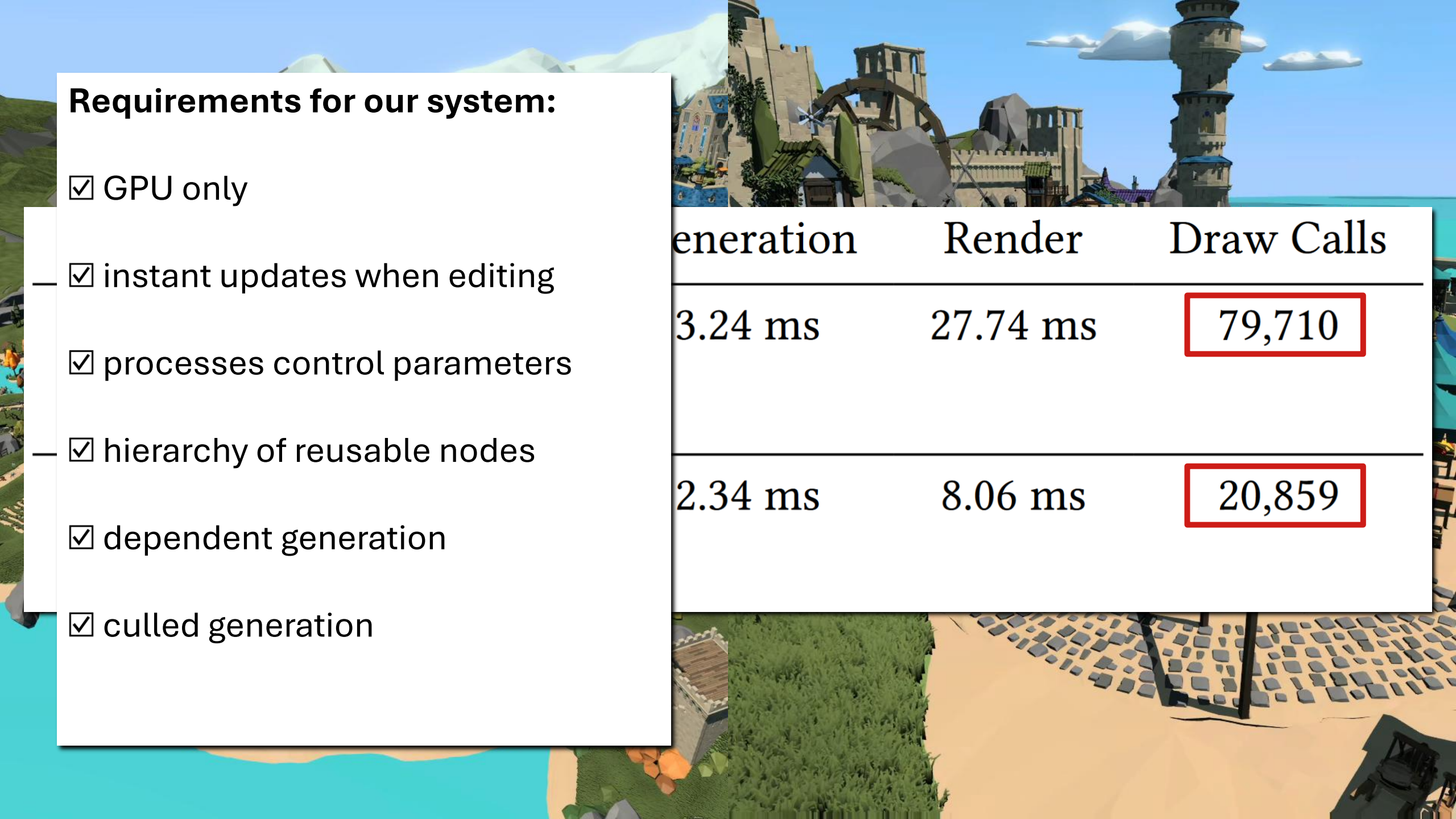
View	Mode	Generation	Render	Draw Calls
Overview 79,710 / 79,710	Baseline	3.24 ms	27.74 ms	79,710
Market 20,859 / 24,068	Baseline	2.34 ms	8.06 ms	20,859



View	Mode	Generation	Render	Draw Calls
Overview 79,710 / 79,710	Baseline	3.24 ms	27.74 ms	79,710
Market 20,859 / 24,068	Baseline	2.34 ms	8.06 ms	20,859

Requirements for our system:

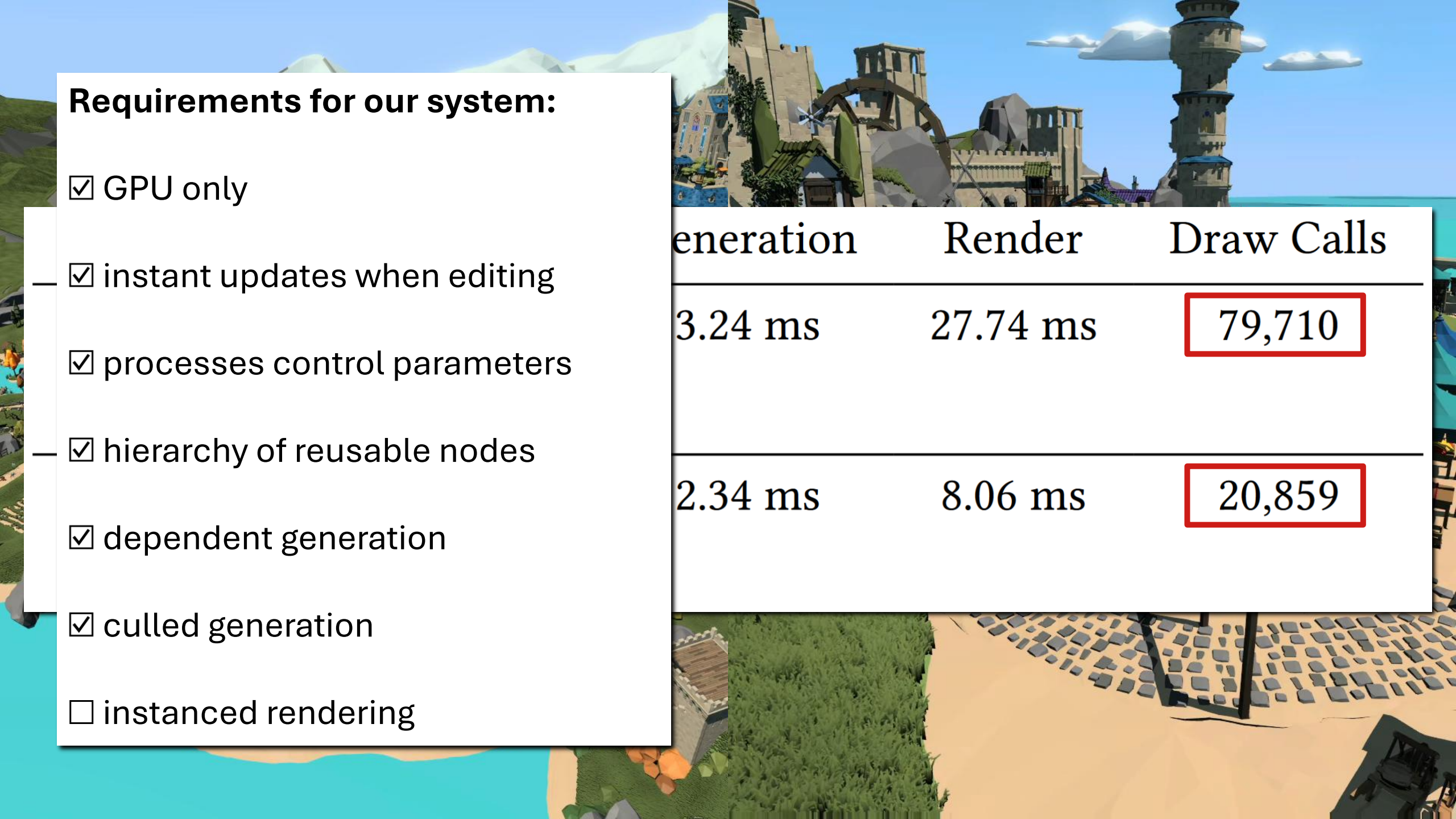
- ☒ GPU only
- ☒ instant updates when editing
- ☒ processes control parameters
- ☒ hierarchy of reusable nodes
- ☒ dependent generation
- ☒ culled generation



eneration	Render	Draw Calls
3.24 ms	27.74 ms	79,710
2.34 ms	8.06 ms	20,859

Requirements for our system:

- ☒ GPU only
- ☒ instant updates when editing
- ☒ processes control parameters
- ☒ hierarchy of reusable nodes
- ☒ dependent generation
- ☒ culled generation
- ☐ instanced rendering



eneration	Render	Draw Calls
3.24 ms	27.74 ms	79,710
2.34 ms	8.06 ms	20,859



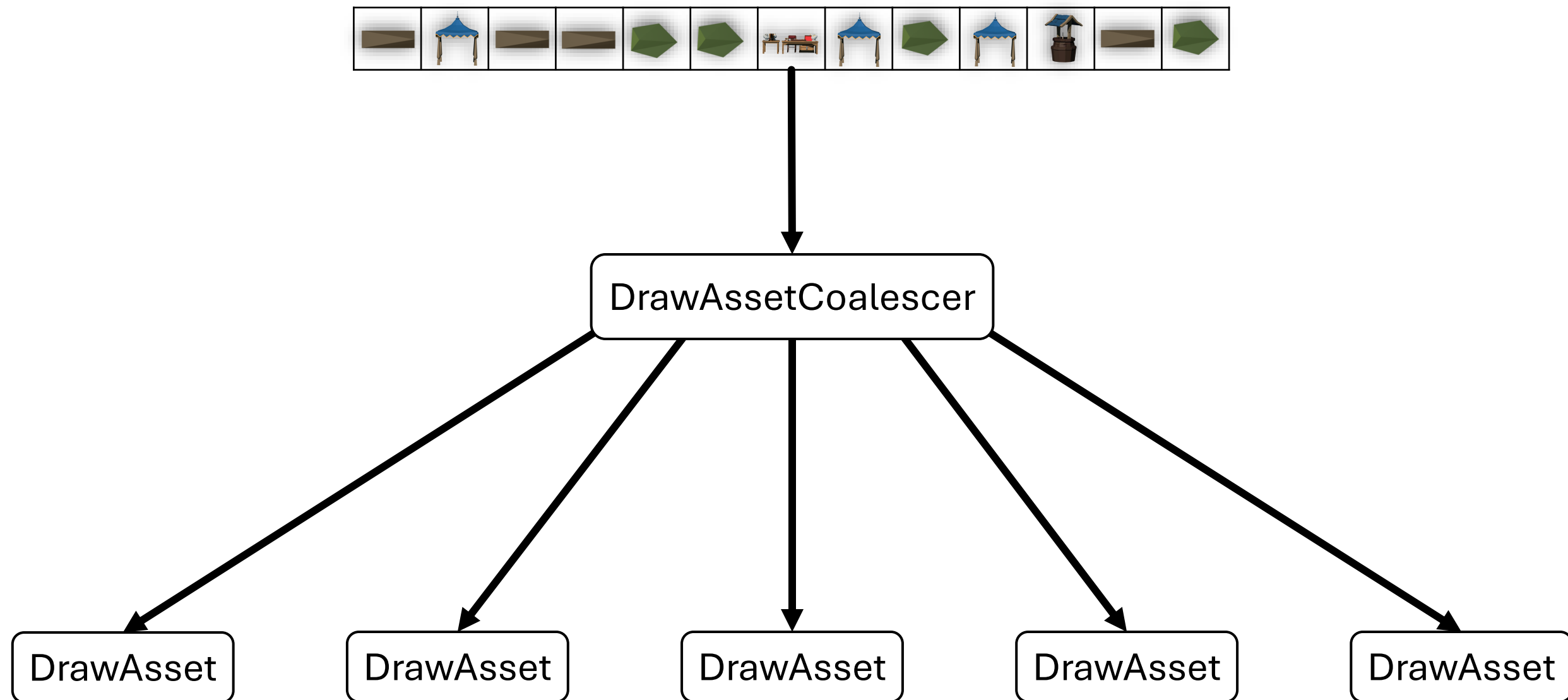
DrawAssetCoalescer

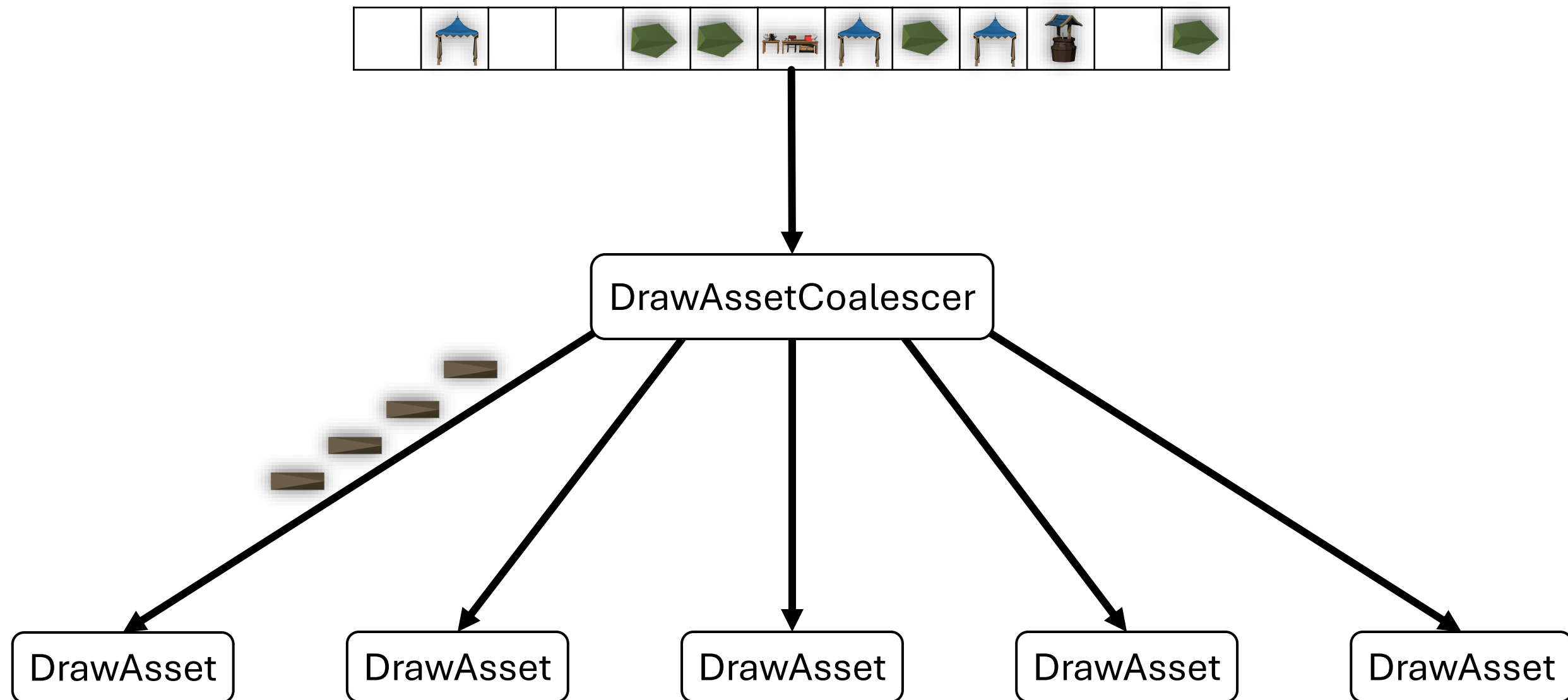
```
[Shader("node")]
[NodeLaunch("coalescing")]
[NumThreads(32, 1, 1)]
void DrawAssetCoalescer(
    [MaxRecords(256)]
    GroupNodeInputRecords<DrawAssetRecord> input,
    [MaxRecords(256)]
    NodeOutput<MeshNodeRecord> DrawAsset
){
```

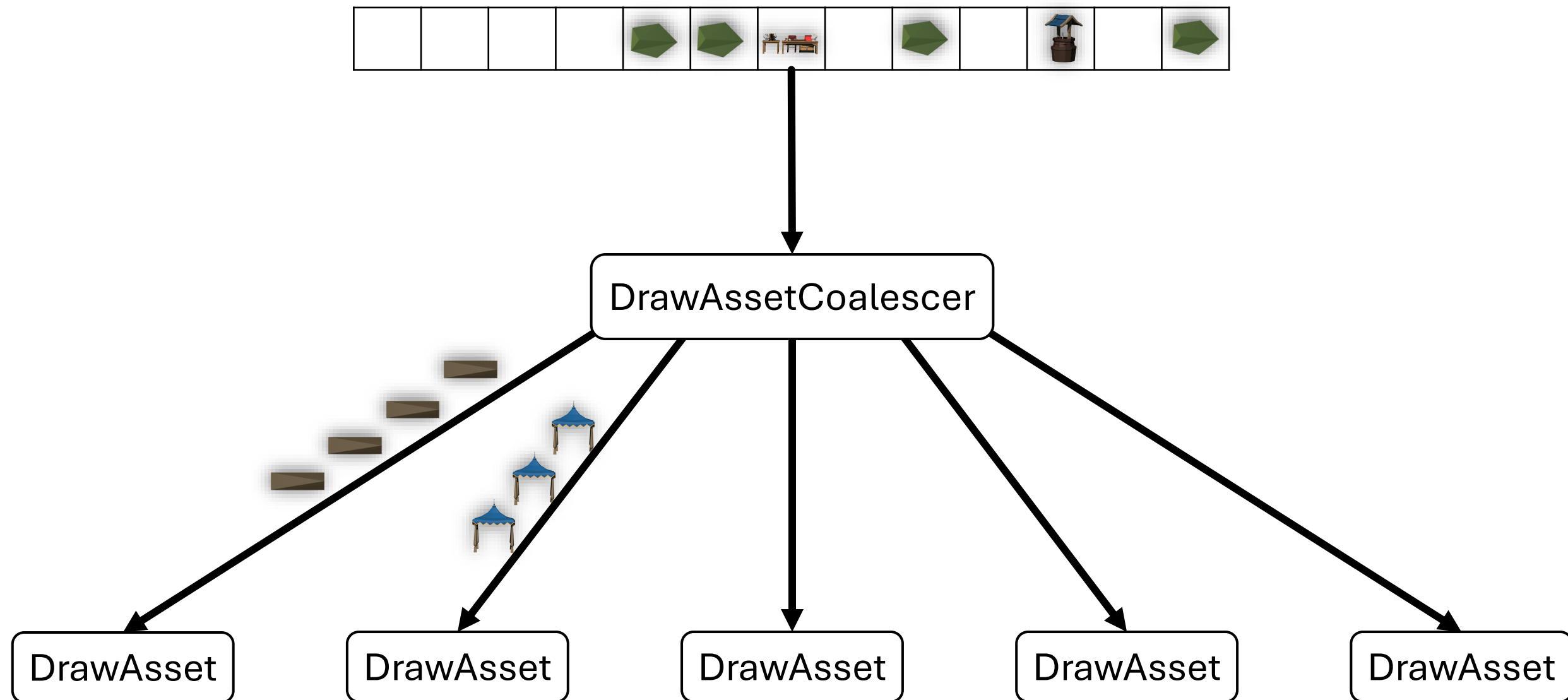


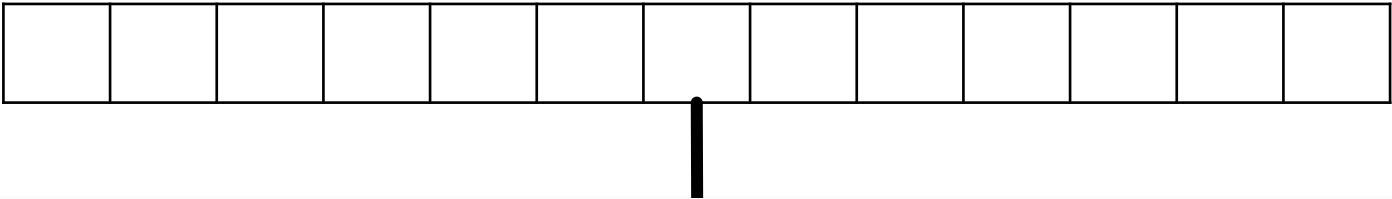
DrawAssetCoalescer

```
[Shader("node")]
[NodeLaunch("coalescing")]
[NumThreads(32, 1, 1)]
void DrawAssetCoalescer(
    [MaxRecords(256)]
    GroupNodeInputRecords<DrawAssetRecord> input,
    [MaxRecords(256)]
    NodeOutput<MeshNodeRecord> DrawAsset
){
```

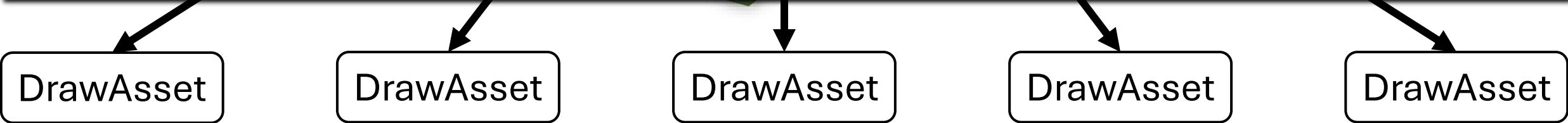



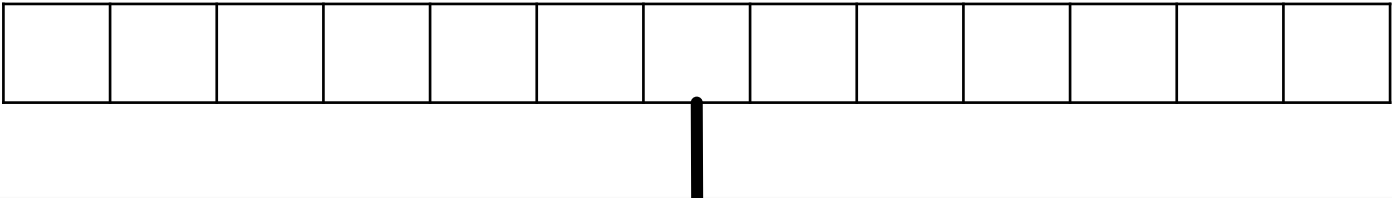




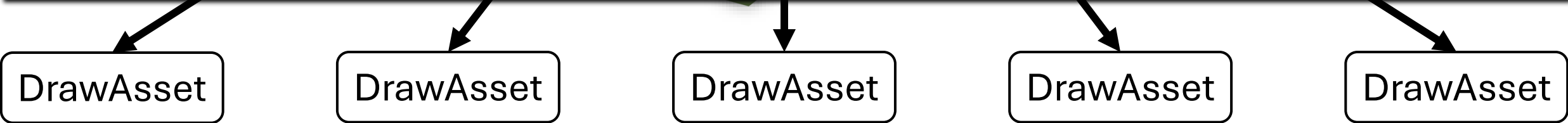


View	Mode	Generation	Render	Draw Calls
Overview 79,710 / 79,710	Baseline	3.24 ms	27.74 ms	79,710
Market 20,859 / 24,068	Baseline	2.34 ms	8.06 ms	20,859





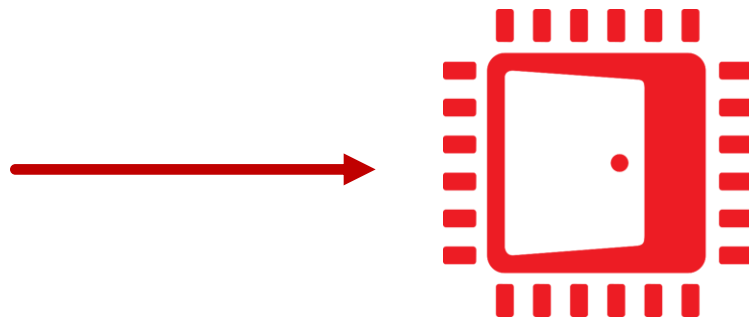
View	Mode	Generation	Render	Draw Calls
Overview 79,710 / 79,710	Baseline	3.24 ms	27.74 ms	79,710
	Coalescing	3.14 ms	0.62 ms	965
Market 20,859 / 24,068	Baseline	2.34 ms	8.06 ms	20,859
	Coalescing	2.29 ms	0.51 ms	493



Requirements for our system:

- ☒ GPU only
- ☒ instant updates when editing
- ☒ processes control parameters
- ☒ hierarchy of reusable nodes
- ☒ dependent generation
- ☒ culled generation
- ☒ instanced rendering

source code



AMD GPUOpen

Procedural grass rendering



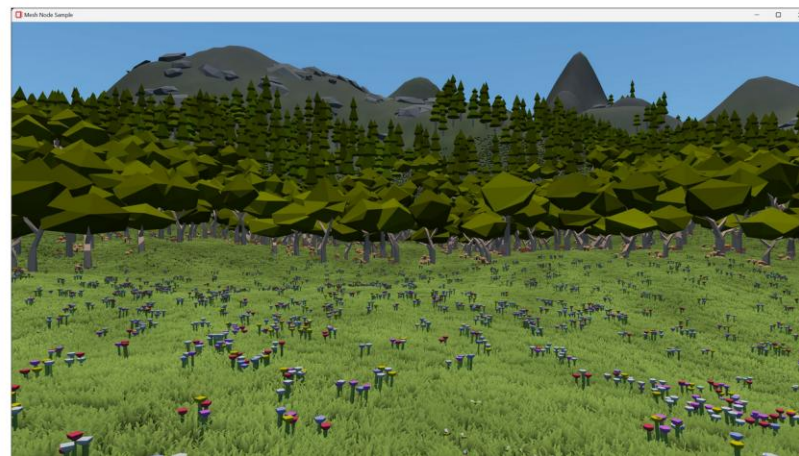
Introduction

Detailed vegetation plays an important part in improving immersion in video games. One part of this vegetation is grass. In this blog post, we want to utilize mesh shaders to generate patches of grass on the GPU. To do this, we took inspiration from Jahrman's and Wimmer's 2017 I3D Paper [Responsive real-time grass rendering for general 3D scenes](#) who utilize tessellation shaders to subdivide predefined blades of grass. This comes with the benefit that additional detail can be generated without needing to store it explicitly. We take this concept further by using one mesh shader thread group to render a whole patch of grass. While in this post we create a stylized meadow to keep things simple, we are confident that our technique can be applied to more realistic scenes, as well.

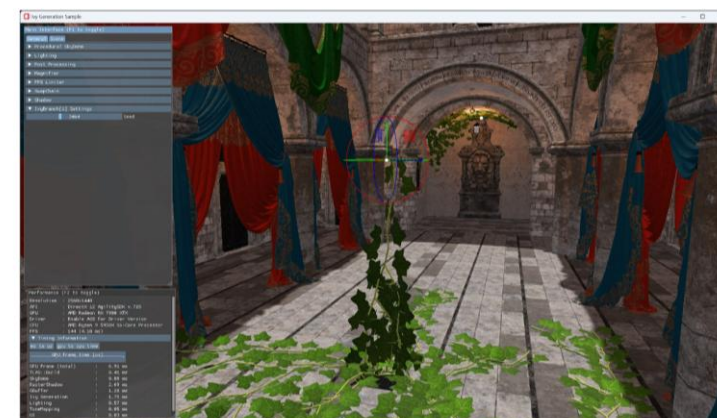
Procedural generation

So far, we've covered how you can add mesh nodes to an existing work graphs application, and how the HelloMeshNodes sample uses them to draw the Koch snowflake fractal. Further, we've discussed some best practices and other tricks to get the most out of mesh nodes. In this section, we're going to take a look at how the procedural mesh node sample uses work graphs and mesh nodes to generate and render a fully procedural world.

The procedural generation mesh node sample shows fully GPU-driven rendering at work. The entire world is fully procedurally generated every frame and rendered entirely on the GPU through multiple different mesh nodes. Many of the procedural effects, e.g., the procedural grass, bees and butterflies, are based on the [GDC 2024 mesh nodes demo](#). You can find the full source code for the sample along with pre-built binaries [here](#) and you can learn more about the procedural effects of the GDC 2024 demo in our [upcoming paper](#), which will be published at [HPG 2024](#).



Work Graph Ivy Generation Sample



Building the sample

Prerequisites

To build the Work Graphs Ivy Generation Sample, you must first install the following tools:

- [CMake 3.17](#)
- [Visual Studio 2019](#)

